
中图分类号: TP391

UDC 分类号: 627

基于本体的多架构模型统一表达方法研究

作者姓名	王宏伟
学院名称	机械与车辆学院
指导教师	王国新
答辩委员会主席	薛庆
申请学位	硕士研究生
学科专业	工业工程
学位授予单位	北京理工大学
论文答辩日期	2020 年 6 月

Research on unified expression method of multi architecture model based on Ontology

Candidate Name: Hongwei Wang

School or Department: School of Mechanical Engineering

Faculty Mentor: Prof. Guoxin Wang

Chair, Thesis Committee: Prof. Qing Xue

Degree Applied: Master of Science

Major: Industrial Engineering

Degree by: Beijing Institute of Technology

The Date of Defence: June, 2020

基于本体的多架构模型统一表达方法研究

北京理工大学

研究成果声明

本人郑重声明：所提交的学位论文是我本人在指导教师的指导下进行的研究工作获得的研究成果。尽我所知，文中除特别标注和致谢的地方外，学位论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京理工大学或其它教育机构的学位或证书所使用过的材料。与我一同工作的合作者对此研究工作所做的任何贡献均已在学位论文中作了明确的说明并表示了谢意。

特此申明。

签名：

日期：

关于学位论文使用权的说明

本人完全了解北京理工大学有关保管、使用学位论文的规定，其中包括：①学校有权保管、并向有关部门送交学位论文的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存学位论文；③学校可允许学位论文被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换学位论文；⑤学校可以公布学位论文的全部或部分内容（保密学位论文在解密后遵守此规定）。

签 名：

日期：

导师签名：

日期：

基于本体的多架构模型统一表达方法研究

摘 要

随着系统复杂度的增加，系统设计与组织效率显著降低，因此工业界广泛采用系统工程方法对系统全生命周期中需求、设计、分析、验证和确认过程进行形式化表达。目前设计师在实现多学科、跨领域交叉融合的复杂系统设计过程中往往采用通用建模语言 SysML、UML 或特定域建模语言如 EAST-ADL、DoDAF 等进行异构模型的设计与开发，在此基础上采用基于 SysML 的 XMI 标准进行不同模型之间的数据交互。然而，多种建模语言采用不同语法和语义进行设计，其不一致性增大了异构模型统一表述的难度；此外，设计上下游不同设计人员采用不同建模工具实现特定模型的开发增加了模型底层数据的交互难度，而 XMI 标准仅对 SysML 提供相关的接口，缺少普适性。

综述，目前业务人员进行系统工程实践时需要一种统一建模方法以实现不同建模语言之间的打通，从而融合不同建模语言的优点，提高设计效率；同时基于上述建模方法设计一种数据交互方法实现系统设计周期中异构模型信息的统一描述及互操作。

本文在 GOPPRR 方法的基础上设计一套以 MBSE 为核心领域的多架构建模方法，可以在此基础上按照需求、功能、逻辑、架构的映射规律实现复杂系统全生命周期中所涉及的所有架构建模；并在此基础上提出一套基于多架构建模的本体框架实现异构模型数据的一致性表达，并通过 SQWRL 及 SPARQL 语言对该框架进行理论验证，通过基于多种建模语言的模型数据与本体框架的双向转化及互操作对该框架进行工程验证。

关键词：多架构建模，MBSE，RFLP 设计流程，本体设计

Research on unified expression method of multi architecture model based on Ontology

ABSTRACT

With the increase of system complexity, the efficiency of system design and organization is significantly reduced, so system engineering method is widely used in industry to formalize the requirements, design, analysis, verification and validation process in the whole life cycle of the system. At present, designers often use general modeling languages such as SysML, UML or domain specific modeling languages such as EAST-ADL, DoDAF to design and develop heterogeneous models in the design process of complex systems with multi-disciplinary and multi domain cross fusion. On this basis, XMI standards based on SysML are used to carry out data interaction between different models. However, many modeling languages are designed with different syntax and semantics, and their inconsistency increases the difficulty of unified representation of heterogeneous models; in addition, different designers at upstream and downstream of design use different modeling tools to develop specific models, which increases the difficulty of interaction of underlying data of models, while XMI standard only provides relevant interfaces for SysML, lacking universality.

In summary, at present, designers need a unified modeling method to realize the communication between different modeling languages, so as to integrate the advantages of different modeling languages and improve the design efficiency; at the same time, based on the above modeling methods, a data interaction method is designed to realize the unified description and interoperability of heterogeneous model information in the system design cycle.

Based on the GOPPRR method, this paper designs a set of Multi-Architecture-Modeling method with MBSE as the core domain, which can realize all the architecture modeling involved in the whole life cycle of

complex system according to the mapping law of requirements, functions, logic and architecture, and then proposes a set of ontology framework based on Multi-Architecture-Modeling to realize the consistent expression of heterogeneous model data, The framework is verified theoretically by SQWRL and SPAQL. The framework is verified by Bidirectional Transformation and interoperability between model data and ontology framework based on multiple modeling languages.

KEY WORDS: Multi-Architecture-Modeling method, MBSE, RFLP design, ontology design

目 录

第 1 章 绪论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	2
1.2.1 MBSE 关键技术	2
1.2.2 基于本体的 MBSE 建模信息交互研究	7
1.2.3 本文拟解决的问题及研究内容	9
第 2 章 多架构建模方法设计	12
2.1 引言	12
2.2 基于元建模方法的模型的构建过程	12
2.2.1 元建模框架	13
2.2.2 建模语言设计方法	14
2.3 基于 GOPPRRE 的元建模设计	15
2.3.1 GOPPRRE 元元模型	15
2.3.2 多架构建模	17
2.3.3 元建模方法对比	20
2.4 小节	22
第 3 章 基于多架构建模方法的系统设计流程	23
3.1 引言	23
3.2 基于系统工程的正向设计	23
3.2.1 需求分析	23
3.2.2 功能分析	26
3.2.3 逻辑分析	27
3.2.4 物理设计	28
3.3 基于 RFLP 的多架构建模流程	29
3.4 本章小结	39
第 4 章 面向多架构建模的本体设计	41

4.1 本体建模与数据交互	41
4.1.1 本体基本概念	41
4.1.2 基于本体的 MBSE 数据交互方法	42
4.2 本体设计方法	43
4.2.1 M3 层本体设计方法	44
4.2.2 M2 层本体设计方法	49
4.2.3 M1 层本体设计方法	58
4.3 本体接口设计	61
4.4 本章小结	63
第 5 章 本体设计方法验证	64
5.1 引言	64
5.2 归纳法验证	64
5.2.1 SysML 模型库转化	64
5.2.2 多种模型库转化	68
5.3 演绎法验证	71
5.3.1 SPARQL 关键元素查询	71
5.3.2 SQWRL 推理验证	73
5.4 本章小结	75
第 6 章 基于无人平台的多架构设计与本体转化案例	76
总结与展望	105
本文工作总结	105
研究工作展望	105
参 考 文 献	107
攻读学位期间发表论文与研究成果清单	111
致谢	112

第 1 章 绪论

1.1 研究背景及意义

随着社会的发展进步，系统设计和组织过程中的复杂度也大大提高，需要考虑的领域知识也往往来自于多个学科^[1]。这种复杂的研究对象被称为系统，它是由不同元素组成的集合，但是这些元素相互作用的结果并不是元素之间的简单叠加，而是在其基础上涌现出更多更复杂的功能^[2]。设计人员为了降低系统设计的复杂度，提高设计效率，必须改变以往“多学科独立设计，依靠实物试验发现问题”的传统模式，用一套科学有效的方法将人员，技术，物质，资金等组织起来。这种新的设计方法就是系统工程（System Engineering, SE）^[3]，作为一种跨学科的设计方法，它可以帮助设计人员理解系统内部的复杂性，并协调各利益相关人员，从而确保在系统全生命周期中均可满足用户的需求。同时，在系统设计过程中涉及到大量的数据，每个设计环节都可以看作是数据的传递与转换。因此，数据的表示在系统设计过程中显得尤为重要。

在系统工程发展初期，设计人员都采用文档的形式来记录和保存系统设计过程中的信息，但是随着信息技术的快速发展，系统复杂度及规模不断提高，文档的规模也不断增加，而且这些数据之间存在一定的关联关系，当某一文档中的数据变化时，其他文档中的数据难以同步进行更改^[4]。因此，这种基于文档的系统工程方法逐渐被基于模型的系统工程（Model-based System Engineering, MBSE）取代^[5]。

MBSE 定义了对系统工程活动建模方法的应用范式，支持需求、设计、分析、验证和确认等全生命周期过程。它将模型作为整个系统的核心，从系统开发的需求分析阶段到最后的系统验收阶段都通过不同模型进行迭代描述，最终完整最终的系统设计^[6]。在系统工程中，模型可以定义为系统的抽象表达，旨在理解和解释系统的某些侧面，例如，采用需求图描述系统设计过程中不同利益相关人对系统的期望。与传统的系统工程相比，MBSE 有以下三个优势^[7]：

（1）知识表示的无二义性。基于文档的表述方法基于自然语言，模型则是规定的形式化表示方法，因此采用模型比文档描述更加清楚；

（2）便于数据的交流传播。系统设计过程中涉及到大量数据，这些数据往往用于表示不同领域的知识，采用模型可以在不同的设计人员之间建立无二义的交流规则，从而确保所有设计人员对于同一概念有相同的理解；

(3) 支持数据的转换。基于模型的系统工程可以基于定义的转化规则实现数据的转换, 如 MBSE 支持从设计模型到仿真模型的转换, 从而实现设计验证的自动化。

MBSE 可以提高系统设计的效率, 但是针对不同领域设计人员往往采用不同的建模语言来辅助建模, 如通用建模语言 SysML^[8] (Systems Modeling Language)、架构分析设计语言 AADL^[9]、业务流程建模语言 BPMN^[10]等。这些建模语言有不同的使用条件, 如 SysML 在嵌入式系统设计中表现较好, BPMN 则更适合描述系统的业务流程。但是如果系统较为复杂, 需要采用不同的建模语言描述同一项目时, 由于建模语言之间的语法和语义规则不同, 因此无法同时采用多种建模语言支持系统设计。

因此本文力图寻找一种通用方法解决 MBSE 中的模型构建问题及模型信息的交互表达问题: 将全生命周期中的模型在更抽象的层次上进行描述, 以实现模型的形式化表达; 为构建好的种类繁杂, 数量繁多的模型用一种简单的标准化方法进行描述, 以实现模型信息的高效传递。

基于模型的系统工程模型构建方法及模型信息表示方法的研究, 其意义在于:

- 1、在目前元建模方法的基础上设计一种通用建模方法, 为 MBSE 的模型构建提供一种更底层的方法, 基于这种建模方法可以实现现有 MBSE 建模语言的设计。同时, 可以为系统全生命周期中的模型提供统一的构建方法, 可以便于各利益相关人了解全部的模型信息。

- 2、提出一种基于本体的信息表示方法作为模型的底层数据保存方式, 与现有研究相比, 该方法可以用更简单的形式表示基于上述建模方法的模型信息, 并且本体的标准结构可以帮助设计师完成该领域知识的设计, 简化设计流程。

- 3、基于本研究可以开发 MBSE 工具来协助实现全生命周期的建模工作, 不需要使用不同的建模工具分别完成建模任务, 同时也不需要学习多种模型交互语言, 大大提高了设计效率。

1.2 国内外研究现状

1.2.1 MBSE 关键技术

在基于模型的系统工程实践过程中, 设计人员往往需要关注其三大支柱: 建模语言、建模方法论、建模工具。其中, 建模语言是用于描述模型的语言, 基于建模语言可以快速地完成系统建模; 建模方法论是设计人员如何基于建模语言组织

和设计模型，针对系统设计的流程作出一系列规范；建模工具则是系统工程实施的技术支持。此外，建模语言的构造方法称为元建模方法，本节 MBSE 关键技术针对 MBSE 建模语言、方法论和元建模方法进行现状研究。

1.2.1.1 建模语言

MBSE 通过建模语言形式化地描述模型，形式化指可以将模型用计算机可以理解的方式进行表达^[11]。目前建模语言可以大致分为两类：完全形式化建模语言如 Modelica，以及半形式化语言如 SysML。本文针对半形式化建模语言进行阐述，这些建模语言往往可以采用图形进行表示，如在 SysML 中采用“需求模块”代表一个关键元素。这种图形化的建模语言表达形式清晰明了，可以减少歧义性，不需要专业的语言知识，也可以理解模型表达的内容。目前，主要应用的 MBSE 建模语言有以下几种：

（1）统一建模语言 UML^[12]：UML（unified modeling language）是软件工程的标准建模语言，支持模型化和软件系统开发的图形化表示。该语言定义了九种类型的图，从不同的侧面对系统进行描述，但是由于 UML 包含的大部分概念，如类、对象、属性、操作等，都来自于面向对象编程语言，而不是来自系统的问题域，因此在工程实际中，UML 的概念与问题域中的概念存在一定偏差^[13]。

（2）系统建模语言 SysML^[8]：SysML 是对象管理组织 OMG 在 UML2.0 的基础上进行重用和扩展形成的。SysML 语言包括结构图、需求图和行为图的九种基本图形。通过这九种基本图可以满足产品设计需求、开发流程、性能验证等全生命周期的开发。SysML 也是当前 MBSE 方法中最常用的一种建模语言，但是与 UML 类似，SysML 的语法和语义设计和工程实际中的问题域并不相同，因此不同领域需要不同的建模语言。一般来说，设计人员根据领域知识设计建模语言及建模环境^{[13][14]}，这种建模语言被称为特定域建模语言或领域建模语言（domain specific language, DSL）。以下的建模语言都属于领域建模语言。

（3）业务流程建模语言 BPMN^[10]：BPMN 是 BPMI 开发的一套标准化的业务流程建模语言。它定义了业务流程图，可以用于设计业务流程操作的图形化模型，该模型定义了业务包含的活动及这些活动操作顺序的流程控制器，为复杂系统提供流程建模的支持。BPMN 可以提供整个系统中全部利益相关人都可以理解的一套语义，但是适用范围仅限在业务流程建模中。

（4）架构分析和设计语言 AADL^[15]：AADL 是一种标准化架构描述语言，用来对嵌入式实时系统的软件和硬件建模，其架构模型既可以用作设计文档，还可

以用于分析或代码生成。这种语言在代码生成及文档设计上有较好的应用，但是在需求分析，架构设计等领域设计效率不高。

MBSE 方法中应用的建模语言还有很多，如在 UML 和 SysML 基础上开发的武器装备体系统一建模平台 UPDM^[16]、以功能分解的方式显示多个功能层级的功能流图 FFBDs^[17]等，每种语言都有其侧重点，在其各自领域的业务建模做出卓越贡献。

1.2.1.2 建模方法论

MBSE 方法论可以用于指导设计人员如何建模，不同领域有不同的工业标准和规范。因此，针对系统问题，建模规范为 MBSE 的实现提供了规范的工作流程，相应的逻辑也为模型的实施与验证提供借鉴依据^[18]。目前较成熟建模规范有下面几种：

(1) Harmony-SE 方法^[19]：Harmony-SE 方法是 IBM 推出的建模工具 Rational Rhapsody 中的一种方法，该方法可以反映经典的 V 型系统生命周期模型，将系统设计划分为系统需求分析、功能分析、综合设计三个阶段，首先分析系统利益相关人全部需求，然后将其转化为系统的功能描述，并确保系统功能可以覆盖全部需求，最后分析系统的物理架构和设计方案。

(2) OOSEM 方法^[20]：OOSEM 是面向对象的系统工程方法 (Object-Oriented System Engineering Method, OOSEM)，该方法的实现过程包括分析需求、定义逻辑体系结构、合成候选分配架构、优化和评估可选方案、验证和确认系统等开发活动^[21]。最初 OOSEM 采用 UML 来表示各种建模元素，在 2006 年 SysML 出现后，相关支持工具得到很大改进，Fridenthal 等在《A Practical Guide to SysML》中介绍和总结了 OOSEM 方法，自此 OOSEM 与 SysML 相结合的应用得到广泛采用，并逐渐成为 MBSE 的一种最佳实践^[8]。

(3) Arcadia 方法^[22]：Arcadia 是法国 Thales 公司推出的建模方法，Arcadia 方法遵循的建模流程为运行分析、系统分析、逻辑架构、物理架构、产品结构分解。相比 OOSEM 方法而言，Arcadia 方法更加适用于系统架构设计，因此目前工业界基于 Arcadia 方法开始有更多的应用和推广。

(4) Vitech MBSE 方法^[23]：Vitech MBSE 是由 Vitech 公司推出的建模方法，该方法主要基于四个主要的系统工程活动：源需求分析、系统行为分析、架构设计、验证与确认。此外，该方法是“洋葱模型”的经典应用，通过迭代每一层的主要系统工程活动，可以有效降低系统开发的风险，因为每一层上的完整解决方案

可以用于早期的评审和确认^[24]。

除以上 MBSE 方法论外，还有如下几种方法论也被设计人员广泛采用：IBM 着重于商业建模的 RUP-SE 方法论^[25]；Dori 提出的将事物表示为对象或过程的对象过程方法（OPM）^[26]；喷气推进实验室（JPL）开发的状态分析法采用基于模型或基于状态的控制架构^[27]等。

1.2.1.3 元建模方法

设计人员根据领域知识设计领域建模语言，但是为每一个建模语言都手工开发建模工具则成本过高，因此设计人员需要某种方法降低建模工具的开发成本，元建模方法就是解决方案之一，其核心是领域专家根据元元模型设计建模语言，再通过代码生成技术生成支持该领域建模语言的工具。大量的工程实践证明，基于元模型的领域建模比基于 UML 的建模效率高 10 倍^[13]，同时采用 DSL 方法也便于进行建模工具的维护^[28]。基于元建模的领域建模效率提高主要是基于以下两个原因^[29]：

（1）领域建模语言贴近工程实践的问题域，对领域内的专家而言，领域建模语言的语法和语义更容易被接受和采纳。

（2）领域建模语言被限定于特定领域，因此从领域模型生成到程序代码较为简单。

因此，元建模方法可以将在元元模型的基础上构建领域建模语言，Bapty 等人提出基于 M0-M3 的建模框架来实现特定领域的建模^[30]。其中，模型是对现实世界的描述，元模型则是对模型的抽象描述，元元模型则是对元模型的进一步抽象表达，建模框架将在本文第二章中进行详细介绍。目前元元模型大致可以分为以下几类：

（1）基于 E-R 图的元元模型。早期的元建模方法基于 E-R 图实现领域建模语言的设计，现在逐渐被面向对象的方法所取代。MetaView^[31]基于 E-R 图扩展提出 EARA/GE(entity aggregate role attribute with graphical extension),并将其作为元元模型；ToolBuilder^[32]在 E-R 图的基础上扩展 ER 模型 EER(extended entity relationship)；AToM3^[33]则直接使用 E-R 模型进行建模。总体而言，基于 E-R 扩展的元元模型,其优点是简单、直观。

（2）基于 MOF 和 UML 类图的元元模型。随着面向对象的建模方法 OOSEM 称为 MBSE 实施的主流方法，设计人员也逐渐开始用面向对象的方法设计元模型，其中最具代表性的就是 MOF(meta-object facility)^[34]和 UML 类图(UML class

diagram)。目前基于 MOF 和 UML 类图的元元模型称为元建模方法的主流，其中，MOMO^[35]直接支持 MOF，MEG^[36]和 GME^[37]则是基于 UML 类图的代表。相比基于 E-R 图的元元模型而言，基于 MOF 和 UML 类图的元元模型表达能力更强，同时，采用 UML 的子集作为元元模型可以大大降低涉及人员的学习成本。但是，MOF 主要考虑元模型的构造及数据集成，在建模工具上的表现较差。例如，MOF 无法表示建模语言的语义信息，同时也没有考虑图、点、边的区别，而是都将其看作对象处理。

(3) 基于元建模工具的元元模型。为了将元建模方法和建模工具紧密结合，这类型的元元模型一般都是在开发元建模工具时根据实际需求设计的。其中最具代表性的是 DOME^[38]和 MetaEdit+^[39]。MetaEdit+提出的 GOPPRR (graph-object-point-property-relationship-role)通过六种元元模型构建领域建模语言，对象(object)和属性(property)的概念与 E-R 模型及 UML 中的概念类似；图(graph)对应 UML 中的图(Diagram)，关系(relationship)的概念则与 DOME 中的 Connector 类似。Kelly 对 MetaEdit+中采用的 GOPPRR 方法做了详细的说明^[40]，其结构图如图 1.1 所示。

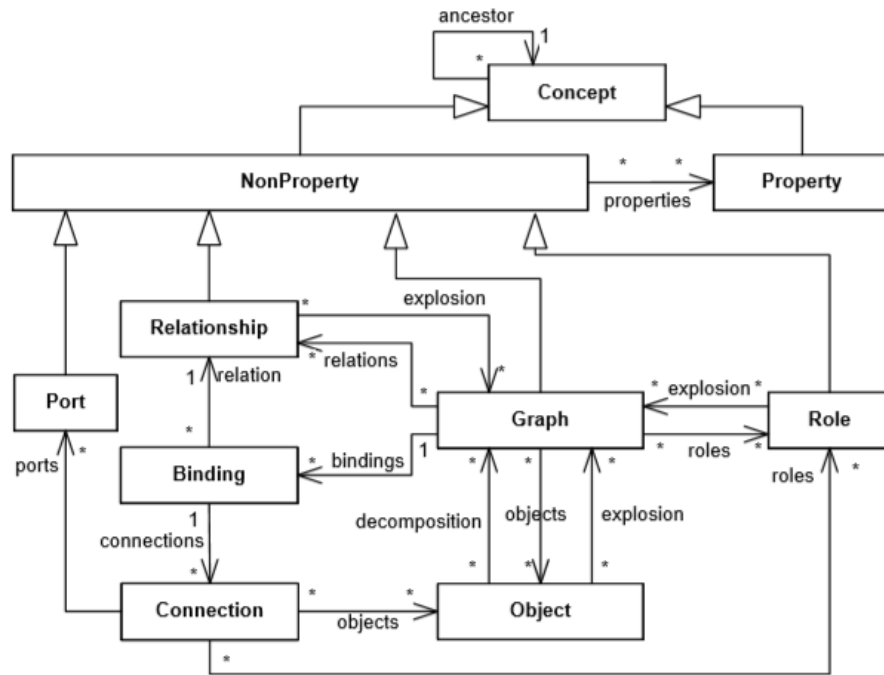


图 1.1 GOPPRR 方法

通过上述文献综述可以看出，目前 MBSE 实施过程已经有非常成熟的技术和知识体系，设计人员可以根据研究问题选择建模语言，根据相应的方法论进行建

模设计。但是随着系统复杂度的提高和组织难度的增大，不同领域的设计人员参与系统设计时会出现以下问题：

(1) 系统设计复杂度逐渐提高，设计难度的提高，设计设计人员可能需要同时基于多种建模语言进行设计，而上述建模语言中的语法和语义规则都不一致，因此无法同时在一个项目中应用多种建模语言；

(2) 尽管方法论并未限制必须使用某种建模语言，但是由于支持它们的工具限制，不同的方法论对应着不同的建模语言。例如 Harmony-SE、OOSEM 通常使用标准建模语言 SysML 和 UML 进行建模；Vitech MBSE 采用功能流图进行功能、行为分析。当设计人员采用多种建模语言时难以判断选择哪种方法论可以提高设计效率。

(3) 采用元建模方法对建模语言进行表达时，基于 E-R 图的元元模型及基于 MOF 和 UML 类图的元元模型都无法很好地与建模工具集成起来，MetaEdit+提出的 GOPPRR 建模方法可以与建模工具较好地集成，但是将连接（connection）和绑定（binding）作为实体又无法准确表达图中的连接关系。

因此，解决上述问题后可以一致性描述多种建模语言，并在此基础上设计一种正向设计的流程，满足需求、功能、逻辑、物理的映射规则，从系统、子系统、部件等多个粒度上对系统进行分析，每层的设计方案满足上一层的系统需求，从而提高设计效率，减少系统组织中造成的人力财力浪费。

1.2.2 基于本体的 MBSE 建模信息交互研究

系统模型是 MBSE 中的主要部分，它由系统建模工具支持。然而，系统全生命周期中涉及到不同的模型，例如硬件设计、软件设计、验证、工程分析和仿真。将系统设计模型与各种类型的模型相互连接，实现模型的自动转换或者对数据的互操作，是 MBSE 方法走向成熟的必要条件。在 MBSE 领域中，存在一些规范的数据交换标准，同时可以在一些建模工具之间进行模型转换。

1.2.2.1 数据交换标准

数据交换标准可以在不同工具之间，对不同形式的模型进行一致性描述。目前工业界中广泛认可并采用的标准有以下三种：

(1) XMI^[41]。XMI（XML Metadata Interchange）是 OMG 组织定义的标准，该标准可以通过可扩展标记语言 XML 来进行数据交换。XMI 可以在不同工具之

间交换 UML 和 SysML 格式模型数据，但并不能交换其他格式的模型数据。

(2) AP233^[42]。AP233 是 ISO 的标准交换协议。它与 CAD，结构、电子和工程分析领域的标准一致。AP233 与 SysML 具有相同的系统工程领域概念，可以作为在 SysML 工具间交换 SysML 模型的中立格式。

(3) FMI^[43]。FMI (Functional Mockup Interface) 是一种用于支持模型转换和联合仿真的标准接口。这种接口可以将不同领域的模型按照统一标准转化为统一格式的组件 (FMU)，从而将不同领域的模型集成起来，再将其在一个执行平台上进行不同学科之间的联合仿真。最新的 FMI 版本是 FMI2.0，已经可以支持 40 多种工具。

综上所述，目前较为认可的数据转化标准有上述三种，但是 XMI 和 AP233 都局限于 UML 或 SysML 工具之间的转化，对于其他模型并无转化能力，而 FMI 主要针对模型的仿真，对于跨语言之间的数据转换效果不佳。同时，这些方法都有不同程度上的信息丢失问题^[44]。

1.2.2.2 模型转换工具

目前已有多种常见的建模工具和仿真工具之间可以实现不同类型模型之间的转换。例如，基于 SysML 构建的模型可以直接转换为 Simulink 模型^[45]，直接打通架构模型到仿真模型，提高设计效率；类似地，基于 SysML 构建的模型可以直接转换为 STK 模型^[46]、VHDL 模型^{[47][48]}、SystemC 模型^{[49][50]}。此外，SysML 和 Modelica 之间的转换则是在 INCOSE 和佐治亚理工学院的研究下不断取得进展^[51]，主要是通过定义多种转换语言来实现模型的转换^[52]，如 QVT、ATL、ETL 等。可以看出，目前的模型转换工具都是基于 SysML 进行的，因此可以将系统模型作为中心存储库，将其直接转化为各种类型的模型。此外，很多工具也同时提供应用程序接口 (API)，允许工具之间进行有限度的模型转换和数据交互。但是这些接口只能适用于特定工具之间的转换，适用性较差。

1.2.2.3 基于本体的数据交互方法

本体是一种基于 RDF 格式的知识表达方式^[53]，由类、实例、属性、关系、函数、规则、公理等组成，可以对特定领域中相关概念及其相互关系的形式化表达。一般来说，本体在系统工程领域多用于构建领域本体，并在此基础上实现领域内知识的推理，但在本文中本体作为一种数据框架，用于实现基于多架构建模方法的工具之间的数据交互。van 等人基于国标提出一种本体构建方法^[54]，但是在处

理中使用了数据库，并且表示方法采取 RDF 格式，使得模型信息的表达不够标准，并在推理能力上有所欠缺。Tobias Walter 等人设计了一个允许使用本体技术来描述和推理领域建模语言的框架^[55]，该方法使用 owl 格式进行信息表示，但是对语言有所限制，不能实现所有语言的统一表达。Pance Panov 等人设计了一种用于表示数据类型的通用本体^[56]，这种本体表示方法可以较好地表示模型信息，但是操作复杂，且使用数据库与本体相连，增加了系统负荷。

1.2.3 本文拟解决的问题及研究内容

一个完整的系统开发过程可以分为分析设计、模型实现、集成验证和系统确认等阶段，如图 1.2 所示的 V 型图，从需求阶段开始进行系统的分析设计，然后通过仿真模型分析验证设计的正确性和合理性，最后进行系统集成产生实物产品。本文针对 V 模型的分析设计阶段，主要解决以下问题：

- （1）当设计人员采用多种建模语言实现需求、功能、逻辑、物理架构的设计与分析时，如何设计元建模方法从而形式化表达全部架构模型？
- （2）采用多种建模语言支持系统设计时如何按照一定的规则实现需求到设计结构的映射？
- （3）基于多架构建模方法构建模型后，如何基于本体实现模型数据的一致性描述？

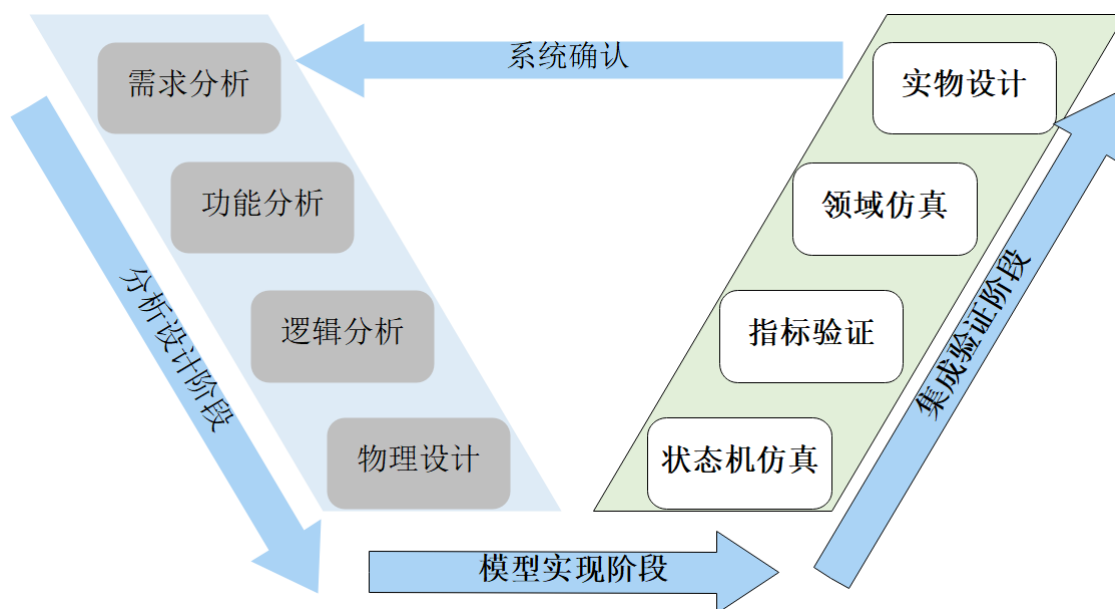


图 1.2 系统设计 V 型图

本文首先基于 GOPPRR 方法设计多架构建模方法，设计人员可以根据领域知

识设计不同的建模语言，也可以将不同建模语言的优势集中到一起以支持系统设计，从而不同领域的设计人员可以对全生命周期中的设计信息有相同的理解，实现信息传递的无歧义性，保证开发效率。然后基于 RFLP 的映射规则实现需求、功能、逻辑和物理架构的设计，最后基于多架构建模方法实现本体框架的设计，实现全生命周期中异构的架构模型信息的一致性表达，并通过本体推理机对本体设计方法进行验证。本文组织结构如图 1.3 所示，其中主要的研究内容分别见第二到五章，具体描述如下：

（1）第二章设计多架构建模方法。首先介绍元建模方法的设计框架，然后在 GOPRR 元建模方法的基础上设计多架构建模方法，该方法可以实现系统全生命周期中所有架构模型的形式化表达。

（2）第三章实现基于 RFLP 的正向设计流程。由 1.2.1 章节中可以看出，基于模型的设计方法论都符合系统的正向设计流程，本文在 Harmony-SE 方法论的基础上，将系统综合细化为系统逻辑分析和物理架构设计两部分。其中，逻辑分析可以提供满足系统功能的多种设计方案，以时序转换的方式描述系统的行为；物理架构设计则是选择最优的设计方案。第三章首先明确需求分析、功能分析、逻辑分析和物理架构设计阶段的出入输出，并对每个阶段的主要活动流程作出定义，然后采用 SysML 语言实现多架构建模方法下的正向设计。

（3）第四章实现基于多架构的本体设计。首先介绍本体建模及本体交互的相关概念，然后将多架构建模方法中的关键元素和本体中的关键元素实现一一映射，该方法可以实现多架构建模下的所有模型的一致性表达，便于不同领域的设计人员实现数据交互。

（4）第五章实现本体设计方法的验证。本节在第四章的基础上实现本体框架的验证，基于多种模型库的转化以验证本体转换的可行性及数据传递的完整性，基于 SPARQL 查询语言和 SQWRL 推理语言对本体框架的逻辑进行验证。

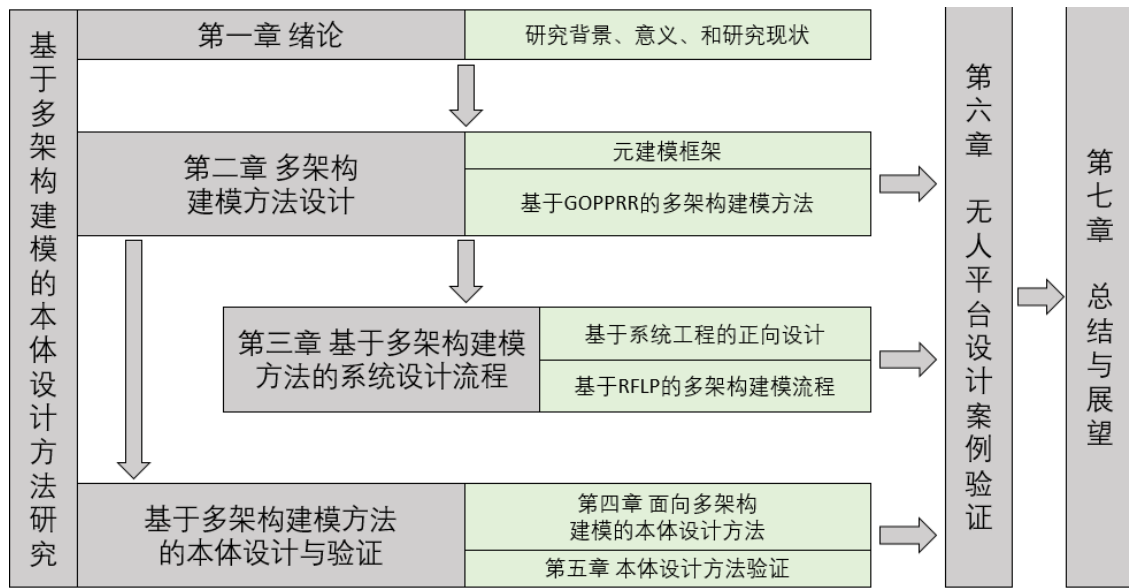


图 1.3 本文组织结构

第 2 章 多架构建模方法设计

2.1 引言

随着系统设计要求增多，复杂度提高，目前复杂系统设计越来越多地需要集成多个领域、多个学科的设计知识。传统的基于文档的系统方法在此过程中会产生大量异构文档，且各文档中数据难以在全生命周期中实现可追溯管理。在此基础上，基于模型的系统工程通过可视化模型来形式化描述系统设计和系统组织过程中的需求、设计、分析、验证和确认过程，解决了传统方法中的交流不畅、数据一致性管理等问题，因此在工业界中被广泛采用。

设计人员广泛地采用统一建模语言 UML 和通用建模语言 SysML 进行系统设计。但是 UML 采用的大部分概念，如类、对象、属性、操作等，都来自计算机领域，实质上是一种面向对象的编程语言，因此在工程问题中与问题域中的概念存在一定偏差，提高了设计人员的理解成本；SysML 在 UML 的基础上作出一定改进，使其概念更加偏向工程问题，但是它并不能适用于所有领域，不同的领域问题可能需要不同的建模语言。然而，不同的建模语言之间语法语义并不一致，为了能在建模工具上同时支持不同建模语言，设计人员需要提取不同语言的共性，对其抽象表达，以提高建模效率。目前基于 E-R 图的元建模方法、基于 MOF 的元建模方法、基于图的元建模方法都是出于这个初衷：基于不同的元元模型支持建模语言的实现，基于建模语言实现领域模型构建。本文中设计的多架构建模方法同样是基于 M3-M0 的设计框架，在 GOPPRR 建模方法的基础上扩展为 GOPPRRE 方法（GOPPRR Extension）以支持系统全生命周期中的架构表达。

本章针对多架构建模方法的实现过程展开论述。其中，设计人员采用建模语言对系统从需求到物理实现的模型都可称为架构模型，多架构建模方法指本文提出的建模方法可以支持多种建模语言实现系统设计。因此，多架构建模方法是一种在系统工程领域构建领域建模语言的方法，可以形式化描述系统设计过程中的全部架构信息。首先介绍元建模方法的框架，以及基于元元模型实现模型表达的过程；其次介绍基于 GOPPRRE 的多架构建模方法的实现过程；最后通过多种元元模型在不同维度的对比说明 GOPPRRE 方法的优势。

2.2 基于元建模方法的模型的构建过程

设计人员在面对不同系统时，由于不同领域的术语、概念、流程和规则不同，

通常需要根据对象系统的需求建立对应的建模语言库及建模环境，这些建模语言统称为特定域建模语言（DSML, domain specific modeling language）。一般而言，设计人员通过建模语言构建系统模型的方法称为建模方法；设计特定域建模语言的方法则称为元建模方法。元建模方法的重点在于如何将建模语言中的共性元素抽象出来，从更高的维度一层层实例化，直到可以完整描述系统某个维度的全部信息。本节主要对元建模方法的框架以及元建模方法的基本步骤进行阐述。

2.2.1 元建模框架

元建模方法是研究建模语言的构建方法，目前都采用 M3-M0 的建模框架以达到更高的抽象层次，从而实现不同模型库的构建。由 M3-M0 层次结构描述的元建模框架如图 2.1 所示。

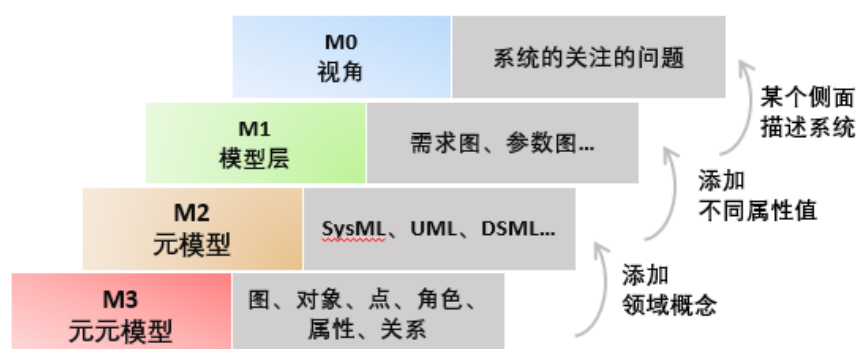


图 2.1 元建模框架

M3-M0 框架将模型的构建过程划分为四个层次：M0 层（系统视角）、M1 层（模型层）、M2 层（元模型层）、M3 层（元元模型层）。如图所示，每一层的元素都是上一层元素的抽象表达，具体含义如下：

（1）M0 表示现实世界中的某种观点，即从某个系统视角来表达系统的关注问题。例如，对某复杂系统进行需求管理及验证时，对应的 M0 的视角为需求图、需求分配矩阵、需求表格等模型需要解决的系统问题。

（2）M1 表示模型层，是现实世界某个视角的抽象描述，如 SysML 中的需求图是系统某种实际需求在建模时的抽象表达。

（3）M2 表示元模型层，表现为具有某种共同特性的模型集合，是模型的元语言。在元模型层可以根据领域知识设计相关领域语言，如 SysML、BPMN、UML 等，用于简化建模流程。

(4) M3 表示元元模型层，代表最高程度的抽象层次，即针对元模型的元语言。一般而言，元元模型都是通过自身描述的，即元元模型之间本身存在引用、连接等关系。例如本文采用的 GOPRR 元建模方法，通过图、对象、点、关系、角色、属性六种元元模型构建元模型时遵循基本的一些原则：对象和点的连接、关系和角色的连接、对象和角色的绑定等。

2.2.2 建模语言设计方法

建模语言的设计方法即元建模方法，元建模方法将元元模型实例化后得到元模型，通过这些元模型可以表述领域内的知识、约束、规则等，最终形成领域建模语言。由于这些建模语言是领域专家根据本领域的概念知识构建而成，因此具备完整的语法和语义描述。参考 Has 对领域建模语言的语法和语义描述^[57]如图 2.2 所示。

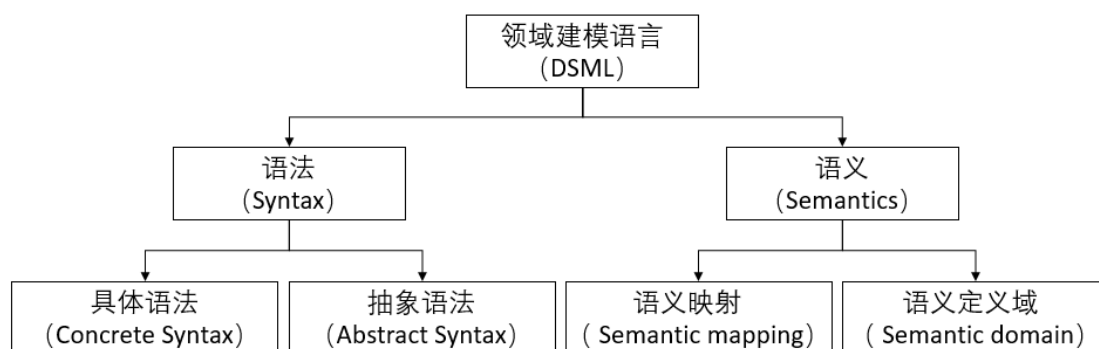


图 2.2 建模语言语法和语义设计

领域建模语言可以用一个五元组来形式化描述，包括具体语法 (C)、抽象语法 (A)、语义定义域 (S)、语法映射 (Mc)、语义映射 (Ms)，可以由以下的公式进行表示： $L=\langle C,A,S,Mc,Ms \rangle$ 。五元组的具体含义如下：

(1) 具体语法：定义了领域语言中元素的表示方法，简单来说就是在建模环境中该元素的样式，具体可以表示为图形形状、文本描述或者二者的混合。(2) 抽象语法：定义了领域语言中元素的概念、元素之间的关系、元素之间的交互规则及约束，并为建模的正确性提供参考。

(3) 语义定义域：通过数学逻辑定义了各元素的含义。

(4) 语法映射：定义了抽象语法到具体语法的映射，记为： $A \rightarrow C$ 。抽象语法定义了各元素的连接规则，具体语法是各元素的表现形式，但是在不同场景下表现方式也可能不同，因此，在不同建模语言中需要定义相应的语法映射规则。

(5) 语义映射：定义了抽象语法到语义定义域的映射，记为： $A \rightarrow S$ 。

此外，语义映射和语义定义域共同描述了领域建模语言的语义，即元素具备什么含义。语义可以进一步划分为结构语义和行为语义，其中，结构语义可以用数学逻辑描述元素之间的组合关系，如需求图中的总需求和功能性需求之间的而关系可以用数学上的集合来表示；行为语义可以用数学逻辑描述元素之间的动态关系，如状态的变化可以用状态机图来表示。

因此，在领域语言构建过程中，在元元模型的基础上添加领域概念、标准、规范，即是定义建模语言的语法和语义的过程。

2.3 基于 GOPPRRE 的元建模设计

2.3.1 GOPPRRE 元元模型

本文在 GOPPRR 元建模方法的基础上设计多架构建模方法以支持系统开发，其中，GOPPRR 代表六种元元模型：图（Graph）、对象（object）、点（point）、属性（property）、角色（role）、关系（relationship）。这些元元模型之间存在一定的引用、关联、绑定等规则和约束，这些规则和约束是构建元模型的基础。经过工程实践，在 Kelly 提出的 GOPPRR 元元模型基础上作出修改后转变为 GOPPRRE 方法，如图 2.3 所示。

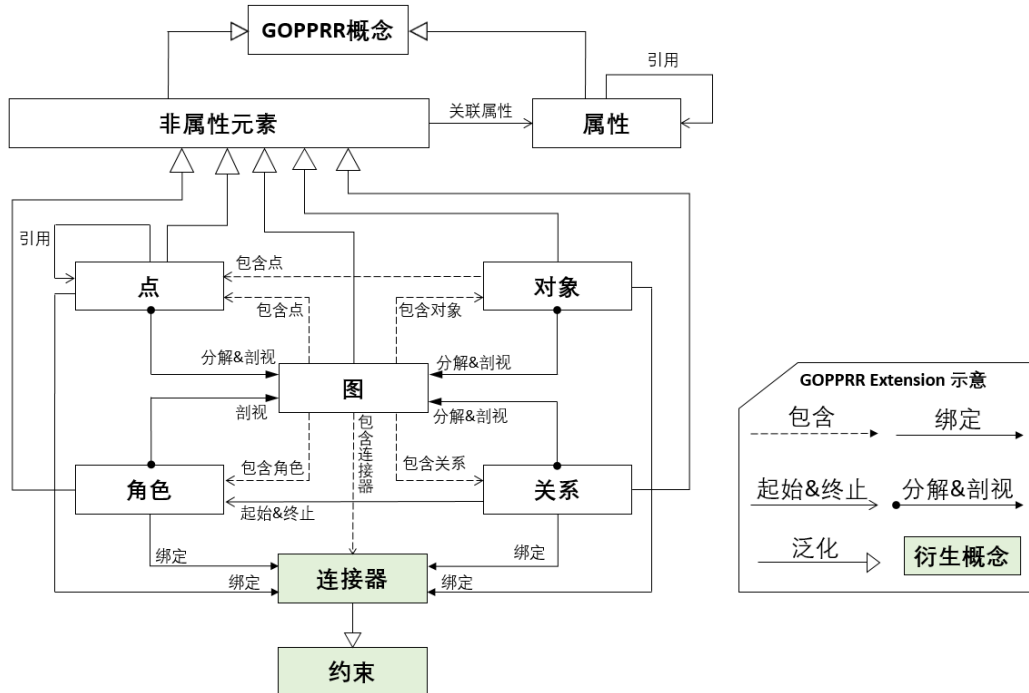


图 2.3 GOPPRRE 元元模型

GOPPRRE 元元模型描述六种概念之间以及概念之间的规则，根据这些规则可以定义元元模型本身，也可以用于定义其他建模语言，如 UML 和 SysML。将这六种元元模型划分为两种：属性元素和非属性元素，非属性元素包括图、对象、点、角色、关系，属性元素用于描述非属性元素的特性。具体的元元模型说明见表 2.1。

表 2.1 GOPPRRE 元元模型

元元模型	名称	描述
graph	图	是对象、关系、角色类型及其绑定关系的集合，描述系统设计过程中的某个视点 1: 可以实现对象、角色和关系的重用 2: 可以定义对象、关系、角色之间的连接关系 3: 可以定义剖视关系，即对象到图的映射规则 4: 可以定义分解关系，即对象、关系、角色到图的映射规则 5: 可以用属性类型描述图的性质
object	对象	是图中的主要元素，是可以独立于关系和角色单独存在，可以用属性类型描述对象的性质
point	点	1: 描述对象与角色连接的端口，用于限定某个对象连接的角色类型。点不能单独存在，必须依附于对象之上 2: 允许点类型之间引用，从而提高点的可重用性
property	属性	1: 描述其余五种类型元素的性质，不能单独存在 2: 属性可以具有不同的数据类型（字符串，文本，数字，布尔值，集合等） 3: 允许属性类型之间的引用，可以继承属性的数据类型，如“出生日期”继承“日期”属性，共用同一数据类型
relationship	关系	表示对象之间的关联，通过角色和对象相连

role	角色	描述对象或对象上的点如何与关系相连接, 表示对象类型在关联中的角色
------	----	-----------------------------------

此外, 元元模型之间存在一定的规则和约束如图 2.3 所示, 具体的关联关系见表 2.2.

表 2.2 GOPPRRE 元元模型连接规则

关联规则	描述
包含	描述图类型和对象、关系、角色类型之间的组合关系
属性连接	描述图、对象、关系、角色、点五种类型元素和属性类型的映射关系
剖视	描述一个对象可以实现到一个或多个图的映射
分解	描述一个对象可以分解成一个图, 这个图 and 对象之间的关系为——映射
绑定	1: 在有点参与的建模中, 用于描述对象上的点和关系两端角色的连接关系 (如 SysML 的内部模块图) 2: 在无点参与的建模中, 用于描述对象和关系两端的角色之间的连接关系 (如 SysML 的需求图) 3: 每个对象的端点应关联至少一个角色
关联	描述对象类型和点类型; 关系类型和角色类型之间的连接关系

2.3.2 多架构建模

采用多架构建模方法对特定领域进行建模设计时, 首先要根据元元模型设计领域语言。在设计语言时, 设计师需要根据领域知识提取相关概念、规则和约束, 并明确系统边界, 对系统内的元素进行初步定义。系统内的元素构成了建模语言中的元模型, 概念、规则和约束则生成了建模语言的语法和语义。

多架构建模方法中的元元模型具备高度的抽象层次, 不包含与领域相关的信息。因此, 采用多架构建模方法进行建模时, 主要的关注点在于如何将元元模型与领域知识结合, 形成一系列可形式化描述的元模型, 进而形成建模语言。下面

以 SysML 中的活动图为例，说明多架构建模方法如何支持建模语言的构建。

活动图是一种用于描述工作流程的视图，可以是简单行为活动的线性描述，也可以是具备条件分支的并行活动。在 SysML 建模规范中，规定了其中的主要元素和工件如表 2.3 所示。

表 2.3 SysML 活动图主要元素

元素	说明
行动 (Action)	代表活动图中的一个操作
控制流 (Control Flow)	活动由控制流关联在一起
叉节点 (Fork Node)	可以连接一个控制流到多个活动，用于表示多个并行活动
合节点 (Join Node)	可以把多个并行活动合并在一起
决定节点 (Decision Node)	是一个条件连接器，可以分离单一控制流到多个分支上，每一个分支都有判断条件，当满足条件时执行活动，否则执行其余活动。
活动开始 (Activity Begin)	开启活动的控制流
活动结束 (Activity Final)	结束活动的控制流

在规定系统中的主要元素后，还需要根据领域知识定义元素之间的关系，例如：活动图中“活动开始”和“活动结束”不能直接相连；“行动”不能作为活动图流程的终端。在构建领域建模语言时，这一步还需要根据专家经验、行业规范等进行补充和完善。

采用多架构建模方法对构建活动图时，首先需要根据六种元元模型对表 2.3 中的关键元素进行定义，定义如表 2.4 所示：

表 2.4 多架构建模方法描述 SysML 活动图

元元模型	元模型
图 (graph)	活动图
对象 (object)	行动、叉节点、合节点、决定节点、活动开始、活动结束
点(point)	
属性 (property)	行动名称
关系 (relationship)	控制流

角色 (role)	控制流始端、控制流终端
-----------	-------------

在设计师采用多架构建模方法归纳系统关键元素后，需要将领域知识和约束添加到元模型中，使其可以完整地描述建模语言的语法和语义。

1. 固定语法用于描述表 2.4 中关键元素的表示方法，以便于领域专家辨识和理解。在多架构建模方法中采用图形化的方式对固定语法进行定义，如“活动图”表示为一个矩形框图，“行动”表示为一个圆角矩形，“控制流”表示为一条实线，“控制流终端”表示为一个箭头。
 2. 抽象语法用于描述元素之间的内部关系，在多架构建模方法中主要是在图、对象中进行定义。
 - 1) 在对象元模型中定义对象上可以附着的点元模型，并在建模时通过“同源点”的设定在对象上添加多个实例点，这些点是由同一个点元模型实例化而来，从而扩展了点的可重用性。
 - 2) 在图元模型中可以定义图的类型、图中元素的连接关系、图中元素与图之间的映射关系。其中，图的类型定义了图元模型中可以关联的对象、关系、角色元模型；图中元素的连接关系定义了三种类型：关系元模型可以关联的角色元模型、角色元模型可以关联的对象元模型（无点元模型参与的建模）、角色元模型可以关联的点元模型（有点元模型参与的建模）；图中元素与图之间的映射关系主要包括分解和剖视两种，例如活动图中的某个“行为”代表系统的一个活动，可以将其映射到另外一幅图中以便详细描述这个活动的内部结构。
 3. 语义用于描述系统中元素的数学逻辑，使系统具备一定的意义。在多架构建模方法中的规则可以粗略地分为以下几类：
 - 1) 事件约束规则：指元模型在实例化时的数量约束，如在一张图中某个对象可以被实例化的次数。
 - 2) 值约束规则：指实例化组件的属性值是受限制的，如字符串类型或矩阵数据类型。
 - 3) 唯一性规则：指每个实例化组件都必须被唯一标识，如每一张图中的所有元素的命名必须不同。
 - 4) 端口约束规则：指点的实例化组件的方向是受限制的，对象元模型设计过程中添加点元模型，并决定点的方向：输入点、输出点、无向点。
- 将领域知识、约束、规范用多架构建模方法表达出来以后，实际上已经完成

了领域建模语言的构建。最后根据实际情况基于建模语言实现特定领域模型的构建，从而形式化地描述系统的某个侧面，在建模过程中如果违背了语法和语义规则，则无法完成建模。

2.3.3 元建模方法对比

目前已经存在多种元建模方法，这些方法都是通过一些抽象概念表示建模元素，本节选择 GME、GOPPRRE、DSL Tools、ARIS、Ecore、Visio 几种元建模方法进行比较。主要是基于两部分进行比较：一是语言支持的建模概念；二是对模块化和重用性的支持。

不同元建模语言使用的建模概念都不同，例如多架构建模方法使用图、对象和点等概念建模。ECore 使用类、参考和属性等概念建模。因此，为了统一比较，首先从这些语言支持的概念中抽象出基本的概念，例如对象类型、关系类型、属性角色类型、点类型、模型类型和属性类型。以基本概念为标准，比较各类元建模语言支持的情况见表 2.5 所示（支持表示为 Y，不支持表示为 N）

表 2.5 不同元建模方法对模型概念的支持情况

建模方法	概念					
	对象	关系	角色	端点	模型	属性
ARIS	Y	Y	N	N	Y	Y
Ecore	Y	Y	N	N	N	Y
GME	Y	Y	Y	Y	Y	Y
DSL Tools	Y	Y	Y	N	N	Y
MS Visio	Y	Y	N	N	N	Y
GOPPRRE	Y	Y	Y	Y	Y	Y

对象类型、关系类型和属性类型是最基本的建模概念，因为所选的元建模语言都通过各自的方式支持这三种概念。有一半的建模语言支持模型类型和角色类型。模型类型的概念能够描述模型的视角，角色类型的概念能够描述更加复杂的关系。在上述比较中，只有 GOPPRRE 元建模语言支持全部建模概念。一般认为，元建模语言支持的建模概念越多，描述能力越强。因此根据支持的建模概念，可做如下排序（≡ 表示支持的建模概念数量相同）。

GOPPRRE≡GME>MS DSL Tools>ARIS>ECore≡MS Visio

在前文中提到元建模方法中涉及到概念之间的互相引用，这些引用可以提高元素的可重用性，提高建模效率。六种元元模型中除 ARIS 和 Visio 外，其他的元建模言都支持继承的概念。Ecore、GOPPRRE、GEM 和 MS DSL Tools 元元模型都支持对象的继承，其中，GOPPRRE 和 GME 还实现了关系、模型类型等元素之间的继承。此外，所有元元模型都支持对元素的分组，并相应有各种容器的概念，如项目、文件夹、模板、语言或 EPackage，具体内容见表 2.6 所示。

表 2.6 元建模语言支持的结构化、重用性和模块化

	ARIS	Ecore	GOPPRRE	GME	MS DSL Tools	MS Visio
继承	n	y	y	y	y	n
分组	Folder	EPackage	Project	Model	Language	Stencil

此外，本文提出的 GOPPRRE 元建模方法与 GOPPRR 方法相比，有以下区别：

(1) 元素之间关联关系的定义：GOPPRR 方法将图中元素划分为对象类型和绑定类型，绑定类型用于描述对象、点、关系、角色之间的连接关系。多架构建模方法将绑定类型描述为元素之间的连接关系，并不作为一种基本类型。

(2) 绑定关系的细化：GOPPRR 方法中将元素间的连接系统称为绑定，多架构建模方法将连接关系作出了细化：在工程实践中，点一般表现为物质或信息交互的接口，因此点的功能和对象的功能是息息相关的，在此情况下，对象和点的链接关系则表现为一种强关联，因此采用“关联”关系来进行描述；角色则一般表现为对象在关系中的身份，因此角色的功能和关系的功能连接也比较紧密，同样采用“关联”关系进行描述；但在建模中，关系两端的角色对于对象并无具体要求，简单来说，设计人员实际上是根据设计需求灵活地改变角色两端可连接的对象，因此采用关联较弱的“绑定”关系进行描述。

(3) 点和角色方向的定义：GOPPRR 方法中对于点和角色的方向并未进行说明，但在工程实践中，多架构建模方法可以提前对点和角色元模型的方向进行定义，从而大大提高建模效率。其中，角色的方向在元模型设计过程中即可进行定义，描述某一角色在一段关系中处于信息/物质的输入端还是输出端；点的方向则在对象元模型的设计过程中进行定义，包含三种方向：输入点、输出点、无向点。与角色不同的是，即使具备相同功能的点元模型在不同的对象上也可能表现为不同的方向，例如“接口点”在某个对象上可能表现为输入端，在另外一个对象上则表现为输出端。

综上所述，多架构建模方法的描述能力更强。对元建模语言支持的模型概念比较中，语言支持的模型概念和描述能力成正比。多架构建模方法、GME 可支持的建模概念最多，描述能力最强，Visio 描述能力最差。在结构化、重用性和模块化方面，多架构建模方法在重用和元素组织方面也没有短板。因此，在几种常用的元建模方法中，多架构建模方法描述能力强，能够支持更加复杂的模型概念，建模描述上也更加完整。

2.4 小节

本章在现有元建模方法的基础上提出一种多架构建模方法，以支持系统全生命周期中多种架构模型的形式化表达。为了在更高抽象程度上支持建模语言的设计与开发，本文首先在 M3-M0 的元建模框架上对建模方法做出规范；其次，为了描述领域内的知识、约束、规则，本文在建模方法中集成了语法和语义的表达能力；然后，在 GOPPRR 建模方法的基础上，添加了“连接器”的概念以描述建模中的一套连接规则，进一步规范了建模元素之间的关联关系，并在此基础上完善了图中元素和图的映射关系。相比传统的元建模方法，本文设计的多架构建模方法可以支持更多的建模概念，并在结构化、重用性和模块化方面表现更加，此外，在工程验证中，多架构建模方法也有较好的表达能力。

第3章 基于多架构建模方法的系统设计流程

3.1 引言

基于多架构建模方法可以构建不同领域的建模语言，但语言本身并不能解释在系统开发过程中的设计组织流程。IBM 提出的 Harmony 是一种针对嵌入式实时开发系统的设计流程，将系统开发划分为需求分析、系统功能分析、设计综合；OOSEM 则将系统设计流程划分为利益相关人需求定义、需求分析、功能分析、架构设计、验证与确认等过程；Vitech 则基于需求分析、功能/性能分析、框架综合设计、确认与验证四个主要的并行活动支持系统开发。

本文采用 RFLP (Requirement、Functional、Logical、Physical) 的映射方法，从需求、功能、逻辑、物理四个维度支持系统正向设计。本章 3.2 节中介绍 RFLP 的正向设计流程，3.3 节中以 SysML 通用建模语言为例，介绍设计师如何按照多架构建模方法构建建模语言，并按照 RFLP 流程支持正向设计。

3.2 基于系统工程的正向设计

3.2.1 需求分析

需求分析的目的是分析系统各利益相关人的需求，并将其转化为可形式化描述的系统需求。在系统工程领域，利益相关人主要包括两种：领域模型库开发人员和领域模型库使用人员。前者包括领域专家、领域建模语言开发人员、建模框架开发人员、信息化工程师等；后者包括架构研发人员、测试人员、用户等。因此，需求分析实际上可划分为两个部分：利益相关人需求定义阶段，以及需求分析生成阶段。其中，前者的目标是在系统定义的范围内获取并记录利益相关人对系统的要求，后者的目的是将定义的需求按照一定的规范表述成电脑可理解的需求模型。需求分析定义系统的功能需求和非功能需求，功能需求定义系统必须实现的功能，非功能需求包括性能需求和接口需求等，其中，性能需求定义系统执行这些功能所需要满足的参数，接口需求定义了系统之间的连接关系。

(一) 利益相关人需求定义阶段

需求定义是系统工程实施的初始过程，通过界定利益相关人进一步地明确系统设计的边界。图 3.1 所示为利益相关人需求定义的过程，并确定在此阶段中系统

的输入输出。

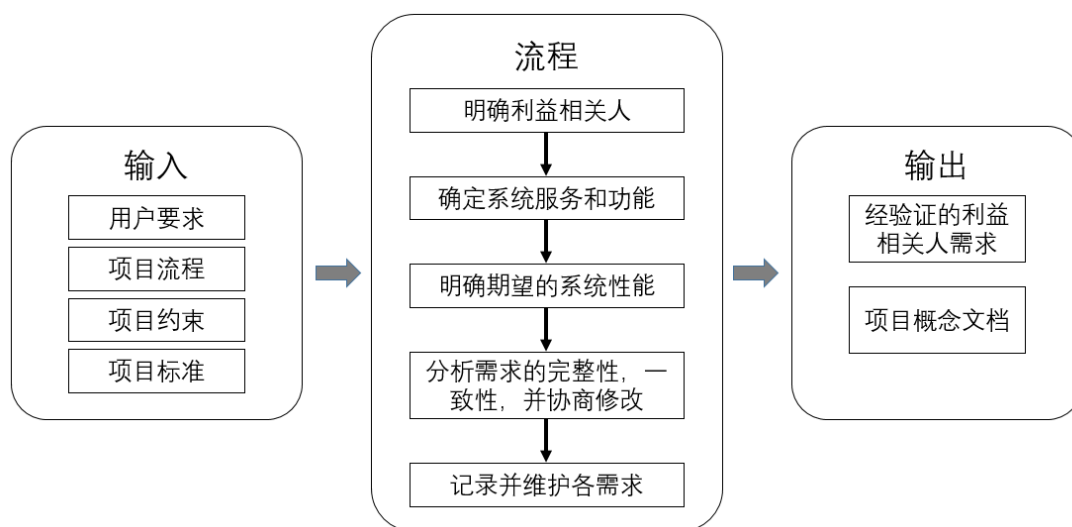


图 3.1 利益相关人需求定义

需求定义过程的输入包括用户要求、项目流程、约束、标准等。其中，系统在设计过程中涉及众多设计人员、操作人员以及测试人员，用户要求定义了上述人员对系统的不同期望；项目流程提供了以往相似项目的设计流程，规范系统的设计范围；项目约束规范了各利益相关人对系统提出的性能、成本等边界约束；项目标准定义了项目最终期望达到的效果及质量要求。

需求定义的输出主要包括两项：经过验证的利益相关人的需求及项目概念文档。众多利益相关人对系统有不同的期望，需求定义后系统会生成一系列正式记录并经过设计人员批准验证的需求，包括系统具备的能力、可以提供的服务质量、成本和性能的约束限制等；同时系统也会生成相应的概念文档，包括系统需求文档、接口需求文档、软硬件需求文档等。需求定义阶段主要包括以下几种活动：

（1）明确利益相关人：利益相关人是与系统有关的团体或个人，在系统全生命周期中以不同的方式参与运行，主要包括系统设计师、系统测试人员、系统使用人员等。

（2）确定系统服务和功能：可以有效地明确系统设计的边界，同时系统提供的服务或功能需要满足各利益需求人的期望。

（3）明确期望的系统性能：确定对系统功能影响较大的关键性能参数，以及为了满足关键参数可能无法完全实现系统的非关键性能。

（4）分析需求的完整性、一致性：系统利益相关人所提需求是否完整解决系统问题，识别并修改不合理的需求。

(5) 记录并维护各利益相关人需求：在系统全生命周期中对需求进行记录和存档。

(二) 需求生成阶段

需求分析及生成的目的是审查评估上一步产生的需求，确定需求的优先级；同时将这些需求转化为所有利益相关人可以理解的技术视图，这种视图可以用规范、图纸或其他方式进行描述，在基于模型的系统工程方法中一般采用可视化模型来表达（如 SysML 中的需求图）。图 3.2 所示为需求分析生成的过程，并确定在此阶段中系统的输入输出。

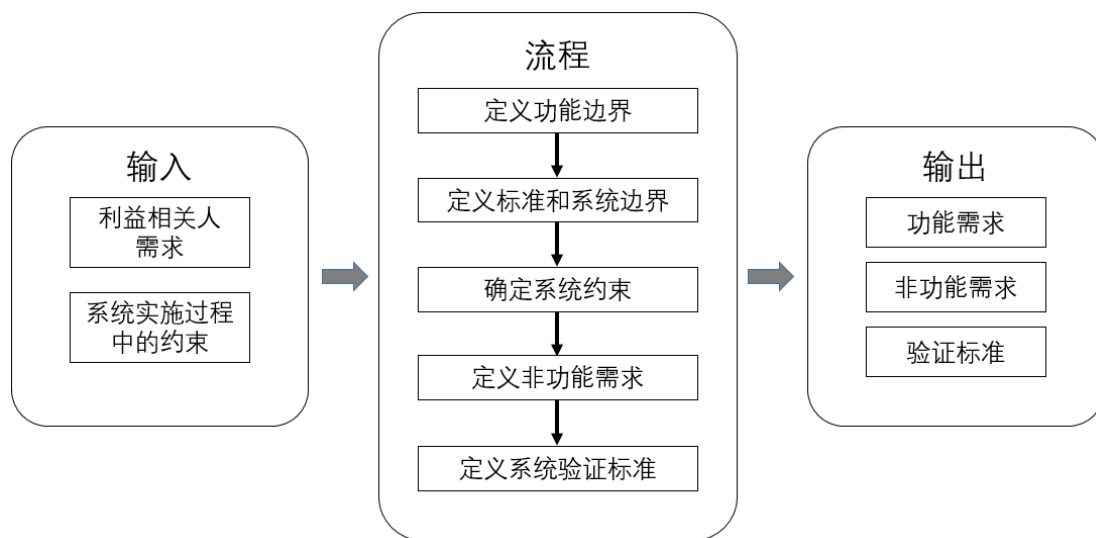


图 3.2 系统需求分析与生成

需求分析与生成阶段的输入主要是需求定义过程中生成的需求与概念文档，此外还包括系统在实施过程中的各种约束，如企业实施的策略、相关行业规范、适用标准以及专家设计经验等。在对系统的功能边界、系统边界、约束、非功能需求、验证标准等进行定义之后，需求分析阶段生成一系列规范描述的功能、性能、接口需求以及系统设计中的约束。这些需求和标准为整个系统设计提供基础，并在系统全生命周期中不断发展与完善。

需求分析阶段主要包括以下几种活动：

(1) 定义功能边界：明确系统在预期的操作环境下可以实现的功能，这些功能要求满足利益相关人的诸多需求及各种约束条件。

(2) 定义标准和系统边界：确定系统最终应该满足的标准，这些标准一部分来源于利益相关人，另一部分来源于企业、行业、领域的标准。此外，需要定义系统设计的边界，并规范该边界与外部系统的交互。

(3)确定系统约束:确定可执行的系统约束,并在众多约束中进行折衷选择,使系统能够最大程度地满足设计需求。

(4)定义非功能性需求:包括性能需求和接口需求。其中,性能需求明确系统在预期环境下可以实现系统功能的程度;接口需求明确系统之间的数据或物质交换效率。

(5)定义系统验证标准:确保上述制定的需求可被验证。

3.2.2 功能分析

功能分析是在设计师不了解系统内部结构的情况下对其可实现需求的能力做出细化,它建立在系统需求分析的基础上,系统需求文档已经对系统的高层功能进行了梳理,功能分析则继续向下分解,直到分解的粒度足以支持系统底层逻辑和物理的实现。图 3.3 所示为功能分析的过程,并确定在此阶段中系统的输入输出。

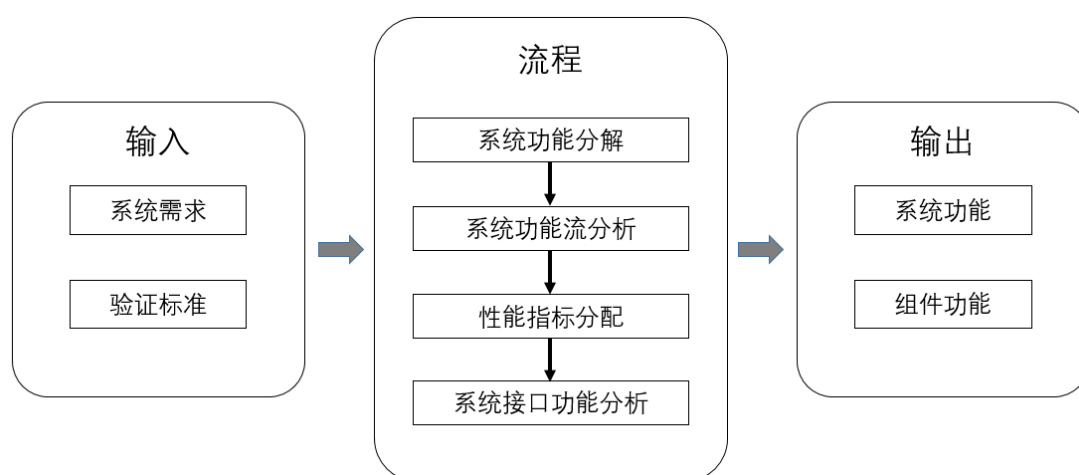


图 3.3 系统功能分析

系统功能分析阶段的输入是需求分析的输出,需求文档中定义的顶层需求经过层层分解,最终得到系统功能的全部描述,包括各组件功能、系统接口功能、功能流及数据流的传递过程等,从而进一步明确系统与用户的交互方式。功能分析阶段主要包含以下几种活动:

(1)系统功能分解:按照系统需求的优先级进行功能的分解,可以将复杂的系统设计问题转化为如何实现若干相互独立的子功能。

(2)系统功能流分析:系统为了实现某些功能而进行的数据流动。

(3) 性能指标分配：按照功能分解的结果，将性能指标进一步细化，直到可以描述系统组件需要满足的关键参数。

(4) 系统接口功能分析：定义不同子系统之间的接口标准，包括数据流和物质流。

3.2.3 逻辑分析

以系统需求和功能作为输入开展系统的逻辑分析，其中，需求代表利益相关人想要设计什么系统，功能代表系统可以实现什么能力，逻辑代表系统实现功能的可选择方案。图 3.4 所示为逻辑分析的过程，并确定在此阶段中系统的输入输出。

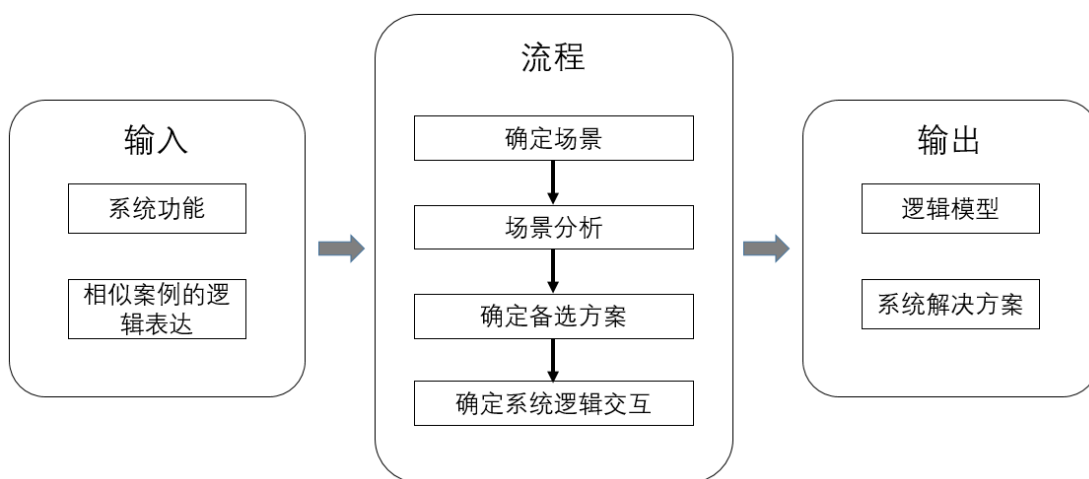


图 3.4 系统逻辑分析

系统逻辑分析阶段的输入是系统功能描述以及相似案例中的逻辑表达，在对某系统进行设计时，可以参考相似系统实现某些功能时的行为表达，从而提高设计效率。该阶段最终得到实现系统功能的诸多可选择方案，但是这些方案不涉及具体的系统结构，只描述系统实现功能时的行为变化。逻辑分析的主要活动如下：

(1) 确定场景：确定逻辑分析的场景，不同场景下系统执行功能不同，功能可实现的程度也不同。

(2) 场景分析：明确该场景下系统的输入、输出、转换关系、系统的目标和约束等，进一步细化系统内部行为变化。

(3) 确定备选方案：在该场景下为了实现系统功能的几种可能的解决方案。

(4) 确定系统逻辑交互：确定随着时间变化不同系统之间的如何交互，主要

关注系统物质流和信息流。

3.2.4 物理设计

系统物理设计是设计过程的最后一环，负责为逻辑分析中的多种解决方案提供具体的系统设计方案，具体到系统中部件的设计参数，并且设计方案需要满足利益相关人需求。图 3.5 所示为物理设计的过程，并确定在此阶段中系统的输入输出。

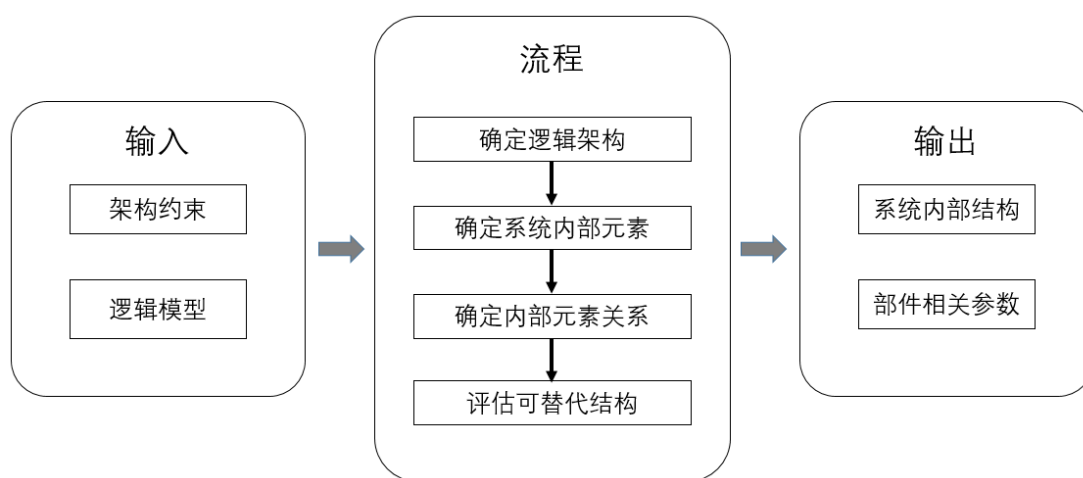


图 3.5 系统物理设计

系统物理设计阶段的输入为上一阶段生成的系统架构模型以及相关的架构约束，最后输出为系统的内部结构及部件相关设计参数。物理设计阶段的主要活动如下：

（1）确定逻辑架构：明确系统在实现功能时有哪些解决方案，系统内的行为是如何变化的。

（2）确定系统内部元素：根据逻辑架构确定系统内部应该有什么部件。

（3）确定内部元素关系：根据系统行为变化确定部件之间的信息流、物质流传递过程，从而确定内部元素之间的交互方式。

（4）评估可替代结构：在实现某种系统功能时，系统可能有多种解决方案，内部元素的组合、元素之间交互方式也都不同，根据利益相关人需求可以对物理架构进行评估。

3.3 基于 RFLP 的多架构建模流程

工程师采用基于需求、功能、逻辑和物理架构的正向设计流程处理 MBSE 中的复杂设计问题，但是由于不同领域中有不同的建模语言，针对不同建模语言有不同的映射规则。SysML 作为系统工程领域的通用建模语言，其余多种领域建模语言都是在此基础上进行拓展。因此，本节以 SysML 建模语言为例，说明多架构建模方法支持 SysML 建模，并通过需求、功能、逻辑、物理的映射方法实现系统的正向设计。

类似地，在对特定领域进行 MBSE 实践过程中，M3-M0 的建模框架提供建模流程，从元元模型出发，基于 SysML 的语法和语义，采用多架构建模方法设计领域建模语言，在此基础上实现系统某个视点的模型化描述；RFLP 架构提供模型设计思路，在领域模型库基础上分别对系统需求、功能、逻辑、物理架构进行建模，辅助系统设计。参考 Magicgrid 的问题分解方法^[58]，本文将特定系统建模分为问题、解决方案、实施方案三个阶段，每个阶段对应需求、功能、逻辑、架构四种视图，以图 3.6 为例。一般来说，系统设计人员通过需求图和用例图来进行需求分析，通过包图、活动图来进行功能分析，通过用例图、时序图、状态机图进行逻辑分析，通过模块定义图、内部模块图来实现系统架构设计。

	流程				
		需求	功能	逻辑	架构
约束	问题 (黑盒)	B1 利益相关人员 要求  要求	B2 用例 包图	B3 系统背景 用例图	B4 系统梗概 包图
	问题 (白盒)	需求分配矩阵 用例图	W2 功能分析 系统用例图	W3 子系统逻辑交互 时序图	W4 系统描述 模块定义图 内部模块图
	方案	S1 系统需求 系统需求图	S2 系统功能 系统级活动图	S3 系统结构 系统级状态机图	S4 系统参数 参数图
		SS1 子系统需求 子系统需求图	SS2 子系统功能 子系统活动图	SS3 子系统结构 子系统状态机图	SS4 子系统参数 参数图
	实施	MetaGraph			

图 3.6 系统设计结构

在领域建模语言设计的过程中，按照 RFLP 方法将建模分为需求分析-功能分析-逻辑分析-架构设计四个层次，依据每个层次中使用到的图元模型进行设计。图元模型的建模流程如图 3.7 所示：

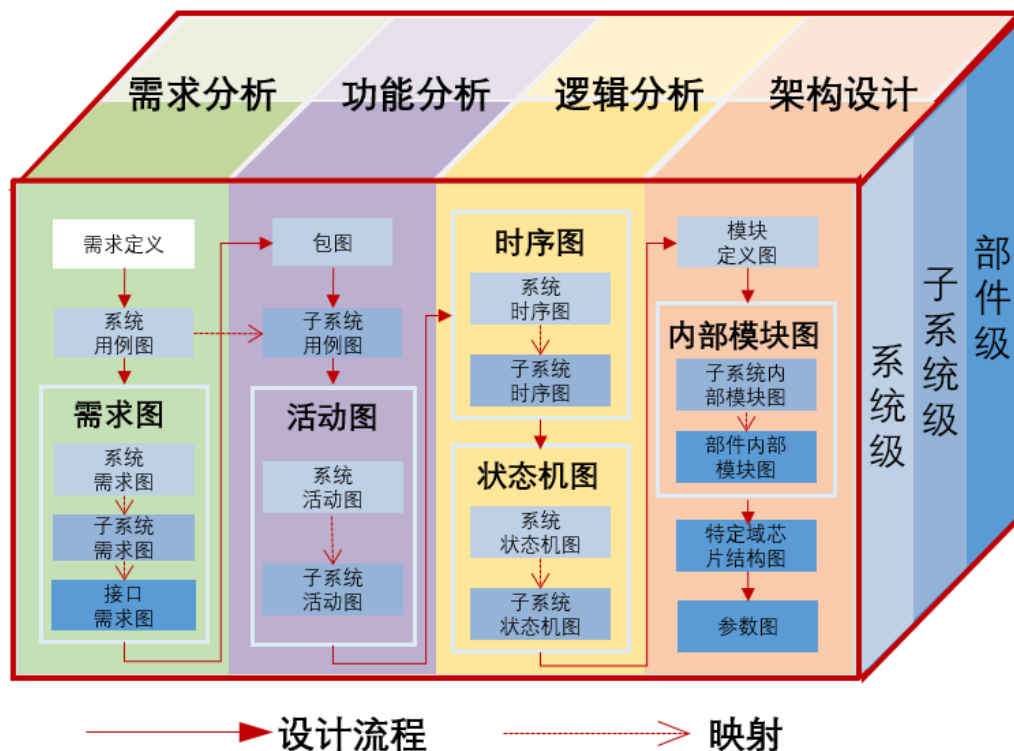


图 3.7 多架构建模流程

设计人员在明确利益相关人要求后进行需求分析、功能分析、逻辑分析和物理架构设计，并通过反复迭代、逐层验证的方法确保异构需求在系统级、子系统级和部件级三个粒度上得以满足。

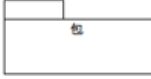
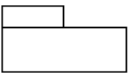
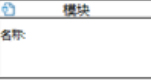
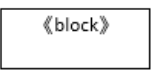
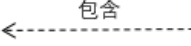
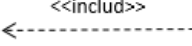
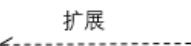
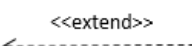
需求分析将系统各利益相关人要求转化为可验证、可分析的需求，在此过程中一般采用需求图和用例图进行描述。其中，需求图是描述特定系统需求的视图，可结构化地描述系统的功能性需求及非功能性需求，并使用包含、复用、满足、细化、验证和追溯六种关系来确立需求之间的可追溯性；用例图是描述系统外部用户与系统内部交互的视图，可以从系统外部观测系统需求满足利益相关人的哪些需求，此外，用例图在需求分析阶段主要用来识别系统在正常运行状况下的运行情况，异常运行状况需要通过活动图等在进行功能分析阶段进行描述。一般而言，用例图可以和需求图关联起来，将系统功能性需求和性能性需求完全覆盖到系统用例中，测试用例也可以对系统需求进行测试与验证。基于多架构建模方法设计的需求图元素及固定语法如表 3.1 所示，用例图元素及固定语法如表 3.2 所示。所

示，其关键元素与 MagicDraw 中进行对比。

表 3.1 需求图构建及描述

需求图					
metaGraph			MagicDraw		
元素	含义	固定语法	元素	固定语法	
对象元模型	包	具备共同特征的元素集合	Block		
	测试用例	测试用例是否满足需求	Test Case Activity		
	约束模块	需求元素满足的约束条件	Constraint Block		
	注释	备注信息	Extended Block		
	需求	利益攸关者的相关需求	Requirement		
关系元模型	包含	需求的分解	derive		
	追溯	弱依赖关系	trace		
	复用	需求的继承关系	copy		
	改善	需求的细化	refine		
	满足	模块满足需求	satisfy		
	验证	测试用例满足需求	verify		

表 3.2 用例图构建及描述

用例图					
metaGraph			MagicDraw		
元素	含义	固定语法	元素	固定语法	
对象元模型	包	具有共性的元素集合 	Package		
	参与者	执行用例的外部操作人员 	Actor		
	接口模块	描述元素之间连接关系 	Actuator		
	模块	建模中基本元素 	Block	《block》 	
	用例	执行的行为 	Use Case		
关系元模型	包含	元素之间的强相关 	Include		
	泛化	内容上的细分 	Generalization		
	关联	元素之间的弱相关 	Association		
	扩展	表示为用例的执行条件 	Extend		

功能分析阶段的重点是将系统的功能性需求转化为系统的功能描述。这阶段的分析是基于系统用例进行的，简单来说将需求分析阶段确认的用例转化为一个可以执行的功能模型，因此，用例图也可以用于描述系统的功能。其中，包图用于描述特定系统实现某些功能所具备的元素，可以从整体对系统进行简单的划分；活动图通过行为活动来描述数据、能量、物质的流动状况。包图和活动图的元素及固定语法分别见表 3.3 和 3.4 所示。

表 3.3 包图构建及描述

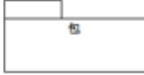
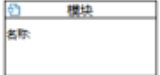
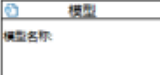
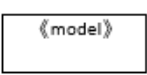
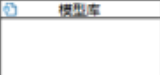
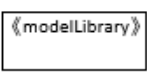
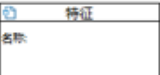
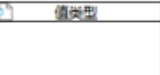
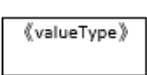
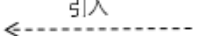
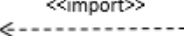
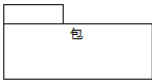
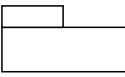
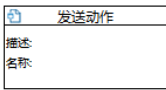
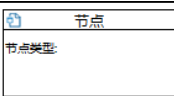
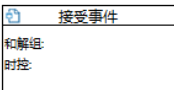
包图					
metaGraph			MagicDraw		
元素	含义	固定语法	元素	固定语法	
对象元模型	包	具有共性的元素集合		Package	
	模块	建模中基本元素		Block	
	接口模块	描述元素之间连接关系		Interface Block	
	模型	某些模块的集合		Model	
	模型库	模型的集合		Model Library	
	特征	元素具备的特殊性质		Signal	
	约束模块	模块需要满足的约束条件		Constraint	
	值类型	模块中的数值类型		Value Type	
关系元模型	泛化	内容上的细分		Generalization	
	关联	元素之间的弱相关		Association	
	引入	引用关系		Element Import	
	组合	结构上的分解		Composition	

表 3.4 活动图构建及描述

活动图					
metaGraph			MagicDraw		
元素	含义	固定语法	元素	固定语法	
对象元模型	包	具有共性的元素集合		Package	
	动作	行为		Action	
	发送动作	动作的传递		Send Signal Action	
	分流	信息流的合并		Merge	
	叉	信息流的分叉		Horizontal	
	节点	事件节点		Node	
	接受事件	条件符合接受事件的传递		Accept Event Action	
	结束	流的结束		Flow Final	
	开始	事件开始		Initial Node	
	选择	根据条件判断执行哪个活动		Dicision	
	终止	事件的终止		Activity Final	
关系元模型	对象流	对象流的传递		Object Flow	
	控制流	控制信息的传递		Control Flow	

逻辑分析阶段主要关注点在于系统实现功能时的逻辑关系，最终得到系统的诸多可执行方案，但是这些方案都只涉及系统的逻辑架构，并不涉及系统实现过程中的具体技术细节。其中，顺序图定义了系统执行不同功能时，信息发送和接受的时间交互关系；状态机图则用于描述一个对象在生命周期内的状态变化情况，

此外，通过状态机图中的状态变化来仿真验证系统的逻辑设计是否正确与合理，进而发现系统的设计和 demand 是否出现偏差。顺序图和状态机图的元素及固定语法分别见表 3.5 和 3.6 所示。

表 3.5 顺序图构建及描述

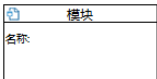
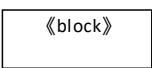
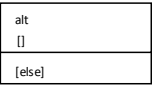
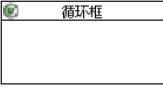
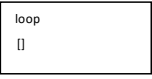
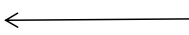
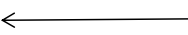
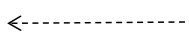
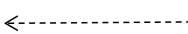
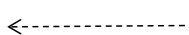
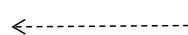
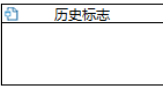
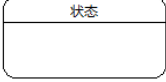
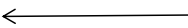
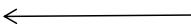
顺序图					
metaGraph			MagicDraw		
元素	含义	固定语法	元素	固定语法	
对象元模型	模块	建模中基本元素		Block	
	操作符	对象线上的连接对象			
	生命线对象	序列图中具备生命线的对象元素		Lifeline	
	选择框	根据条件选择选择框中的活动进行操作		Alternatives	
	循环框	根据循环条件在循环框中循环操作		Loop	
关系元模型	传递消息	消息在生命线之间的传递		Send Message	
	创建消息	消息的创建		Create Message	
	回复消息	消息的返回传递		Reply Message	

表 3.6 状态机图构建及描述

状态机图					
metaGraph			MagicDraw		
元素	含义	固定语法	元素	固定语法	
对象元模型	叉	信息流的分叉		Horizontal	
	开始	状态开始		Initial	
	终止	状态终止		Fina State	
	历史标志	历史事件		Deep History	
	选择	根据条件选择状态的转换方式		Dicision	
	状态	状态机图中主要元素		State	
关系元模型	转换	状态的转换		Transition	

物理设计阶段是对系统的物理结构、软硬件架构进行设计，以满足系统的逻辑结构，最终得到系统需要的各个组件，以及为了满足系统的各项需求，各组件所满足的各项参数。其中，模块定义图定义了系统的主要模块及其关系表达；内部模块图定义了分系统内部、组件内部的关系；参数图定义了系统内部参数传递及约束表达。模块定义图、内部模块图、参数图的元素及固定语法分别见表 3.7、3.8 和 3.9 所示。

表 3.7 模块定义图构建及描述

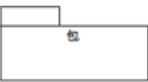
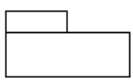
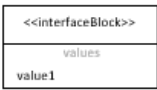
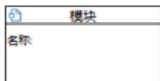
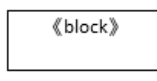
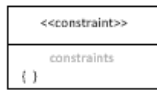
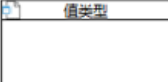
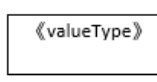
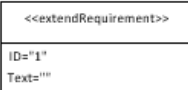
模块定义图					
metaGraph			MagicDraw		
元素	含义	固定语法	元素	固定语法	
对象元模型	包	具有共性的元素集合		Package	
	接口模块	描述元素之间连接关系		Interface Block	
	模块	建模中基本元素		Block	
	约束模块	模块需要满足的约束条件		Constraint	
	值类型	模块中的数值类型		Value Type	
	注释	备注信息		Extended Block	
关系元模型	泛化	内容上的细分		Generalization	
	关联	元素之间的弱相关		Association	
	项目流	数据的流动		Item Flow	
	组合	结构上的分解		Composition	

表 3.8 内部模块图构建及描述

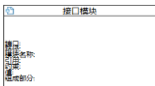
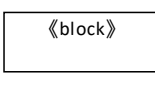
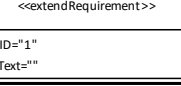
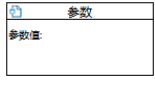
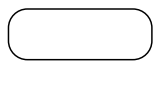
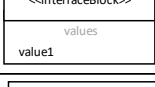
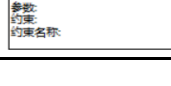
内部模块图					
metaGraph			MagicDraw		
元素	含义	固定语法	元素	固定语法	
对象元模型	接口模块	描述元素之间连接关系 	Interface Block		
	内部模块	图中的基本元素 	Block		
	注释	备注信息 	Extended Block		
关系元模型	流	对象流的传递 	Item Flow		

表 3.9 参数图构建及描述

参数图					
metaGraph			MagicDraw		
元素	含义	固定语法	元素	固定语法	
对象元模型	参数	模块定义图中的参数 	Parametric Property		
	接口模块	描述元素之间连接关系 	Interface Block		
	约束模块	模块需要满足的约束条件 	Constraint		
关系元模型	参数流	参数的传递 	Binding Connector		

综述，具体的元模型库设计如表 3.10 所示：

表 3.10 SysML 建模语言设计

图	关系	对象	目的
需求分析			
需求图	包含、追溯、复用、改善、满足、验证	包、测试用例、约束模块、注释、需求	支持特定系统的功能、性能、接口等需求的形式化表达
用例图	包含、泛化、关联、扩展	包、参与者、接口模块、模块、用例	支持特定系统利益相关人员和系统功能交互关系描述
功能分析			

包图	泛化、关联、引入、组合	包、模块、接口模块、模型、模型库、特征、约束模块、值类型	支持特定系统整体划分, 具备共同特性的元素集合
活动图	对象流、控制流	包、动作、发送动作、分流、叉、节点、接受事件、选择、开始、终止、结束	支持特定系统利益相关人员执行某功能的具体活动描述
逻辑分析			
顺序图	传递消息、回复消息、创建消息	模块、操作符、生命线对象、选择、循环	支持特定系统中活动的时间交互关系描述
状态机图	转换	叉、开始、终止、选择、状态、历史标志	支持特定系统中系统状态变化表达
物理设计			
模块定义图	泛化、关联、项目流、组合	包、接口模块、模块、约束模块、值类型、注释	支持特定系统内部主要模块及其关系表达
内部模块图	对象流	接口模块、内部模块、注释	支持特定系统分系统、部件内部元素及其关系表达
参数图	参数流	参数、接口模块、约束模块	支持特定系统内部参数传递及约束关系表达

此外, 由于 SysML 在系统工程领域广泛应用, 目前基于领域知识设计的领域建模语言也大多基于 SysML 的语法和语义进行构建。因此, 设计人员在采用领域建模语言对系统进行建模设计时, 也可以基于需求、功能、逻辑和物理架构的映射方法进行设计。

3.4 本章小结

本章在现有系统设计流程的基础上采用 RFLP 的映射方法支持设计人员进行建模分析。本文首先分析基于工程学的正向设计流程中需求分析、功能分析、逻辑分析和架构设计阶段中的输入输出, 并对每个阶段中的设计流程进行规范; 在此基础上, 基于多架构建模方法设计 SysML 建模语言, 并在此基础上采用 RFLP

流程设计 SysML 九种图的映射方法，最终可将其拓展为与 SysML 具备类似语法和语义的领域建模语言中视图的映射方法，从而辅助设计人员实现系统的正向设计。

第 4 章 面向多架构建模的本体设计

4.1 本体建模与数据交互

4.1.1 本体基本概念

本体作为一种形式化的、对于共享概念有明确的显式表示，能够更好地表示各种概念、术语所隐含的知识^[59]。随着知识工程的发展和语义 web 技术的广泛应用，基于本体的知识表示方法已经在各种行业中得以推广和使用。其中，OWL (web ontology language, OWL) 是一由国际万维网组织 W3C 推荐和使用的基于本体的语义 web 建模语言，这种语言的核心在于采用 RDF (Resource Description Framework, RDF) 框架对概念进行描述，即采用“主语+谓语+宾语”的三元结构来描述概念、实例，以及它们之间的语义关系。

目前 W3C 提出三种类型的 OWL: OWL-lite、OWL-DL 和 OWL-full。这三种类型的本体在表达能力和推理能力上有所区别。其中，OWL-lite 是三者中语法结构最简单的一种，当需要描述的本体只包含一些简单类，并且类之间只具备简单约束时适合采用 OWL-lite 格式；OWL-DL 相比 OWL-lite 而言表达能力更强，它的数学基础是描述逻辑 (Description Logics, 即 DL 的由来)，可以用于描述一阶逻辑，因此 OWL-DL 可以进行自动推理，使计算机理解本体中的分类层次，以及各种概念之间的逻辑是否正确；OWL-full 是三种形式中表达能力最强的，但是由于它的表达能力全部基于用户定义，无法实现自动推理。本文为了实现建模过程中各元素的完整表达，同时需要基于元素之间的逻辑关系实现自动推理验证，因此采用 OWL-DL 进行设计。下面对 OWL 中基本组成元素进行说明：

(1) 类 (Class): 表示具备相同性质的个体的集合，这些形式用与、或、非等数学形式表达。

(2) 个体/实例 (Individual): 表示在该领域中我们关注的对象，需要注意的是，OWL 本身并不使用唯一命名假设，即不同命名的实例可能对应相同的个体，但是在 GOPPRR 建模中，不同命名的实例必定代表不同的个体，不同的个体采用不同的系统名称来表示。因此面向多架构建模的本体框架中，默认个体之间是不同的。

(3) 对象属性 (Object property): 描述实例之间的关系。对象属性主要参与

到 OWL 的推理机制中，通过对象属性可以定义类之间的等价关系。此外，OWL 对于子类的描述也采用了对象属性，表述为“`rdfs: subClassOf`”。对象属性具备一定的性质：

A:传递属性 (Transitive Property): 如果一个对象属性 P 被申明为传递属性，那么对于任意 x 、 y 、 z : $P(x, y) + P(y, z) \rightarrow P(x, z)$

B:对称属性 (Symmetric Property): 如果一个对象属性 P 被声明为对称属性，那么对于任意的 x 和 y : $P(x, y) = P(y, x)$

C:函数型属性 (Functional Property): 通过函数型属性只能连接一个个体。如果一个属性 P 被声明为函数型属性，那么对于任意 x 、 y 、 z : $P(x, y) + P(x, z) \rightarrow y = z$

D:反函数属性 (Inverse Functional Property): 该函数的逆属性是函数型属性，即对于一个给定的个体，有且仅有一个个体可以通过反函数属性与该个体相连。如果一个属性 P 被声明为反函数型属性，那么对于任意 x 、 y 、 z : $P(y, x) + P(z, x) \rightarrow y = z$

(4) 数据类型属性 (Datatype Property): 描述个体与 XML 数据类型值之间的关系，同样可以参与到 OWL 的推理当中。

(5) 标注属性 (Annotation Property): 用于对类、属性、实例添加信息，区别于对象属性和数据类型属性，标注属性一般只为本体中的元素添加基本的注解，但是本身并不参与到 OWL 的推理中。

4.1.2 基于本体的 MBSE 数据交互方法

基于模型的系统工程大大提高了系统的设计和效率，在其实施过程中应该具备以下能力^[60,61]。

(1) 多架构的表达能力: 可以清楚地描述系统设计过程中的各种信息，这些信息不仅来自多领域多学科，同时也会设计到不同设计工具之间的数据集成。

(2) 可追溯的表达能力: 可以描述系统中不同设计信息之间的关联关系。这种可追溯的能力一般依托于建模工具才能得以表现，因此，这种建模语言还应该能够被计算机理解，即具备一定的形式化表达的能力。

(3) 语义表达能力: 能够清楚地表达设计元素之间的数学逻辑，即语义关系。

(4) 推理能力: 根据预先定义好的公理和设计规则，系统可以推理出设计元素之间可能具备的关系，并且排除不合理的设计方案，辅助设计人员做出决策。

其中，多架构的表达能力是系统工程实施的基础。系统设计需要涉及到多学科知识的耦合，不同领域的工程师采用不同软件工具进行设计。但是这些数据往往存储在不同工具的信息系统中，无法直接进行调用和共享。例如，在无人驾驶汽车的设计过程中，实现电控系统功能的组件有多种型号可以选择，但在结构设计中，不同类型的组件和性能参数存储在无人车架构数据库中，这就需要系统设计人员和结构设计人员进行协商沟通，这样才能手动到架构数据库中查看并进行选择，造成了设计时间的极大浪费。因此，系统工程实践过程中所面临的挑战之一是如何将系统全生命周期中多种工具中的数据进行一致性表达，例如，如何实现建模工具中系统模型与仿真工具中仿真数据之间的互通。

综上，为了解决 MBSE 在复杂产品设计过程中的数据交互困难问题，本文在已有的研究成果之上，设计一种面向多架构建模方法的本体框架，该框架可以为特定领域设计人员和用户提供底层的数据支持。首先根据本体中类、关系、实例、申明等基本元素与建模中元元模型、元模型、模型、语法与语义一一对应，并在此基础上，根据本体推理机来检查建模过程中是否满足语义要求，对于不满足的设计则提出优化改进的方案，从而提高设计人员的设计效率。

4.2 本体设计方法

在系统过程领域中，本体更多地应用于描述特定领域的概念信息，如生物医药本体库、发动机设计本体库、航天系统设计本体库等。但是这些本体只能描述本领域内的知识，很难被其他领域的专家和设计人员理解。本文则面向多架构建模方法设计基于本体的建模框架，不仅可以解决工具之间数据表达不一致的问题，同时采用相同的框架描述便于被系统全生命周期中所有利益相关人理解，从而减少沟通成本，提高设计效率。要实现本体对系统全生命周期中全部数据的统一表达，必须考虑以下因素：（1）本体框架可以完整表示建模过程中的全部信息，根据工程实践，这些信息主要包括：元元模型、元模型、模型、元元模型之间的关联规则、建模语言的语法语义、模型之间的连接关系、元模型的系统属性、模型的系统属性及自定义属性等。（2）本体框架与模型之间可以保持一致性，具体来说，多架构建模方法是建模工具的基础，本体框架则为建模工具提供底层的数据存储方法。建模工具中的模型信息变化时本体文件必须相应改变，本体文件改变时也相应地改变建模工具中的模型数据。与多架构建模方法对应，基于本体的设计框架同样采用 M0-M3 的建模框架，其转化方法如图 4.1 所示。

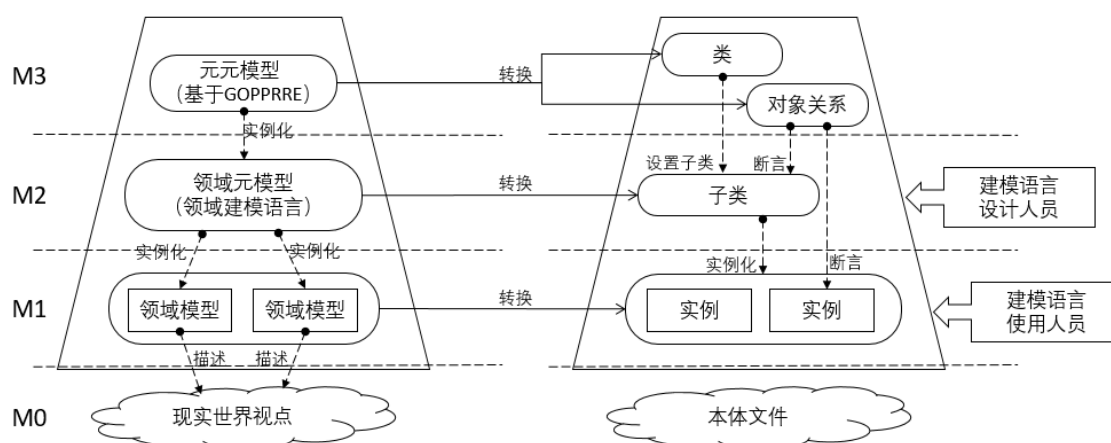


图 4.1 多架构建模与本体框架转化

图 4.1 介绍了本体文件的基本结构，定义了一些多架构模型到 OWL 本体文件的转化方式：在本体中用类（Class）表示建模中六种最基础的元元模型信息；用对象属性（Object-property）表示建模时元模型之间、模型之间的语法关系，例如包含、绑定、映射等；用子类（Sub-Class）表示元模型的信息；用实例（Individuals）表示模型信息。

4.2.1 M3 层本体设计方法

M3 代表元元模型层，在建模中用于描述建模的六种基本元素：图、对象、关系、角色、点、属性，以及这六种元素之间的一些约束和规则。与其相对应，本体中主要用于描述多架构建模过程中元元模型的信息，以及定义建模元素之间的关系，两者分别采用本体中的“类”和“对象属性”进行描述。

“类”在本体中用于描述一种概念的集合，因此在逻辑上具备描述元元模型这种抽象概念的基础。在六种元元模型之外，本体还定义了四种层次相同的类：project、language、connector、plugin。本体中用于描述 M3 层元元模型的信息如表 4.1 所示。

表 4.1 本体中描述 M3 层元元模型信息

类（OWL :Class）	描述
project	本体中包含的项目信息，一个本体文件可以描述多个项目信息，多个相似项目之间可以实现元模型的重用
language	用于形式化描述项目的元模型及其关系的集合，设计人员根据领域知识设计的建模语言，一个项目可以基于多

	种建模语言进行描述，一个建模语言也可以应用到多个项目中，从而提高建模语言的重用能力。例如，SysML 建模语言可以被同时应用于多个项目。
graph	对象、关系及它们连接关系的集合
object	图中最基本元素，可以独立于关系、角色等单独存在
point	对象上的接口
property	用于描述其余五种元模型的特性
relationship	用于描述对象之间的连接关系
role	在关系的一端，与对象或点连接
connector	用于描述图中存在的连接关系或者映射关系。如在“需求图”中允许“需求”元模型和“测试用例”元模型之间通过“验证”关系进行关联；允许“需求”元模型映射到“参数图”中，以验证系统设计参数是否满足系统需求
plugin	图中包含的插件，其子类包括四种：Cell、GanttChart、SwimLane、SysMLDrawingComponent，分别用于描述矩阵插件、甘特图插件、泳道、SysML 绘图插件。需要注意的是，这四种插件的信息可以由 GOPRR 进行描述，如矩阵插件可以由多个对象元模型的实例化组件进行描述。但由于目前较常用的建模工具如 MagicDraw、EnterpriseArchitecture 都支持采用插件模式描述图中信息，相比较而言，采用插件可以减少沟通成本，如采用甘特图插件可以跟家直观地描述项目的时间进度。在本体中设计插件类可以实现类似信息与本体的直接转化。

此外，根据多架构建模方法的要求，元元模型之间有一定的约束规则如表 4.2 所示。与其对应，本体中设计对象属性用于描述建模元素之间的关系，主要包括绑定、映射、包含、关联等。本体中用于描述建模元素之间关系如表 4.2 所示。

表 4.2 本体中建模元素之间关系

对象属性 (Object property)	定义域	映射域	描述
------------------------	-----	-----	----

binding			对象或对象上的点与角色的绑定关系
<i>-roleBindingObject</i>	role	object	角色和对象的绑定关系
<i>-roleBindingPoint</i>	role	point	角色和对象上点的绑定关系
mapping	object、point、role、relationship	graph	建模时的映射关系
<i>-decompose</i>			对象到图的分解关系
<i>-explode</i>	object	graph	对象到图的剖视关系
hasProperty	graph, object, relationship, role and point	property	图、对象、点、角色、关系五种元模型和属性元模型的关系
including			
<i>-graphIncludingOR</i>	graph	object, relationship, role	图元模型包含对象、关系、角色
<i>--graphIncludingObject</i>	graph	object	图中包含对象元模型
<i>--graphIncludingRelationship</i>	graph	relationship	图中包含关系元模型
<i>-graphIncludingConnector</i>	graph	connector	图中包含连接器元模型
<i>-graphIncludingPlugin</i>	graph	plugin	图中包含插件

- <i>languageIncludingGOPPRR</i>	language	graph, object, property, relationship, role and point	语言包含 GOPPRR 六种元素
- <i>projectIncludingLanguage</i>	project	language	项目与语言的关系
link			连接器中的连接关系
- <i>linkFromRelationship</i>	connector	relationship	连接器的始端是关系元模型
- <i>linkToObject</i>	connector	object	连接器的终端是对象元模型
- <i>linkRelationshipAndRole</i>	connector、relationship	role	关系与角色的连接关系
- <i>linkObjectAndPoint</i>	connector、object	point	对象与对象上点的关联关系
connect	connector	connector	在 connector 实例中，用于关联一个关系两边的两个连接器实例

下文对几种对象属性进行说明，包括绑定关系（用于描述角色和对象或者角色和点之间的弱连接关系）；映射关系（用于描述对象、点、角色、关系到图的映射关系，包括分解和剖视两种）；关联属性（用于描述图、对象、点、关系、角色五种元素和属性元素之间的关联关系）；包含关系（分别描述项目与语言的关系；语言和 GOPPRR 六种元素的关系；图和对象、关系、连接器、插件的关系）；关联关系（描述连接器中连接的始端、终端、对象和点的连接以及关系和角色的连接）；连接关系（描述对象间存在的两个连接器的连接关系）。

（1）设计 Object property: including，该对象属性包含五种子属性：
projectIncludingLanguage、languageIncludingGOPPRR、graphIncludingOR、

graphIncludingConnector、graphIncludingPlugin。其中，projectIncludingLanguage 用于描述建模语言对项目的支撑情况，在一个本体文件中可以同时描述几个项目信息，每个项目可以基于不同的建模语言进行设计，通过这种设计方法可以实现建模语言的可重用性，避免设计人员无效且重复的设计过程；languageIncludingGOPRR 用于描述支撑建模语言的元模型种类，建模语言实际上是描述元模型及元模型之间的关系，而元模型的连接关系则在连接器元模型中进行说明，此处则用于描述设计人员针对某领域设计了哪些元模型以支持系统建模分析，并且在本体中进一步设计六种子属性用于说明建模语言和六种元素之间的关系，例如设计 Object property: languageIncludingGraph 来描述建模语言中有哪几种图元模型，设计人员从什么视角对系统进行分析；graphIncludingOR 用于描述图元模型和对象、关系元模型的关系，点和角色的种类则分别在对象元模型和关系元模型中进行申明，而关系、对象、点、角色四种元素在图中的连接方式则在连接器中进行申明；graphIncludingConnector 用于描述图元模型和图中连接器元模型的关系，一个图元模型对应一个连接器元模型，连接器中主要描述两种信息：图中的连接方式和映射关系；graphIncludingPlugin 用于描述图元模型和图中插件的关系，插件种类主要有四种：甘特图插件、矩阵插件、泳道插件和 SysML 绘图插件。

（2）设计 Object property: binding，该对象属性包括两种子属性：roleBindingObject 用于描述无点元模型参与的建模，表示角色和对象的绑定关系；roleBindingPoint 用于有点参与的建模，表示角色和对象上的点之间的绑定关系。“绑定”关系相对于“关联”关系来说描述的强度较弱，主要是用于描述一种关系两端的角色可以匹配到哪些对象或者对象上的点。

（3）设计 Object property: hasProperty，用于描图、对象、点、角色、关系五种元模型关联的属性元模型。与前文的 GOPRR 相对应，这五种类型的元素都是“非属性元素”，可以由“属性元素”进行说明。

（4）设计 Object property: link，用于描述一个连接器中的关联关系，该对象属性包括四种子属性：linkFromRelationship 用于描述连接器在连接时始于某个关系元模型；linkRelationshipAndRole 用于描述连接器中关系和角色的连接关系；linkToObject 用于描述连接器的终端对象；linkObjectAndPoint 用于描述连接器中对象和点的连接关系。相比于“绑定”关系而言，“关联”关系强调的连接更加紧密，linkRelationshipAndRole 描述一种关系在连接时两端的角色可以是什么类型，这种类型大多数情况下是比较固定的，如“关联”关系两端的角色一般是“关联始

端”和“关联终端”；linkObjectAndPoint 描述对象在连接时具备的接口类型，这种类型同样也是比较固定且容易理解的，如设计一个对象元模型“芯片”时，它可以关联的点元模型也是明确的：“正引脚”和“负引脚”。因此，在对象上的点、关系两边的角色确定以后，设计人员根据实际情况设计角色元模型可以连接哪些对象元模型或点元模型，即设计绑定关系，从而完善连接器中的连接规则。

(5) 设计 Object property: mapping，用于描述建模过程中的映射关系，设置两个子类：decompose 和 explode。其中，decompose 用于描述对象、点、关系与图之间的分解关系；explode 用于描述对象、关系、点和角色与图之间的剖视关系。如前文所述，“分解”主要用于描述层级关系，一般是一对一的映射，如将一个对象“功能性需求”分解为“功能性需求图”，用于描述在系统中具备的功能性需求种类及其相互关系。“剖视”主要用于描述组件的某个侧面，一般是一对多的映射，如将一个对象“无人车”进行剖视，可以分别用“无人车种类”、“无人车性能指标”、“无人车价格”等多个图描述这个对象不同方面的信息。

(6) 设计 Object property: connect，用于描述两个对象之间连接器的连接关系。“关联”关系和“绑定”关系一起定义了“关系→角色→点→对象”或者“关系→角色→对象”，但是建模中设计人员不仅想知道对象和关系的连接情况，更想知道对象之间的连接情况。如“智能车一号”与“智能车二号”通过“跟随”关系进行连接，两个连接器分别描述了“智能车一号”与“跟随”的连接情况及“智能车二号”与“跟随”的连接情况，连接关系则用于描述两个连接器的连接情况，从而完整地描述出“智能车一号”和“智能车二号”之间的连接情况。

4.2.2 M2 层本体设计方法

与多架构建模方法对应，本体在 M2 层设计元模型相关的信息，主要包括元模型的申明属性和元模型之间的关系，前者可以描述建模语言的固定语法和元模型的系统属性，后者可以描述建模语言的抽象语法和语义信息。本节首先通过本体中类的概念对元模型进行说明，再通过本体中等价类的概念对建模语法中的抽象语法和语义进行说明，然后通过本体中申明属性的概念对元模型的固有属性进行说明。

在本体中类表示具备相同特性的个体的集合，同时，子类也可以继承父类的全部特性。例如，基于车辆设计领域构建本体时，设计类“车辆”用于描述在陆地上用轮子转动的交通工具，同时设计“智能车”、“新能源车辆”等作为“车辆”的

子类，这些子类都具备“车辆”的全部性质。因此，面向多架构建模方法设计本体时，可以采用子类的概念来描述元模型，元模型继承元元模型的所有特性。

本文第二章中对多架构建模方法中元模型之间的关系作出详细的说明，其中点和属性由于可以实现对自身的引用，因此其表述与其他几种元模型有所区别。在设计点元模型时，首先设计基本点元模型，在设计对象元模型时同时需要确定对象上可以附着的点元模型及点元模型作为接口的方向，此时的点元模型是引用自基本点元模型，是对基本点元模型的重用和继承。与点元模型设计类似，在设计属性元模型时，首先设计基本属性元模型，明确属性的数据类型，之后在设计其余五种元模型时，需要确定元模型可以关联的属性元模型，以及该属性元模型可以被引用的次数。例如，设计基本属性元模型“日期”，对象元模型“人类”关联属性时，可以设计属性元模型“出生日期”，该属性元模型时引用自“日期”元模型。与其对应，在设计本体中的属性和点时，将基本元模型作为第一层子类，延伸的元模型作为第二层子类，子类具备父类的全部性质。在 Protégé 中进行可视化展示，以某需求图为例，本体文件中类描述如图 4.2 所示。

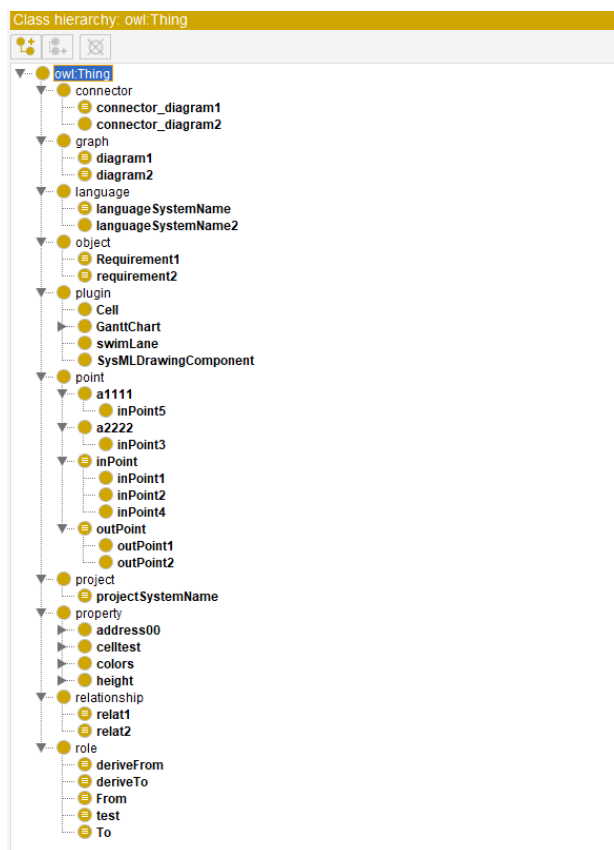


图 4.2 本体描述某建模语言元模型

类只能描述建模语言中元模型的种类，采用本体中等价类的概念描述元模型

之间的关系，即建模语言的抽象语法以及相关语义。下文分别对项目、语言、图、对象、关系、角色、点、连接器的等价类设计方法进行说明。

项目 (project): 采用 projectIncludingLanguage 关联项目和语言，结构如图 4.3 所示。

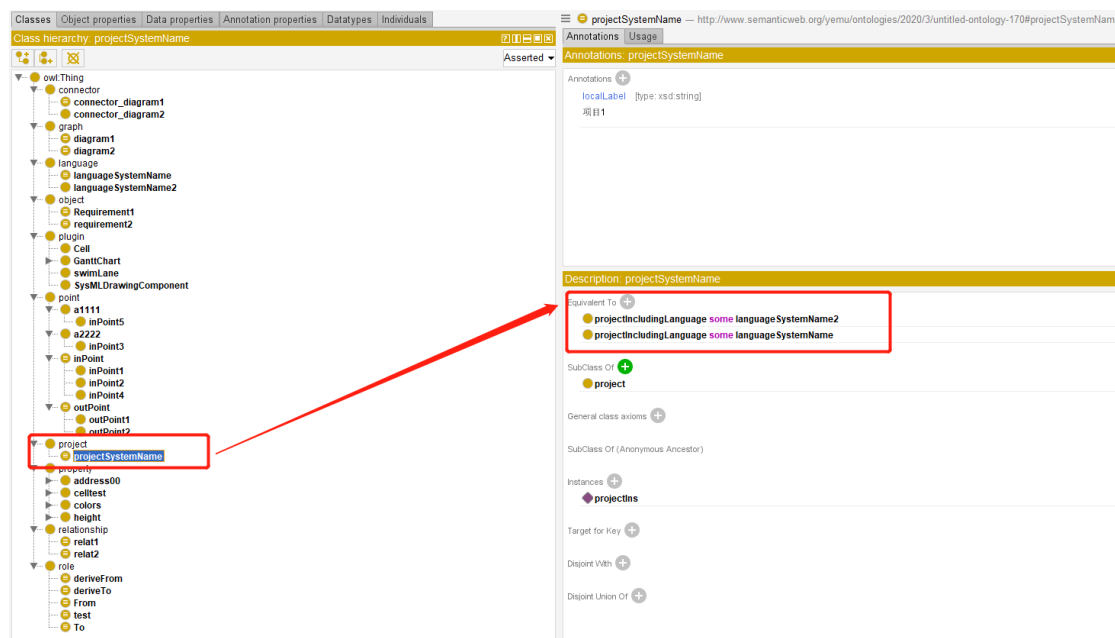


图 4.3 本体中描述“项目”

语言 (language): 采用 languageIncludingGOPRR 的六种子属性关联语言和 GOPRR 六种元素，如图 4.4 所示。

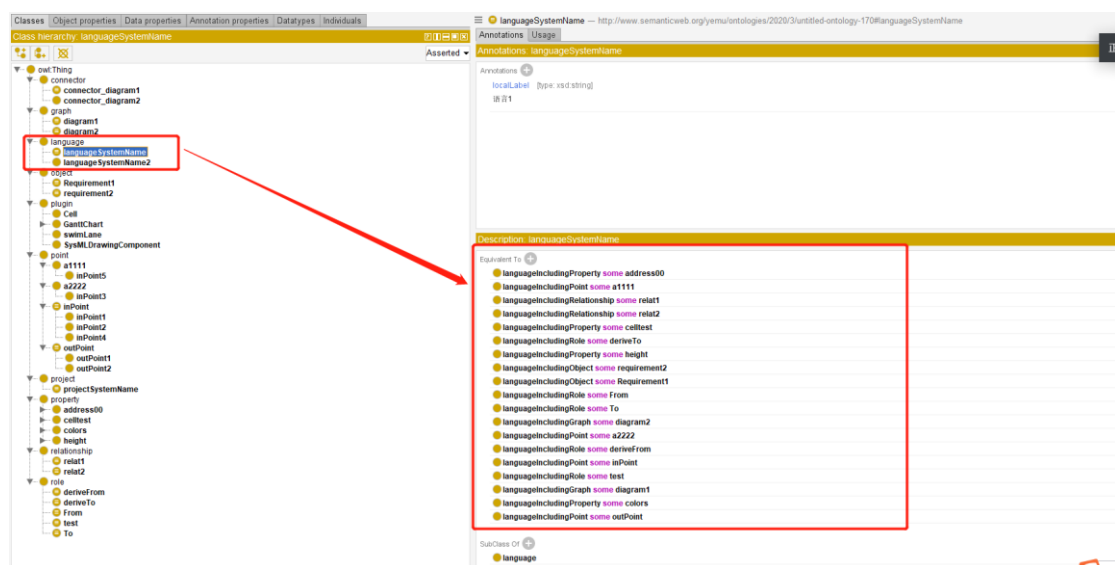


图 4.4 本体中描述“语言”

图 (graph): 采用 hasProperty 描述图元模型和属性元模型的关系；采用

graphIncludingObject 描述图元模型中可以包含的对象元模型；采用 graphIncludingRelationship 描述图元模型中可以包含的关系元模型；采用 graphIncludingConnector 描述图元模型和连接器之间的关系，一个图元模型对应一个连接器元模型；采用 graphIncludingPlugin 描述建模时图中可以包含的插件，如图 4.5 所示。

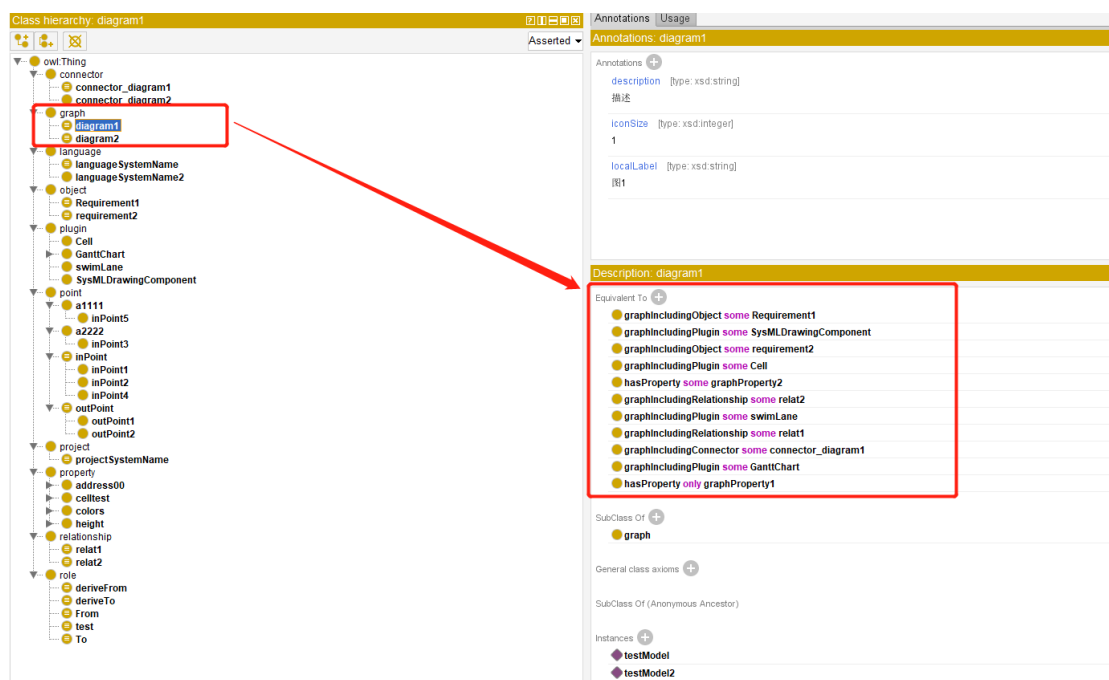


图 4.5 本体中描述“图”

对象（object）：采用 hasProperty 描述对象元模型和属性元模型的关系，采用 linkObjectAndPoint 描述对象和点的关系。需要注意的是，在设计对象元模型时需要同时描述对象元模型上可以附着的点元模型，以及点元模型在建模时表现的数据或物质流动方向。因此，在对象元模型的等价类描述中，需要同时定义上述两种信息。例如，描述对象元模型关联点元模型，且在此关联中点方向为输入点，对应的本体描述方法为：object linkObjectAndPoint some inPoint and pointDirection value “input”。其中，object 为对象元模型，linkObjectAndPoint 为对象属性，inPoint 为点元模型，pointDirection 是一种用于描述对象上点方向的数据属性，若点表现为数据的输入，则后面的字符串为“input”，若表现为数据的输出，则后面的字符串为“output”，若未表现出明显的方向，则后面的字符串为“noDirection”。本体中对象的等价类描述如图 4.6 所示。

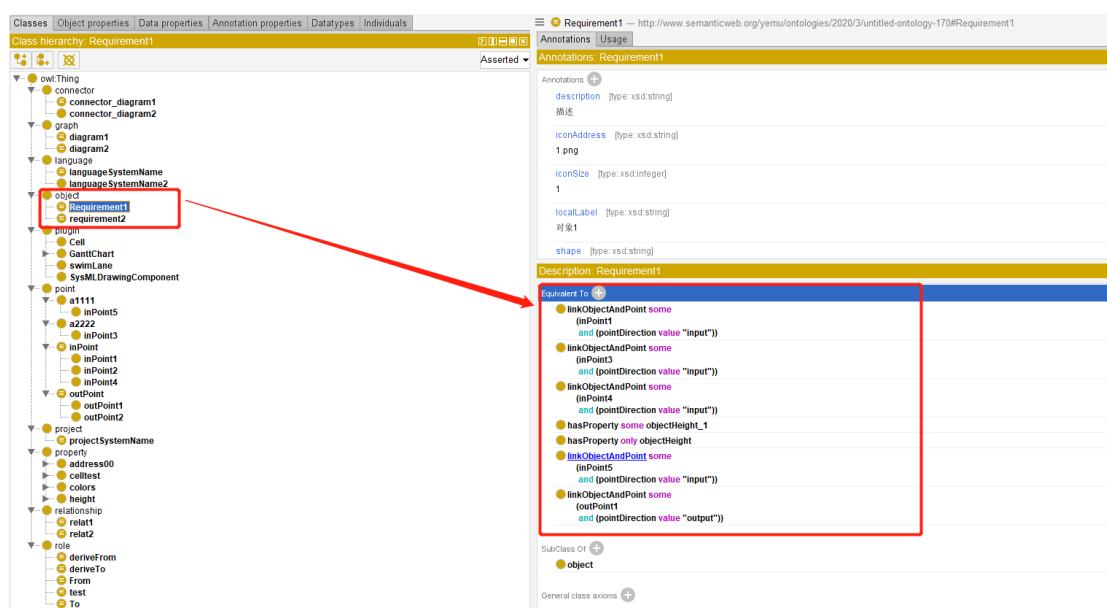


图 4.6 本体中描述“对象”

关系 (relationship): 采用 `hasProperty` 描述关系元模型和属性元模型的关系, 采用 `linkRelationshipAndRole` 描述关系和角色的关系, 本体中关系的等价类描述如图 4.7 所示。

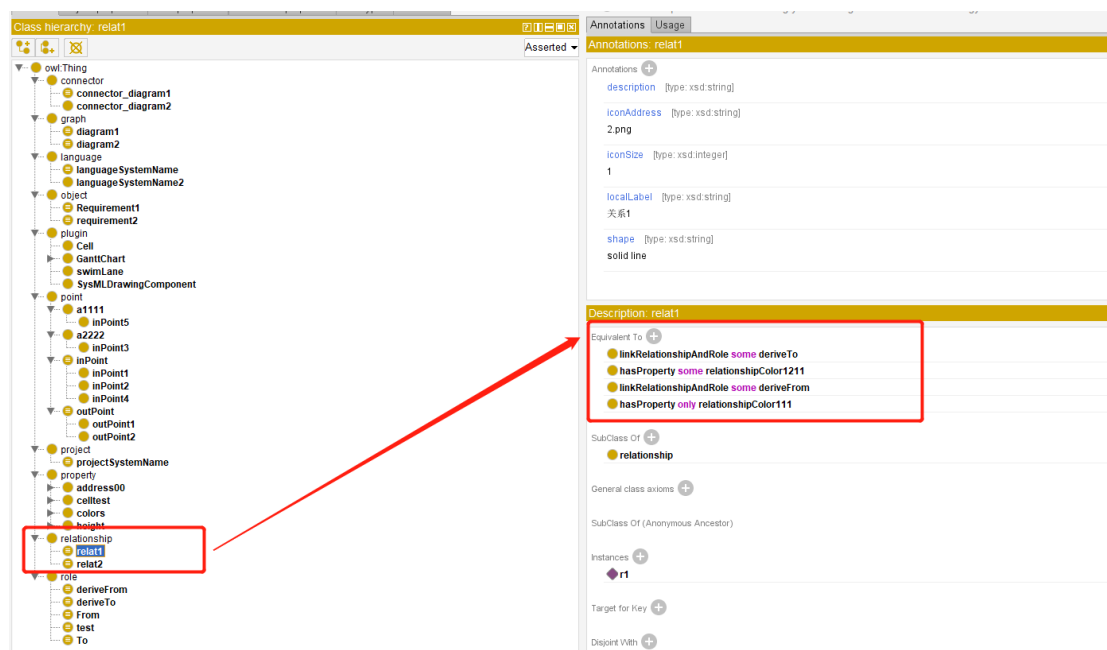


图 4.7 本体中描述“关系”

角色 (role): 采用 `hasProperty` 描述角色元模型和属性元模型的关系。区别于点元模型, 同一点元模型在不同的对象上可能表现为不同的方向, 但角色元模型的方向是其固有属性, 不因关系元模型的变化而变化。例如, “接口点”附着到两

个对象上时，接口的方向可能是输入，也可能是输出；但是“信息流始端”在不同的关系上都表现为信息的输出端，因此，角色元模型的等价类中只需要说明角色元模型关联的属性，方向应该直接在其类声明中进行说明。本体中关系的等价类描述如图 4.8 所示。

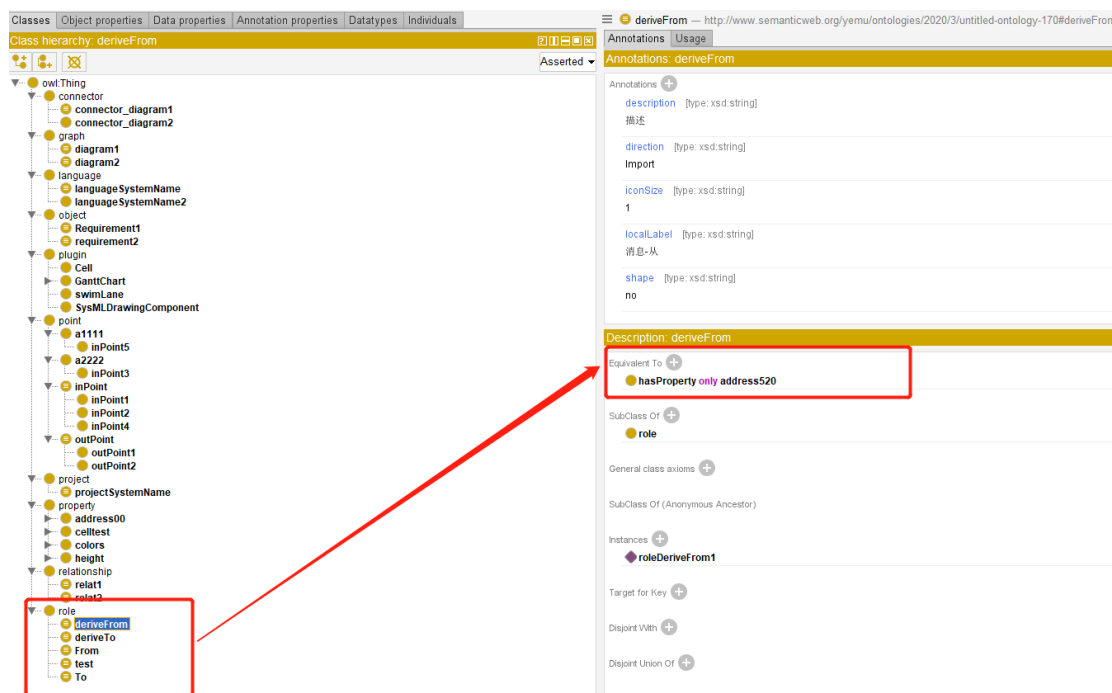


图 4.8 本体中描述“角色”

点 (point): 采用 hasProperty 描述点元模型和属性元模型的关系，点的方向在对象的等价类中进行说明。本体中关系的等价类描述如图 4.9 所示。

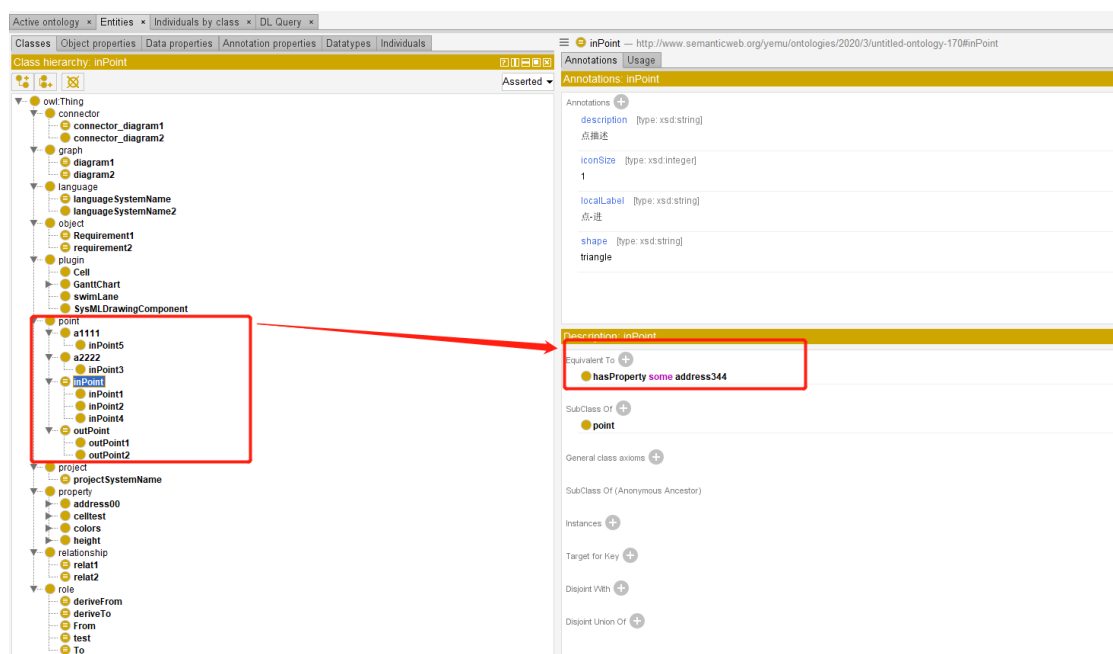


图 4.9 本体中描述“点”

连接器（connector）：一个图元模型对应一个连接器元模型，用于描述图中的连接关系及映射关系，其中连接关系又分为有点元模型参与的建模和无点元模型参与的建模两种。下文分别对三种关系作出说明。

（1）有点元模型参与的建模情况：采用 `linkFromRelationship` 描述连接器元模型和关系元模型的关系（连接器始于关系），采用 `linkRelationshipAndRole` 描述连接器元模型中关系元模型和角色元模型的关系，采用 `linkToObject` 描述连接器元模型和对象元模型的关系（连接器终于对象），采用 `linkObjectAndPoint` 描述连接器元模型中对象元模型和点元模型的关系，采用 `roleBindingPoint` 描述连接器中角色元模型和点元模型的绑定关系。本体中对于有点参与的连接元模型等价类描述如图 4.10 所示。

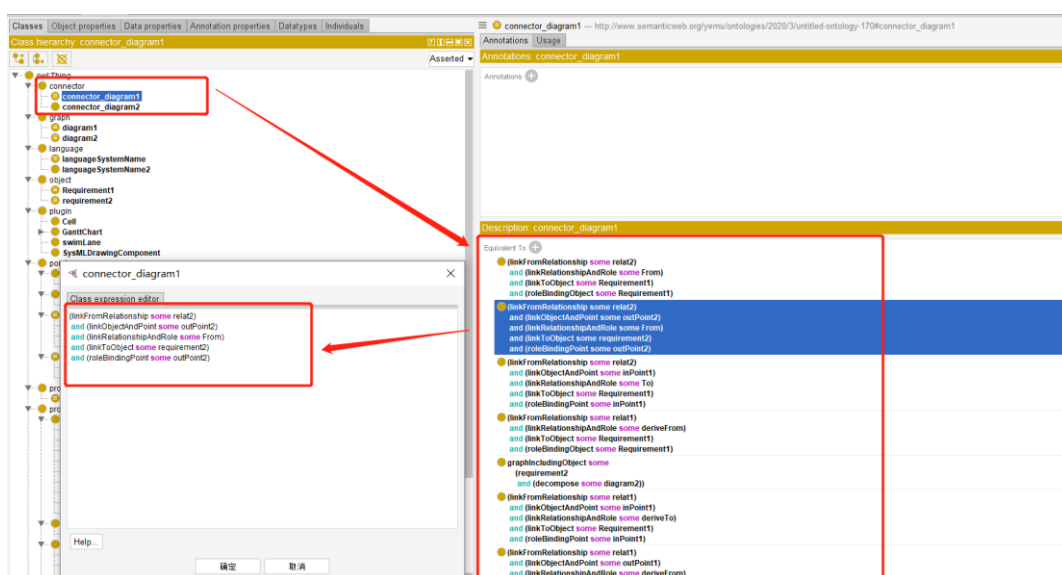


图 4.10 有点元模型参与的连接器描述

(2) 无点元模型参与的建模情况: 采用 `linkFromRelationship` 描述连接器元模型和关系元模型的关系 (连接器始于关系), 采用 `linkRelationshipAndRole` 描述连接器元模型中关系元模型和角色元模型的连接关系, 采用 `linkToObject` 描述连接器元模型和对象元模型的关系 (连接器终于对象), 采用 `roleBindingObject` 描述连接器中角色元模型和对象元模型的绑定关系。本体中对于无点参与的连接器的元模型等价类描述如图 4.11 所示。

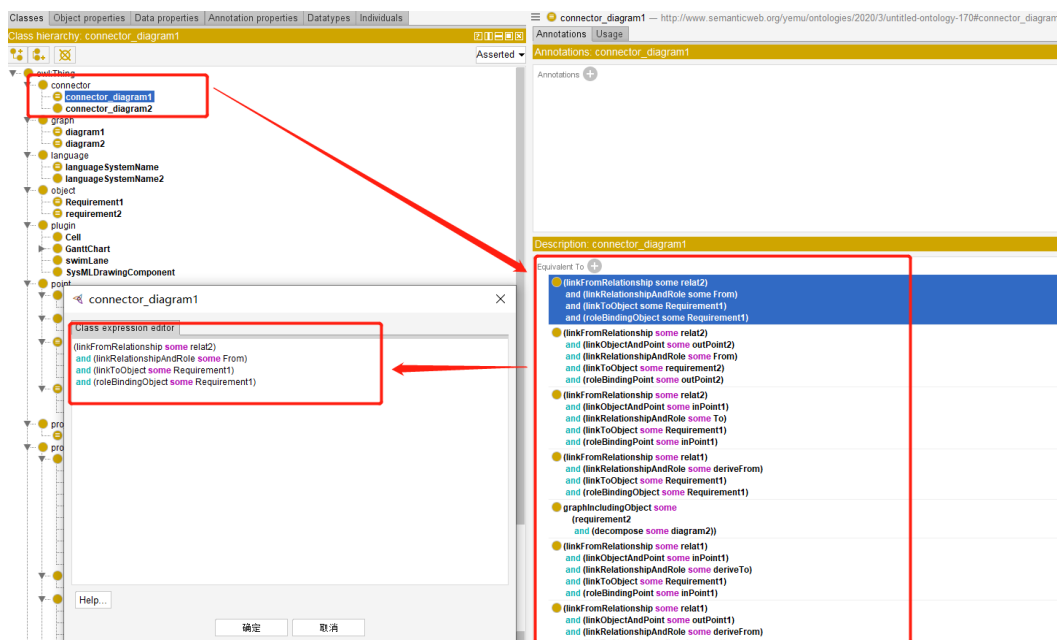


图 4.11 无点元模型参与的连接器的描述

(3) 映射关系：映射关系主要包括图中的对象、点、关系、角色元模型向图元模型的分解和剖视关系。图 4.12 以对象元模型向图元模型的剖视为例进行说明，首先通过 `graphIncludingObject` 描述图中进行映射的对象，再通过 `explode` 描述该对象元模型向图元模型的剖视关系。

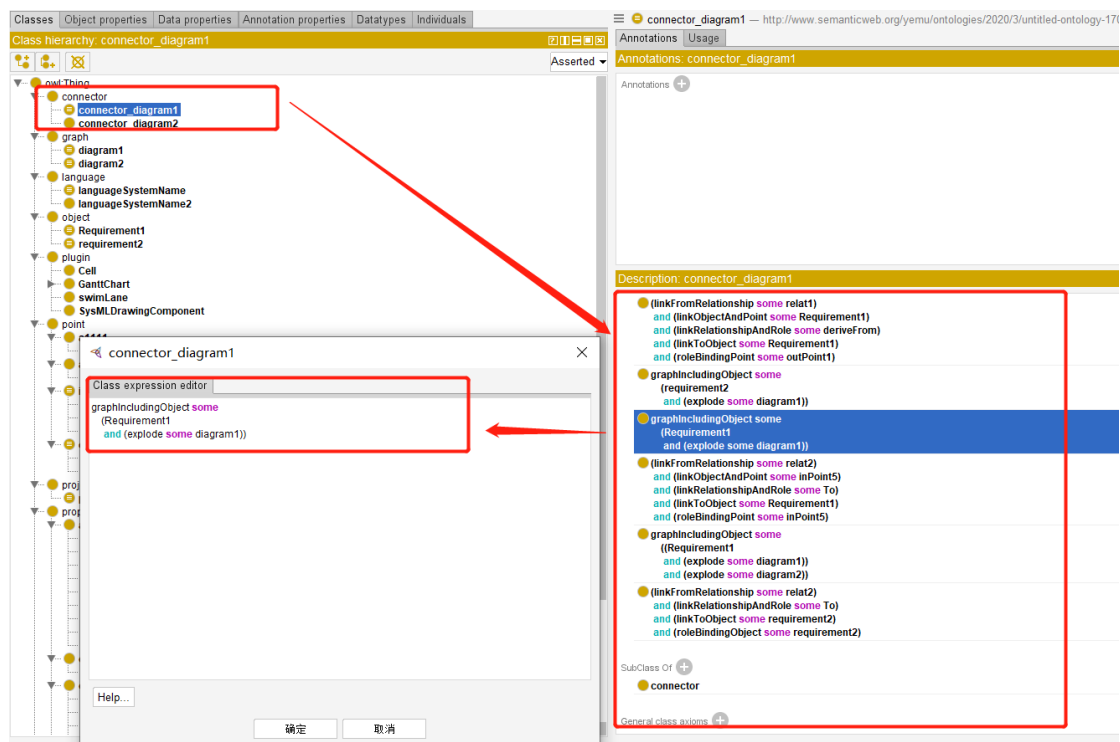


图 4.12 本体中描述映射关系

上述几种等价类描述了元模型之间的连接或映射关系，即建模语言的抽象语法。而建模语言的语义信息则用等价类中的属性约束进行说明。属性约束指类与类之间通过某种关系进行连接时可实例化的数量，包括 **Some**、**Only**、**Min**、**Exactly**、**Max** 五种。以对象的等价类为例进行说明，**some** 可以描述在建模时对象元模型实例化组件通过对象属性可以关联多个点元模型实例化组件，**Only** 指仅可以关联一个点实例化组件，**Min** 指可以关联至少 n 个点的实例化组件，**Exactly** 指可以关联 n 个点的实例化组件，**Max** 指最多可以关联 n 个点的实例化组件。以对象等价类为例说明建模语言的语义约束如图 4.13 所示。

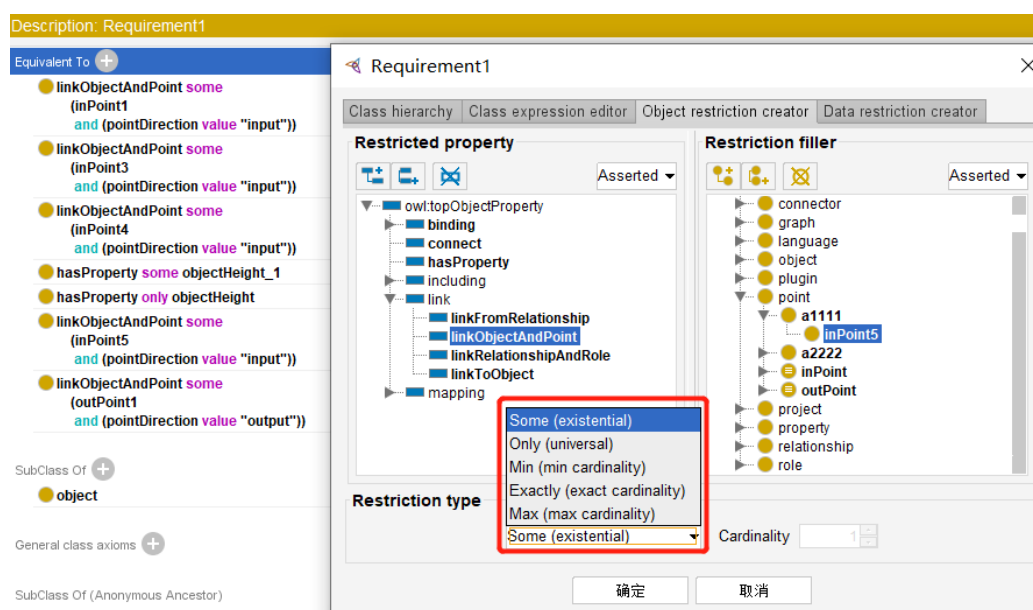


图 4.13 “对象”元模型的语义约束

此外，设计人员根据建模语言进行建模时一般依托于某种建模工具，建模语言的固定语法及建模时的附加信息也需要进行说明，这些信息则通过类的申明属性中进行描述。类申明中主要包括五种属性：元模型的本地名称（localLabel）、元模型的图标（icon）、对元模型的基本说明（description）、元模型的固定语法（shape）、方向（direction）。其中，localLabel 可用于描述项目、语言、图、对象、点、关系、角色、属性八种类的本地名称，本体中的类用于描述元模型的系统名称，系统名称是元模型的唯一标识符，本地名称则是用户对元模型的命名，是可以重复的；icon 则用于描述元模型在建模工具中的图标，不同的元模型有不同的图标；description 则对图、对象、关系、点、角色、属性六种元模型进行备注，用户可以自定义元模型的标注信息；shape 用于描述对象、点、关系、角色四种元模型的固定语法，即建模时元模型的形状，其中，对象的固定语法有矩形、人形、圆角矩形、圆、包图、椭圆、菱形七种，点的固定语法有矩形、圆形、三角三种，关系的固定语法有实线、加粗实线、虚线、加粗虚线四种，角色的固定语法有实心箭头、虚心箭头、箭形、实心菱形、虚心菱形、无形状六种；direction 用于描述角色元模型的方向。

4.2.3 M1 层本体设计方法

如图 4.1 所示，本体的 M1 层采用实例（Individual）用于描述多架构建模过程

中的实例信息，在本体结构中主要包含三部分：采用申明属性描述实例化组件的位置、颜色等信息；采用对象属性断言（Object Property Assertions）描述实例之间的关系；采用数据属性断言（Data Property Assertions）描述属性实例化组件的属性值。其中，申明属性不参与 OWL 的推理机中，对象属性断言和数据属性断言参与 OWL 的推理当中，判断实例间关系是否满足类中定义关系，实例数量是否满足建模语言的语义约束。

与 M2 层中的申明属性不同，在 M1 层中申明属性主要用于描述模型组件在画布中的位置、组件的颜色、组件的显示方式等。模型的申明属性主要包括以下四种：模型组件的本地名称（localLabel）、模型组件在画布中的形状（shape）、模型组件在画布中的位置（location）、模型组件的颜色（color）。用于描述模型组件的申明属性如表 4.3 所示。

表 4.3 模型组件的申明属性

类型	申明属性	描述
本地名称	localLabel	用户自定义的模型组件名称
组件形状	shape	组件在画布中的形状
位置	startLocation	模型组件的起始位置
	endLocation	模型组件的终止位置
	modelSize	模型组件在画布中的大小
	polyLineLocation	关系组件在画布中的折点位置
颜色	fillColor	模型组件的填充颜色
	lineColor	模型组件的边界颜色
	textColor	模型组件中的文本颜色
显示模式	lableDisplay	关系组件名称是否在图中显示
	imageMode	对象组件的图片表示模式，即采用图片表示一个对象组件

propertyMode 对象组件的属性表示模式，即在框图中表示对象的属性

decompositionMode 对象组件的分解模式，直接表示对象到图的分解

此外，采用对象属性断言表示模型组件之间的关系，需要注意的是，实例的对象属性申明和等价类中的描述保持一致。例如，在对图的实例进行对象属性申明时，采用 `hasProperty` 描述图和属性组件的关系；采用 `graphIncludingObject` 描述图和对象组件的关系；采用 `graphIncludingRelationship` 描述图和关系组件的关系；采用 `graphIncludingConnector` 描述图元模型和连接器之间的关系，如图 4.14 所示。

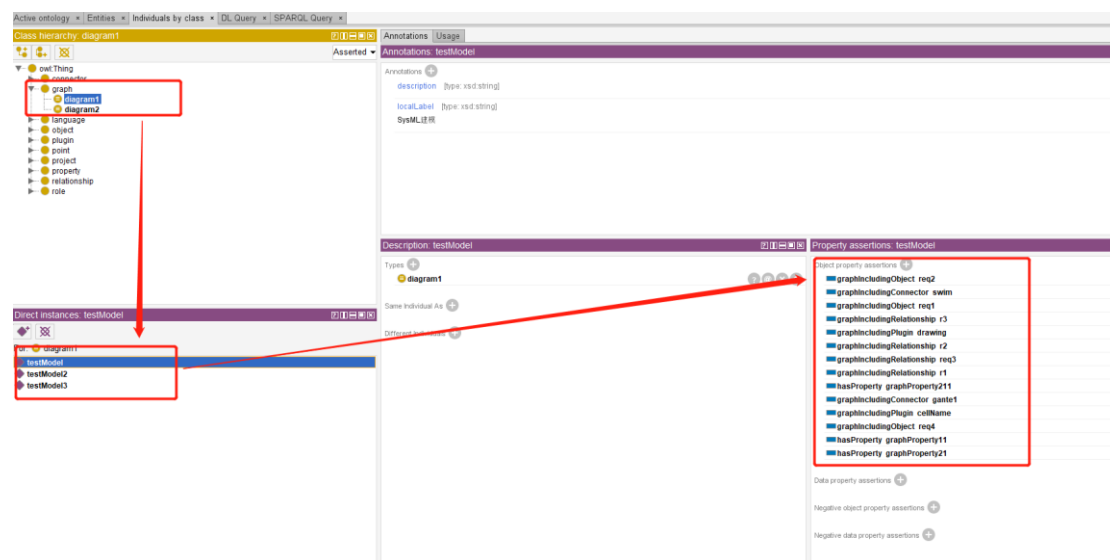


图 4.14 图实例化组件申明

采用数据属性断言来表达建模中的自定义属性值。需要注意的是，只有属性元模型的实例化组件可以采用数据属性断言来描述属性值，其余几类模型组件通过 `hasProperty` 与属性组件进行关联，再给对应的属性组件进行赋值，如图 4.15 所示。

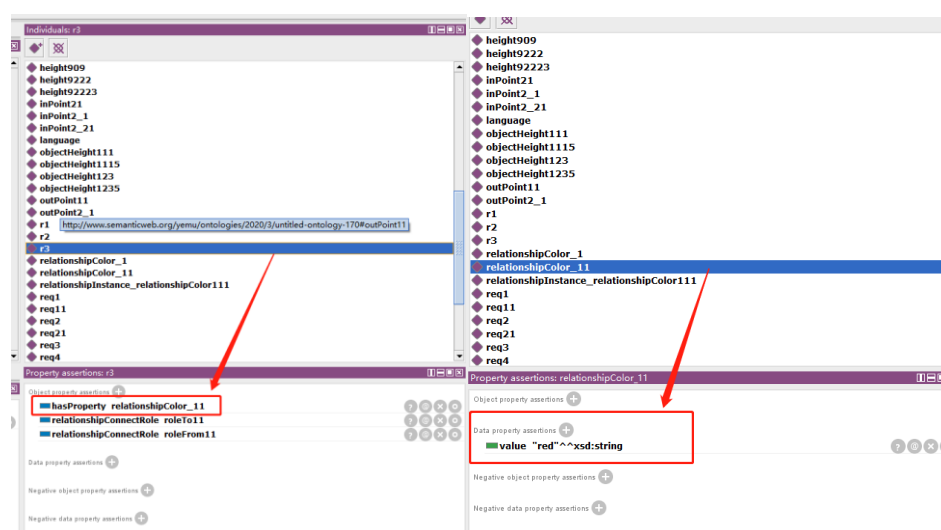


图 4.15 数据属性断言描述自定义属性值

4.3 本体接口设计

本文基于 java 编程语言来实现多架构模型与本体框架的互相转化，采用 Protégé OWL API 完成本体的生成及读取，具体的接口如表 4.4 所示。

表 4.4 接口设计

owl	API
本体读取	
创建本体模型	BasicConfigurator.configure();
通过 model 包中的 ModelFactory 创建本 体模型，使用 owl 语 言创建本体模型	OntModel ontModel = ModelFactory.createOntologyModel(OntModelSpec.OWL_ MEM);
读取文件路径	ontModel.read("file/Goppr.owl");
读取文件路径	ontModel.read(new FileInputStream("path"), "");
获取本体中的类	ontModel.listClasses();
获取类名	String className = c.getModel().getGraph().getPrefixMapping().shortForm(c.g etURI());
获取父类	c.listSuperClasses();
获取父类类名	c.listSuperClasses().getURI();
获取子类	c.listSubClasses();

获取子类类名	c.getModel().getGraph().getPrefixMapping().shortForm(sb. getURI());
迭代显示与类有关的 属性	for (Iterator<OntProperty> ipp =c.listDeclaredProperties(); ipp.hasNext();)
迭代显示所有实例 获取实例的属性值	for (Iterator oi = c.listInstances(); oi.hasNext();) String pryValueOfIndiv = io.getPropertyValue(classPryOfIndiv).toString();
获取数据属性	pryValueOfIndiv.substring(pryValueOfIndiv.indexOf("#") + 1)
本体生成	
创建本体模型	ModelFactory.createOntologyModel(OntModelSpec.OWL_ MEM);
新建类	ontModel.createClass();
创建父类	ontClass.addSuperClass();
创建子类	ontClass.addSubClass();
创建 annotation property	AnnotationProperty p = ontModel.createAnnotationProperty();
给 class 添加上述属 性的属性值	ontClass.addProperty(p,"");
新建 object property 给 object property 添 加阈值和约束	ObjectProperty g =ontModel.createObjectProperty(); g.setDomain(); g.getRange();
新建 dataType property	DatatypeProperty q = ontModel.createDatatypeProperty();
新建实例	Individual i = ontModel.createIndividual("实例名 “ , class);
为实例添加属性值	i.setDatatypeValue(q,"");
生成本体文件	ontModel.write(new FileOutputStream);

在设计好相关接口后将其应用于 MetaGraph 中，如图 4.16 所示。在多架构建模工具 MetaGraph 中构建模型后，将其作为输入导入 OWL 生成器中生成相应的本体文件；也可以将该框架下的本体文件作为输入加载到 OWL 解析器中，通过解析过程在建模工具中展示建模信息。

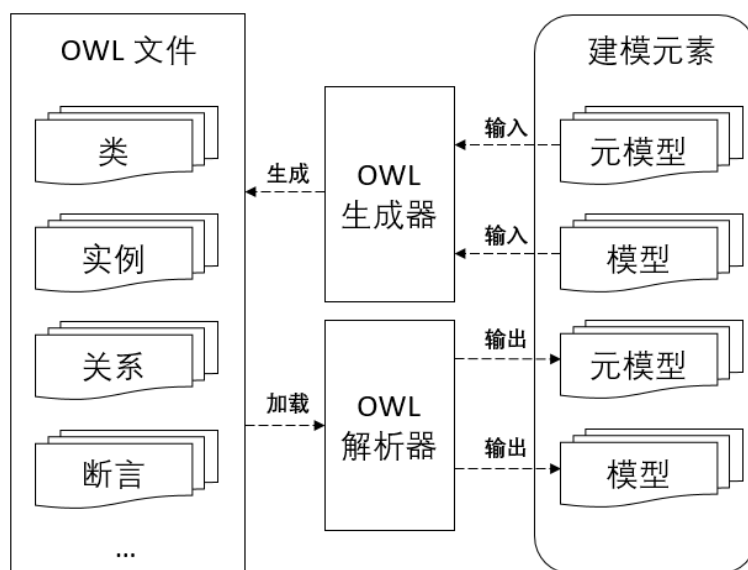


图 4.16 本体转换过程

通过多架构模型及本体之间的接口设计，可以实现异构模型信息与本体之间的互相转换，进一步地，在建模工具 MetaGraph 中可以实现模型与本体的自动转换，提高设计效率；设计人员在此基础上可以提取模型中的相关信息，以便在后续的系统验证阶段提供数据支持。

4.4 本章小结

系统设计过程中会产生大量数据，这些数据采用不同的模型进行表达。为了实现数据的一致性表达，本章在多架构建模的基础上提出一种本体的设计方法，首先在多架构建模的 M3-M0 框架中实现建模信息和本体元素的一一映射，分别在元元模型层、元模型层、模型层中将建模元素转化为本体表达，此基础上实现模型数据到本体文件的一致性描述；在此基础上基于 Java 语言实现多架构模型到本体的接口设计，实现两者的自动转化及互操作。

第5章 本体设计方法验证

5.1 引言

本文主要解决 MBSE 实践过程中的数据交互问题，多架构建模方法可以从更高层次的抽象度上实现模型的构建，基于多架构建模方法的本体框架则可以实现模型信息和本体文件的互相转化。多架构建模方法的可用性及高效性在第二章中做出描述，本章节则分别通过演绎法和归纳法对本体框架的可行性进行验证。演绎法和归纳法是科学研究中两种主要的逻辑思维方法，其中，演绎法可以从许多个例中获得比较具备概括性的规则；归纳法可以通过既定的普遍性结论推导出个别性结论。在本章节中，通过 SysML、UML、DODAF 多种模型库和本体文件的相互转化归纳验证本体转化方法的可行性，通过 SPARQL 查询语句和 SQWRL 推理语言对本体文件进行演绎验证，从而证明生成的本体文件逻辑正确性及可靠性。

5.2 归纳法验证

本文采用 MetaGraph 建模工具进行建模，该工具基于多架构建模方法进行建模，在此基础上通过上文设计的接口可以实现模型与本体文件的互转换。采用归纳法对该本体建模框架进行验证，本文基于 MetaGraph 设计了 SysML、Arcadia、UPDM 等多种模型库，并将其转化为本体文件，然后将本体文件导入到建模工具中，通过最终模型库是否与之前设计的模型库保持一致验证该本体框架是否合理。验证方法如图 5.1 所示。

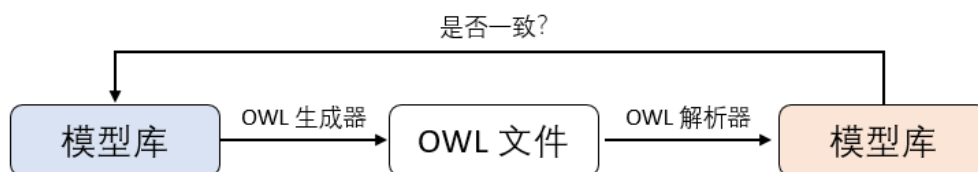


图 5.1 基于归纳法的本体验证

5.2.1 SysML 模型库转化

如前文所述，SysML 建模语言通过九种图实现对系统的描述。基于 MetaGraph 建模工具将 SysML 中关键元素转化为图、对象、点、角色、关系、属性六种元素，

并通过连接器实现六种元素之间的约束设计。按照 OMG 标准, 基于 MetaGraph 工具设计的 SysML 模型库可以描述 9 种图元模型、73 种对象元模型、11 种点元模型、10 种属性元模型、31 种关系元模型以及 62 种角色元模型, 如图 5.2 所示为 SysML 模型库中六种类型的元模型。

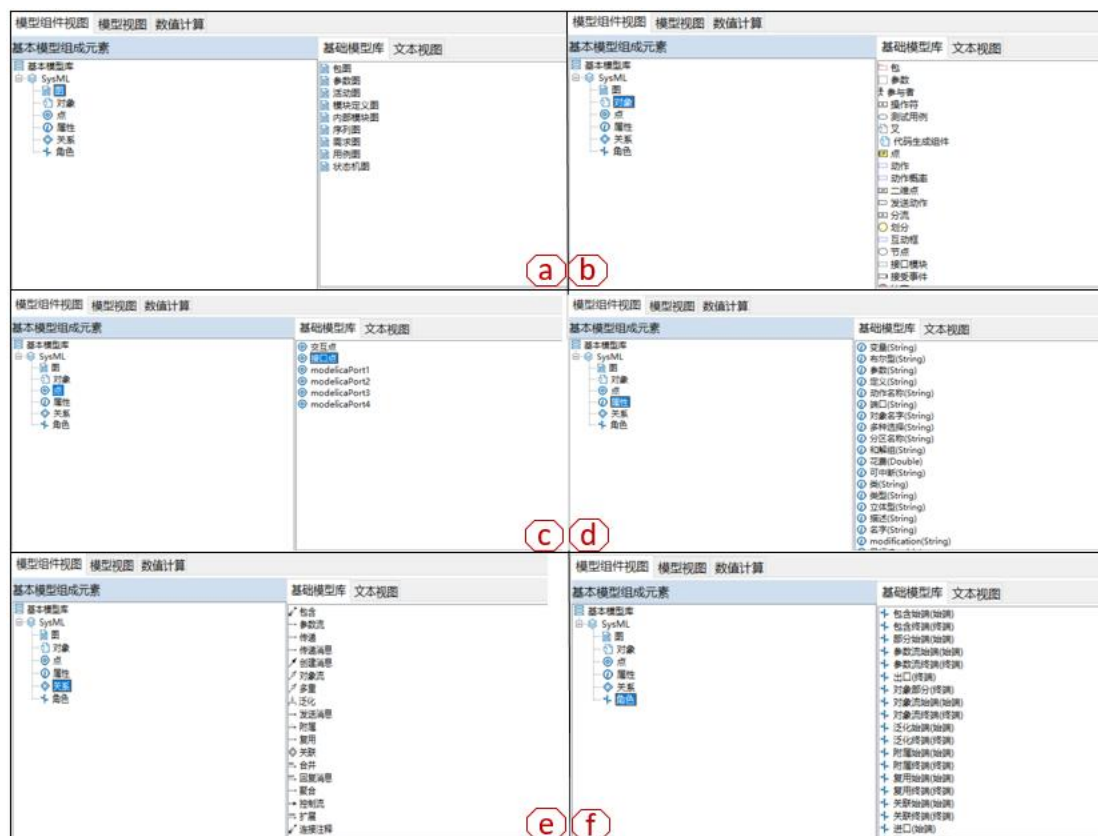


图 5.2 SysM 模型库

如图 5.2 所示, SysML 模型库构建符合 OMG 标准, 并且与 MagicDraw 建模工具作出对比。其中, 图 a-f 所示分别为 SysML 中的图、对象、点、属性、关系、角色元模型, 图 5.3 所示以需求图为例说明 SysML 中连接关系的实现方法。

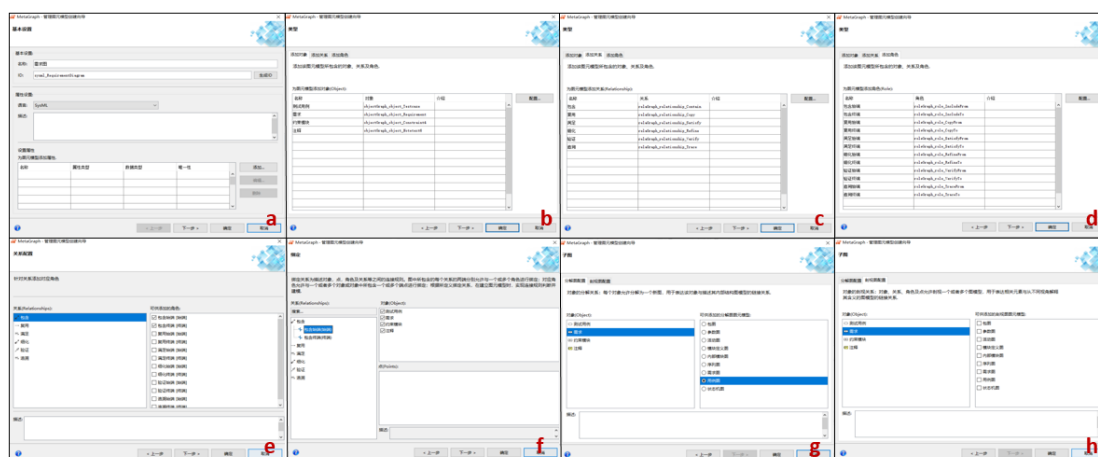


图 5.3 SysML 需求图设计方法

其中，图 a 对图元模型的基本信息进行设计，包括图元模型的名称、ID、属性、描述；图 b 表示需求图中可以包含“测试用例”、“需求模块”“约束模块”、“注释”四种类型的对象元模型；图 c 表示需求图中可以包含“包含”、“满足”、“复用”、“细化”、“验证”、“追溯”六种关系元模型；图 d 表示需求图中可以包含“包含始端”、“包含终端”、“满足始端”、“满足终端”、“复用始端”、“复用终端”、“细化始端”、“细化终端”、“验证始端”、“验证终端”、“追溯始端”“追溯终端”十二种角色元模型；图 e 表示需求图中的关系元模型配置方法，如“包含”关系两端可以添加“包含始端”和“包含终端”两种角色元模型；图 f 表示需求图中的绑定关系，即关系元模型两边的角色元模型和对象元模型或者对象元模型上的点进行绑定，在建模时，对象间的连接规则如果不符合绑定规则就会提示建模错误；图 g 和图 h 分别表示需求图中的元素与图元模型之间的分解和剖视关系。

在工具中建立好 SysML 模型库后，按照设计好的接口将其转化为本体文件如图 5.4 所示。

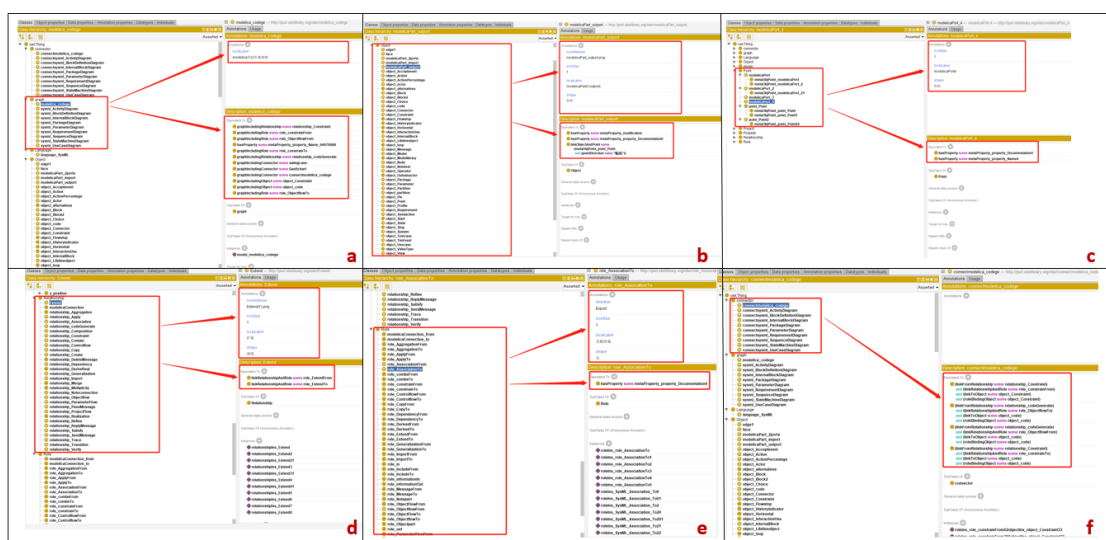


图 5.4 SysML 模型库本体设计

如图 a 所示为 SysML 中图元模型的相关描述，通过申明属性描述图元模型的本地名称等标注属性，通过等价类描述图元模型与对象、点、关系、角色、属性元模型的关系；图 b 所示为 SysML 中对象元模型的相关描述，通过申明属性描述其图标大小、图标本地路径、元模型本地名称、元模型的固定语法等信息，通过等价类描述对象元模型上添加点元模型后，点所表示的数据或物质流动方向，以及对象元模型与属性元模型的关系；图 c 所示为 SysML 中点元模型的相关描述，通过申明属性描述其图标大小、图标本地路径、点元模型的固定语法，通过等价类描述点元模型与属性元模型的关系；图 d 所示为 SysML 中关系元模型的相关描述，通过申明属性描述其固定语法、本地名称等信息，通过等价类描述关系元模型两端可以添加的角色元模型种类，以及关系元模型和属性元模型的关系；图 e 所示为 SysML 中角色元模型的相关描述，通过申明属性描述其本地名称、固定语法、角色所代表的方向等信息；通过等价类描述角色元模型和属性元模型的关系；图 f 所示为 SysML 中图元模型对应的连接器元模型的相关描述，通过等价类描述连接器中关系、角色、点、对象之间的连接关系，以及图中元素与图元模型的分解和剖视关系。

经过验证，基于该框架设计的本体中，class:graph 下包括九个子类用于代表九种图元模型，class:object 下包括 73 个子类代表 73 种对象元模型，class:point 下包括 11 个子类代表 11 种点元模型，class:property 下包括 10 个子类代表 10 种属性元模型，class:relationship 下包括 31 个子类代表 31 种关系元模型，class:role 下包括 62 个子类代表 62 种角色元模型。此外，在 MetaGraph 中可以实现该模型库和

本体文件的互相转换，在转换过程中保持模型数据完整与正确。

5.2.2 多种模型库转化

与 SysML 模型库类似，本文在多架构建模方法的基础上实现了 Arcadia、UPDM、BPMN、EAST-ADL、UML 等模型库的设计与本体生成，下文以 Arcadia 和 UPDM 两种模型库为例对本体框架进行验证。其中，Arcadia 方法是法国 Thales 公司推出的建模方法，近些年来在工业界中被广泛应用于系统架构设计和系统运行场景分析；UPDM 是 OMG 组织建立的一个针对美国国防部体系架构框架（DoDAF）和英国国防部体系架构框架（MoDAF）的 UML 行业建模标准。

Arcadia 建模方法遵循的建模流程是运行分析-系统分析-逻辑架构-物理架构-产品结构分解，因此在需要对系统架构进行分析设计时比 SysML 更容易被设计人员接受。基于多架构建模方法设计 Arcadia 模型库时，按照上述运行流程构建图元模型，每个图元模型中的关系元模型和对象元模型如表 5-1 至表 5-4 所示。

表 5-1 运行分析元模型表

图	关系	对象
运行实体分解	包含	实体、角色
运行能力分析	包含、参与、能力泛化、扩展、 实体交互	实体、角色、 能力
运行活动交互	活动交互	活动
运行架构图	包含、活动交互、实体交互	活动、角色、 实体
运行活动分解	包含	活动

表 5-2 系统分析元模型表

图	关系	对象
系统角色	包含、角色泛化	角色、系统
系统功能流图	功能交互	系统功能、角色功能
系统架构图	包含、实体交互、 功能交互	系统、实体、角色、 角色功能、系统功能
系统功能分解图	包含	系统功能、角色功能

表 5-3 逻辑架构元模型表

图	关系	对象
逻辑功能流图	功能交互	逻辑功能、角色功能
逻辑组件分解图	包含	逻辑组件
逻辑架构图	包含、实体交互、	系统、实体、角色、逻辑组

功能功能分解图	功能交互 包含	件、角色功能、逻辑功能 逻辑功能、角色功能
表 5-4 物理架构元模型表		
图	关系	对象
物理功能流图	功能交互	物理功能、角色功能
物理组件分解图	包含	逻辑组件
物理架构图	包含、实体交互、 功能交互	系统、实体、角色、物理 组件、角色功能、物理功 能
物理功能分解图	包含	物理功能、角色功能

按照图元模型中包含的对象元模型和关系元模型设计 Arcadia 模型库如图 5.5 所示。其中，图 a-f 分别表示 Arcadia 模型库中的图、对象、点、属性、关系、角色六种类型的元模型种类。

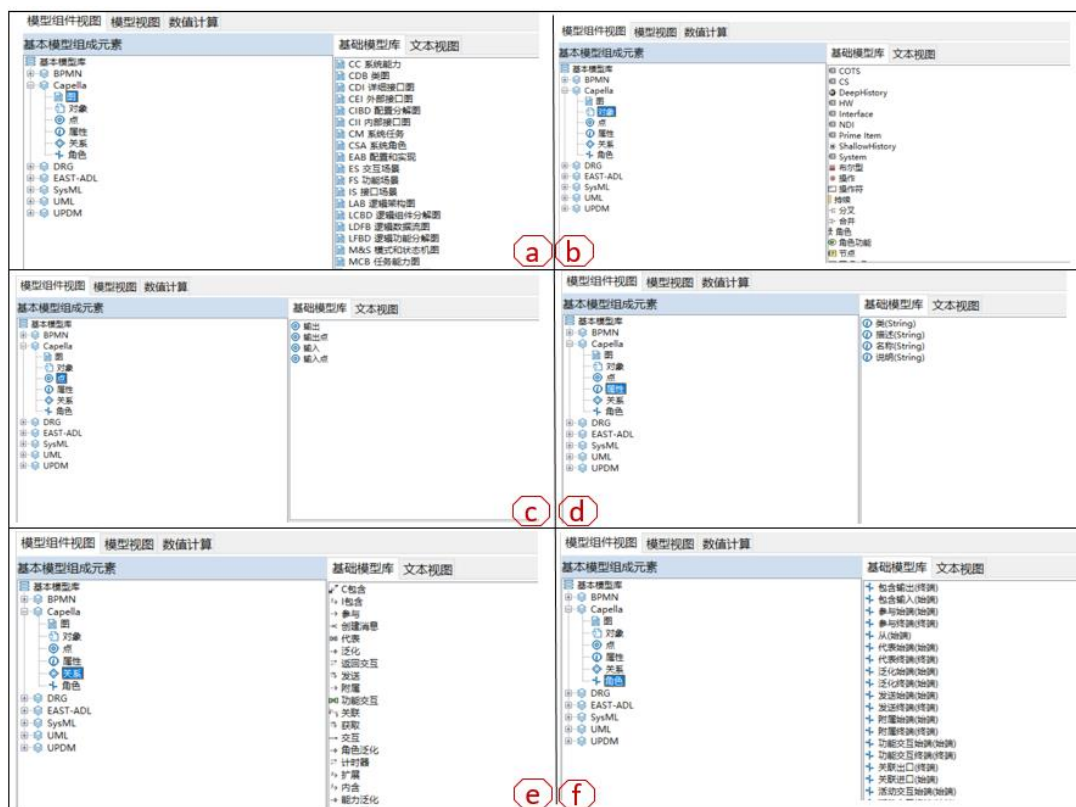


图 5.5 Arcadia 模型库设计

UPDM 则是通过全景视点、数信视点、标准视点、能力视点、作战视点、服务视点、系统视点、项目视点这八种视点，采用 52 种图元模型对体系作战中的场景进行描述。构建方法与 Arcadia 方法类似，其设计结构如图 5.6 所示，图 a-f 分别表示 UPDM 模型库中的图、对象、点、属性、关系、角色六种类型的元模型种

类。

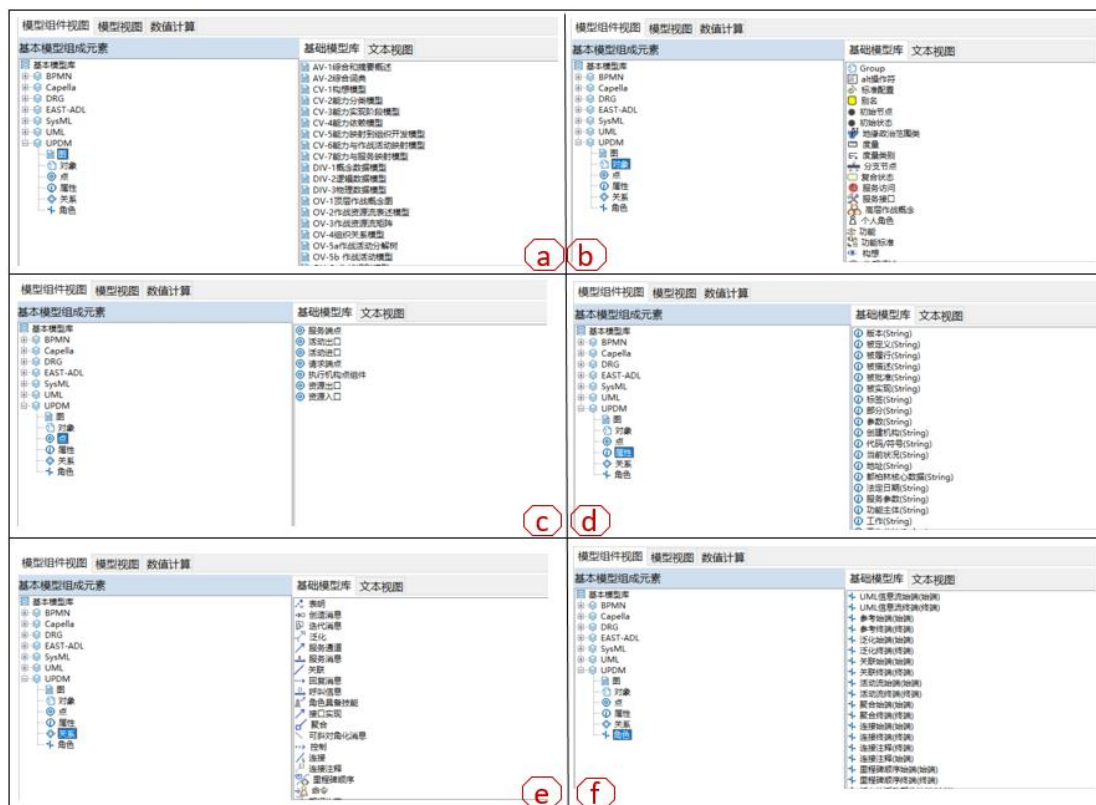


图 5.6 UPDM 模型库设计

与其类似，基于多架构建模方法分别构建 BPMN、EAST-ADL、UML 三种模型库，最终将其转化为一份本体文件，该文件可以同时实现 SysML、Arcadia、UPDM、BPMN、EAST-ADL、UML 六种模型库的表达，如图 5.7 所示，该文件中包括 3014 个类，除去项目、语言、图、对象、点、关系、角色、属性、连接器九个类代表元元模型，其余 3005 个类表示上述模型库中包含的 3014 种元模型。

Ontology metrics:	
Metrics	
Axiom	79389
Logical axiom count	18274
Declaration axioms count	3070
Class count	3014
Object property count	32
Data property count	2
Individual count	9401
Annotation Property count	35
Class axioms	
SubClassOf	3005
EquivalentClasses	5806
DisjointClasses	0
GCI count	0
Hidden GCI Count	719
Object property axioms	
SubObjectPropertyOf	26
EquivalentObjectProperties	0
InverseObjectProperties	0
DisjointObjectProperties	0
FunctionalObjectProperty	0
InverseFunctionalObjectProperty	0
TransitiveObjectProperty	0

图 5.7 六种模型库转化本体指标

5.3 演绎法验证

上节通过六种模型库的构建及本体转换验证了本体文件表达的数据完整且有效，本节则通过本体推理机验证该本体框架的逻辑是否正确。采用本体标准查询语言 SPARQL(Simple Protocol and RDF Query Language, SPARQL)对本体框架中关键元素进行查找及分析；采用本体推理语言 SQWRL(Semantic Query-enhanced Web Rule Language, SQWRL)对本体中的规则进行推理分析，最终验证本体框架中构建的规则正确，并且集成描述的信息完整。

5.3.1 SPARQL 关键元素查询

本文设计的本体框架是一种基于 OWL 的本体结构，而这种结构是基于三元组构建的。SPARQL 查询语言可以在这种三元组的基础上进行查询，简单而言，在采用 SPARQL 查询语言进行本体信息查询时，首先在本体中调取三元组的已知条件，然后根据本体中设计的公理判断是否有满足条件的个体，如果有的话将结果输出，最终根据输出结果与实际建模情况进行对比，从而验证该本体设计是否正确。

基于多架构建模方法生成的本体中，主要包含项目、语言、图、对象、关系、

角色、点、属性、连接器九种重要元素，因此本文主要针对上述九种元素进行查询。包括以下几种查询：

- (1) 查询某项目可以基于哪几种建模语言实现辅助设计；
- (2) 查询元模型对应的元元模型；
- (3) 查询图中包含的对象、关系；
- (4) 查询关系两端可关联的角色；
- (5) 查询对象两端可关联的点；
- (6) 查询具备某属性的对象；

SPARQL 查询是基于个体间的关系进行的，而本文中设计的本体框架已经将个体之间的关系明确规定下来，即本体中的对象属性，因此下述 SPARQL 查询语句可以用于查询该框架下的所有本体文件。

```

1. PREFIX owl: <http://www.w3.org/2002/07/owl#>
2. PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
3. PREFIX xsd: http://www.w3.org/2001/XMLSchema#
4. //查询项目可以基于哪几种建模语言进行设计
5. select ?project ?language
6. where{
7.   ?project < http://www.semanticweb.org/yemu/ontologies/2019/7/untitled-ontology-
163#projectIncludingLanguage > ?language
8. }
9. //查询元元模型对应的元模型
10. select ?metaModel ?meta-metaModel
11. where {
12.   ? metaModel rdfs:subClassOf ? meta-metaModel
13. }
14. //图中包含的对象
15. select ?graph ?object
16. where {
17.   ?graph <http://www.semanticweb.org/yemu/ontologies/2019/7/untitled-ontology-
163#graphIncludingObject> ?object
18. }
19. //图中包含的关系
20. select ?graph ?relationship
21. where {
22.   ?graph <http://www.semanticweb.org/yemu/ontologies/2019/7/untitled-ontology-
163#graphIncludingRelationship> ? relationship
23. }
24. //关系两端可关联的角色
25. select ?relationship ?role
26. where {
27.   ?graph <http://www.semanticweb.org/yemu/ontologies/2019/7/untitled-ontology-
163#graphIncludingRelationship> ? relationship;
28.   ?relationship <http://www.semanticweb.org/yemu/ontologies/2019/7/untitled-ontology-
163#linkRelationshipAndRole> ?role
29. }
30. //对象两端可以关联的点

```

```

31. select ?object ?point
32. where {
33. ? graph <http://www.semanticweb.org/yemu/ontologies/2019/7/untitled-ontology-163#graphIncludingObject> ? object;
34. ? object <http://www.semanticweb.org/yemu/ontologies/2019/7/untitled-ontology-163#linkObjectAndPoint> ?point
35. }
36. //查询具备某属性的对象
37. select ?object ?property
38. where {
39. ? graph <http://www.semanticweb.org/yemu/ontologies/2019/7/untitled-ontology-163#graphIncludingObject> ? object;
40. ? object <http://www.semanticweb.org/yemu/ontologies/2019/7/untitled-ontology-163#hasProperty> ?property }

```

5.3.2 SQWRL 推理验证

SQWRL 是一种基于 SWRL 的推理语言, 首先定义一系列用于查询的运算符, 然后通过本体中的 SWRL 规则实现查询。其中, SWRL 是一种常用的规则建立语言, 用于定义本体中的规则类知识。SWRL 规则基于 Horn 逻辑, 是一阶逻辑的可判定子集, 是由隐含规则 (Imp)、原子 (Atom)、变量 (Variable) 和内置规则 (Build-in) 等部分组成。一个比较典型的 SWRL 规则如下所示:

$$Q_1 \wedge Q_2 \wedge Q_3 \wedge \dots Q_{n-2} \wedge Q_{n-1} \wedge Q_n \rightarrow P_x$$

在这条 SWRL 规则中, $Q_1, Q_2 \dots Q_{n-1}, Q_n$ 和 P_x 都表示原子, 每个原子中包含变量, 原子之间通过交运算符进行连接, 表示规则的主体, P_x 表示规则的头部。上述规则的含义为: 当 $Q_1, Q_2 \dots Q_{n-1}, Q_n$ 全部发生时, 则 P_x 一定会发生。

SQWRL 是在 SWRL 规则语言的基础上对内置规则进行扩展而成的语义推理语言, 因此其表示方法与 SWRL 类似。与其不同的是, SQWRL 是根据归则生成相关推理结果, 其核心运算符是 `sqwrl:select`。例如, 在对图中对象之间的关系进行推理时, 可设计推理语言如下:

```

1. graph(?Graph) ^ connector(?Connector) ^ relationship(?Relationship) ^ object(?ObjectInput) ^ object(?ObjectOutput) ^ graphIncludingConnector(?Graph, ?Connector) ^ linkFromRelationship(?Connector, ?Relationship) ^ linkToObject(?Connector, ?ObjectInput) ^ linkToObject(?Connector, ?ObjectOutput)
2. -> sqwrl:select(?Relationship, ?ObjectInput, ?ObjectOutput)

```

其中 ? 后连接变量, 表示该变量是类的一个实例, 如上述推理语句中表示 Graph 是 graph 的一个实例变量, ObjectInput 和 ObjectOutput 分别是 object 的两个实例变量; graphIncludingConnector、linkFromRelationship 和 linkToObject 分别表示图和连接器、连接器和关系与对象之间的关系; 最终查询一个关系组件两端连

接的对象组件。从上式可以看出，SQWRL 推理语句包括两部分：右边是一个 SQWRL 的核心运算符，可以是一个或者多个变量，用这些变量构成查询结果表的各个列；左边是一个类似于 SWRL 的规则构造语句，将概念原子和关系通过布尔与运算联系起来。此外需要注意的是，由于 OWL 以开放世界假设为前提，认为知识是无限集合，因此 SQWRL 只支持布尔与（ $\wedge$ ）运算，并不支持布尔或（ $\vee$ ）和布尔非（ $\neg$ ）。在本文中采用 SQWRL 方法对以下几种情况进行推理验证：

- (1) 验证关系两边对象是否正确；
- (2) 验证关系的输入端是否正确；
- (3) 验证图中绑定关系是否正确；

与 SPARQL 类似，SQWRL 推理语言也是基于对象属性进行查询及推理的，因此下述推理语言同样适用于全部基于该框架设计的本体文件。

- (1) 验证关系两边对象是否正确：

```
1. graph(?Graph) ^ connector(?Connector) ^ relationship(?Relationship) ^ object(?
  ObjectInput) ^ object(?ObjectOutput) ^ graphIncludingConnector(?Graph, ?Connector)
  ^ linkFromRelationship(?Connector, ?Relationship) ^ linkToObject(?Connector,
  ? ObjectInput) ^ linkToObject(?Connector, ? ObjectOutput)
2. -> sqwrl:select(?Relationship, ?ObjectInput, ? ObjectOutput)
```

- (2) 验证关系的输入端是否正确：

```
1. graph(?Graph) ^ connector(?Connector1) ^ connector(?Connector2) ^ relationship
  (?Relationship) ^ object(?ObjectInput) ^ object(?ObjectOutput) ^ graphIncludingC
  onnector(?Graph, ?Connector1) ^ graphIncludingConnector(?Graph, ?Connector2) ^
  linkFromRelationship(?Connector1, ?Relationship) ^ linkFromRelationship(?Conne
  ctor2, ?Relationship) ^ linkToObject(?Connector1, ? ObjectInput) ^ linkToObject(?
  Connector2, ? ObjectOutput) ^ connect(?Connector1, ? Connector2)
2. -> sqwrl:select(?Relationship, ?ObjectInput)
```

- (3) 验证图中的绑定关系是否正确：

```
1. graph(?Graph) ^ connector(?Connector) ^ relationship(?Relationship) ^ object(?
  ObjectInput) ^ object(?ObjectOutput) ^ role(?FromRole) ^ role(?ToRole) ^ graphInc
  ludingConnector(?Graph, ?Connector) ^ linkFromRelationship(?Connector, ?Relatio
  nship) ^ linkToObject(?Connector, ? ObjectInput) ^ linkToObject(?Connector, ? Obj
  ectOutput) ^ linkRelationshipAndRole(?Relationship, ? FromRole) ^ linkRelationshi
  pAndRole(?Relationship, ? ToRole) ^ roleBindingObject(?FromRole, ? ObjectOutput)
  ^ roleBindingObject(?ToRole, ? ObjectInput)
2. -> sqwrl:select(?Relationship, FromRole, ? ObjectOutput, ? ToRole, ? ObjectInput)
```

将 SPARQL 和 SQWRL 查询推理出的信息与建模信息对比，可证明本体设计方法具备逻辑表达的能力，即本体表达的逻辑正确性及可靠性。

5.4 本章小结

本章在前文设计的本体框架的基础上进行验证，首先基于 MetaGraph 建模工具实现多种模型库转化为本体文件，再将本体文件转化为模型库，可以通过生成的模型库与标准模型库一致证明本体框架的理论正确性；然后通过 SPARQL 和 SQWRL 对本体文件进行推理，将查询到的数据与建模数据进行对比从而验证该本体框架的逻辑正确性，以及在工程案例中的可用性。

第 6 章 基于无人平台的多架构设计与本体转化案例

无人平台是一种在地面行驶的能执行特定作战任务的复杂装备，是机械化、信息化、电子化和智能化融合的产物。随着高新技术的发展和作战场景的复杂化，其系统复杂性也随之增加^[62]。与一般武器装备相比无人平台有两大特点：无人平台在接受到外界信息后需要自主或半自主地完成决策过程以处理不确定性问题；无人平台一般在复杂极端的作战环境下执行任务，对其控制系统的精确度提出了更高的要求。因此，无人平台研发过程中会面临更大的挑战：

（1）无人平台涉及领域知识繁杂，如机械、电子、通信、软件工程等多学科领域，传统装备研发依靠学科专家分别对各自领域进行设计，无法实现跨学科交叉融合；

（2）传统的无人平台研发在概念设计阶段缺乏有效的验证方法，一般依靠最终的实物验证系统可靠性，反复轮次多，设计成本高；

（3）目前平台设计人员在系统全生命周期中采用不同工具进行数据交互，如基于文档的需求/设计报告、基于表格/数据库等形式的需求管理、基于三维建模工具的模型信息管理等，导致不同设计人员难以理解全部信息。

（4）传统的平台研发主要依靠专家经验或设计模板，在设计需求改变时无法系统性地调整参数以满足系统变化。

因此，本文针对无人平台的研发挑战，基于多架构建模方法设计领域建模语言，并在此基础上实现需求、功能、逻辑、物理架构的映射分析，最终通过本体实现模型数据的一致性表达，并实现数据的可追溯传递。

无人作战平台作为一个复杂系统，其设计过程包括机械、电子、控制等学科的交叉融合。该系统在功能上分为机动平台分系统、作战载荷分系统、终端控制分系统三部分。其中，机动平台系统负责平台的整体机动性能，实现位移，越障等功能；作战载荷系统负责平台的整体作战性能，实现瞄准，射击，躲避敌军，协同作战等功能；终端控制系统负责接受主控台的命令实现相应的功能。总体体系结构图如图 6.1 所示。

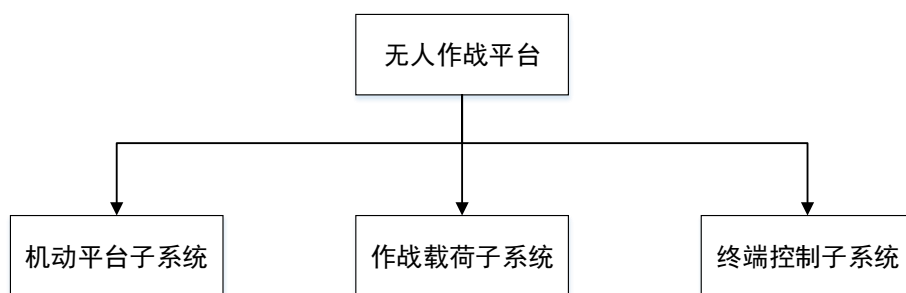


图 6.1 无人平台总体体系结构

无人平台的设计过程如图 6.2 所示。首先，设计总师根据上层要求创建型号，然后统计收集系统所有利益相关人的要求，最后按照需求分析、功能分析、逻辑分析、物理架构设计的思路进行无人平台的初步技术方案生成，然后基于本体框架提取模型信息，并基于状态机仿真及指标验证的方法实现对系统设计方案的验证和确认，最终进行系统评审。

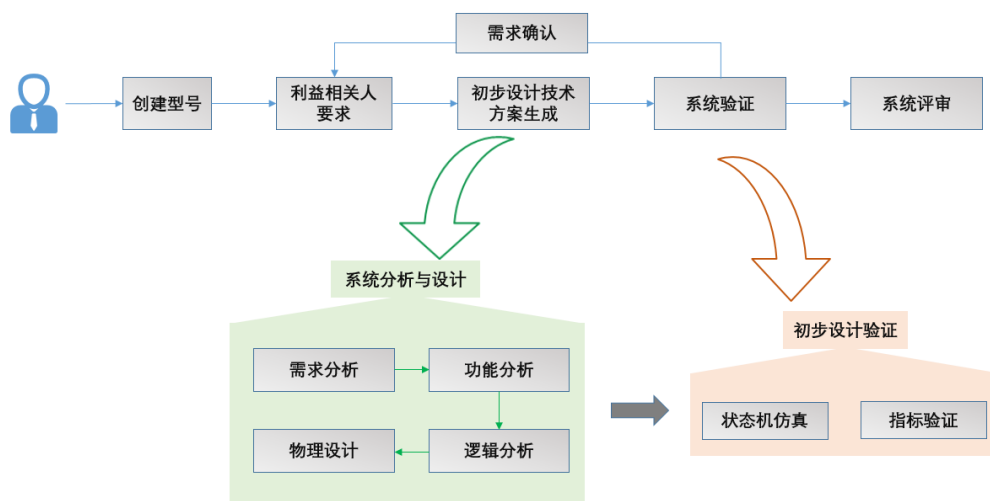


图 6.2 无人平台设计过程

一、系统分析与设计

在提取系统中利益相关人对系统设计的要求后，采用多架构建模方法实现模型构建，部分模型如表 6.1 所示。

表 6.1 无人平台模型构建

种类	模型图	目的
需求分析		
需求图	系统总体需求	描述无人平台功能、性能、接口需求，可调用 OSLC 中的 URI 实现需求的一致性描述
	机动平台需求	描述机动平台重量、载荷、自定位精度、探测等性能

	求	指标
	终端控制系统需求	描述无人平台终端控制中人机交互、实时控制等体系需求及工作时间等性能指标需求
	作战载荷分系统需求	描述无人平台作战载荷系统与射击精度、携弹量、可视距离等性能指标需求
功能分析		
包图	平台整体包图	对系统中涉及的用户、需求、功能、组件分别进行统一描述
	平台刹车包	对系统在刹车场景中用到的模块进行统一描述
用例图	系统运行用例图	描述系统操作人员与运行过程中的行驶、定位、越障、导航、射击、信号处理等功能交互
	地面管理用例图	描述终端操作人员与平台区域搜索、跟踪管理等功能的交互
	行驶用例图	描述操作人员与无人平台在行驶场景下行驶、避障、待命等功能的交互
	设备与地面维护用例图	描述机动平台操作人员与运载、检测、悬挂等功能的交互
活动图	无人平台运行活动图	描述无人平台根据现场环境执行行驶、作战、返航、待命功能的活动流
	作战活动图	描述作战载荷系统接受作战指令后的行为活动
	执行区域搜索活动图	描述地面控制站根据分布式无人平台发送搜索区域的行为活动
	刹车场景活动图	描述无人平台在刹车时机动平台的行为活动
	区域探测活动图	描述无人平台接受搜索指令后的行为操作
	设备及地面保障活动图	描述无人平台启动后地面检测及状态检测的行为操作
	无人平台自主作战活动图	描述无人平台自主决策跟踪、拦截、作战的行为操作

逻辑分析		
状态机图	系统运行状态机图	描述无人平台待命、避障、战斗、返航四种状态随时时间的变化情况
	嵌入式系统运行状态机图	描述无人平台嵌入式系统就绪、运行、挂起、中断、延迟五种状态之间的转换情况
	行驶状态机图	描述无人平台匀速、减速、加速、图像收集等八种状态之间的转换情况
顺序图	作战任务时序图	描述无人平台中操作人员、控制器、武器弹药、传动、运载五个模块执行相关活动的时间序列
	区域搜索时序图	描述无人平台执行区域搜索用例时操作者、系统、无人平台、终端平台之间相关活动的时间序列
	嵌入式系统时序图	描述无人平台接受命令后操作人员、手持操作板、总线、处理器、显示器之间活动的时间序列
物理设计		
模块定义图	无人平台模块定义图	描述无人平台各组件及组件之间约束模块之间的关系
内部模块图	机动平台内部模块图	描述机动平台内部元素及元素之间的关系
	控制系统内部模块图	描述终端控制系统内部元素及元素之间的关系
	作战载荷系统内部模块图	描述作战载荷系统内部元素及元素之间的关系
	机械接口内部模块图	描述机动平台内部模块之间的机械接口关系
	刹车用例内部模块图	描述无人平台执行刹车用例时系统内部模块之间的关系
参数图	系统运行参数图	描述无人平台重量、续航能力、电机型号、电池容量等参数之间的关系
	直线行车参数图	描述无人平台执行直线区域搜索命令后速度、加速

	数图	度、距离、刹车力度之间的关系
--	----	----------------

（1）系统需求描述

无人作战系统项目的启动始于用户的需求和创新模式应用，需求的定义、分析、创新应用、管理不仅是项目最早启动和开展的工作，而且还贯穿于系统生命周期。需求分析的主要工作就是通过对项目利益相关人员分析、需求的捕获与分析，梳理出项目的顶层系统需求，并将系统需求根据不同的功能架构逐级准确、清晰、无歧义、可追溯地传递和物理实现。

在实现需求捕获的过程中，传统的设计方案是凭借设计者的专家经验或者以往的设计方案中抽取设计参数，在 MBSE 的需求管理中，首先提取利益相关人的粗略的、不成体系的、可以文本描述的要求，然后从中提出规范的、有条理、可追溯的需求指标。在本案例中，提取出的系统要求（Need）包含下面三条：

1、无人平台中涉及机械、电子、控制等多领域的交叉融合，因此系统需要同时考虑多系统的耦合性能，做到协同设计。

2、无人平台划分为三个子系统，子系统必须满足基本的作战要求。

3、无人平台具备一定的自主决策能力，满足在敌后自主搜寻的功能。

为了实现无人平台功能和性能之间的协同，需要综合考虑系统中的参数指标。设计参数指标是指已给定的设计时所必须满足的要求，若未满足，则视为设计不合格，主要包括整车整備质量、装载重量、尺寸、总体要求等性能要求以及使用条件约束、使用维修要求等。从参数指标中提取系统需求，并将其划分为功能需求和性能需求，以便在后续建模中实现功能和性能的协同设计。其中，功能要求包括行驶功能（爬坡功能）、作战功能、供电功能；性能需求包括工作时间、车重、响应时间、续航、精度等。实际上，上述指标作为系统整体指标，可以将其分解到各个子系统中，所有满足约束条件的参数分配都可以称为合理的设计方案。

为了准确、清晰、可追溯地表达系统生命周期中所有利益相关人的需求，按照系统工程的分层/级、边界划分、功能独立和相互交联的工程实践与分析应用原理，可以将地面无人机动平台作为一个整体研究。表 6.1 所示为系统的总体需求指标记录表，包含行驶速度、爬坡能力、续航时间、重量、响应时间、续航里程、电池容量、射击距离、射击精度等十项。总需求见图 6.3，按照功能需求和性能需求将需求指标进行分解得到各子系统需求图如图 6.4-6.6 所示。根据设计要求将指标分解到不同的子系统中，以用例的形式进行展示，如图 6.7 所示为终端控制分系统需求中的地面控制器需求进一步细化出的无人平台地面站控制器需求表，

MetaGraph 采用需求表的形式对需求指标进行分解。

表 6.1 无人化作战平台设计需求指标记录

需求 ID	需求标题
WRPT-RS001	行驶速度
WRPT-RS002	爬坡角度
WRPT-RS003	连续工作时间
WRPT-RS004	重量
WRPT-RS005	响应时间
WRPT-RS006	续航里程
WRPT-RS007	电池容量
WRPT-RS008	射击距离
WRPT-RS009	射击偏差 R50 偏差
WRPT-RS0010	射击精度

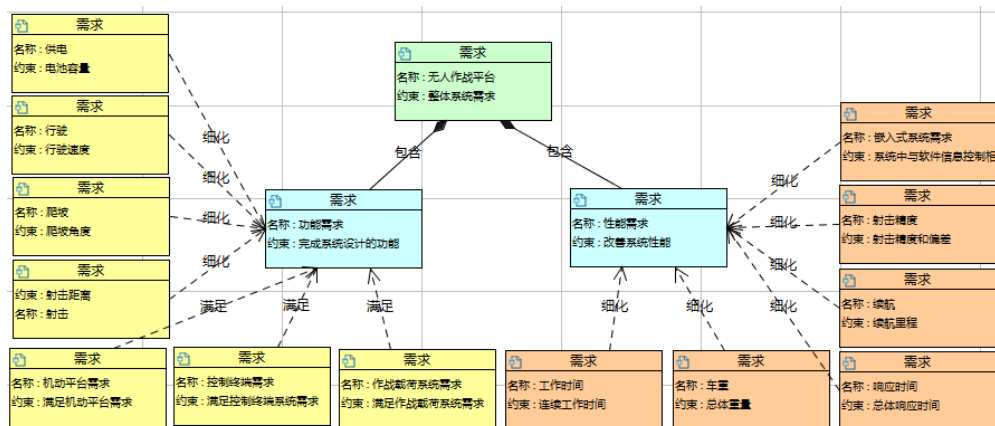


图 6.3 无人作战平台总体需求

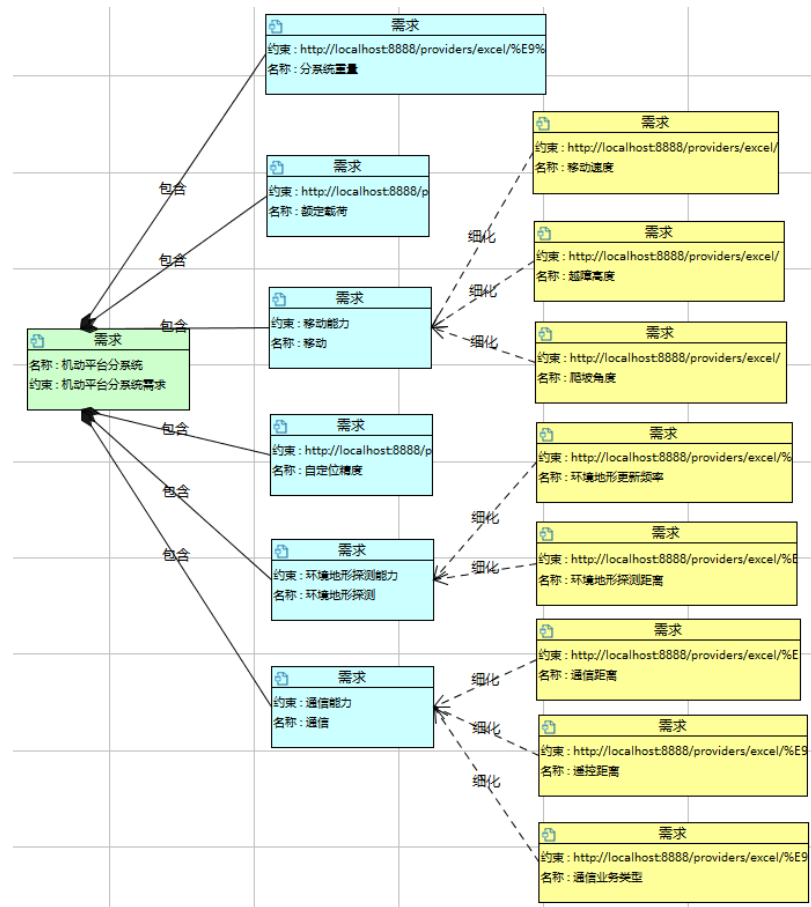


图 6.4 机动平台分系统需求

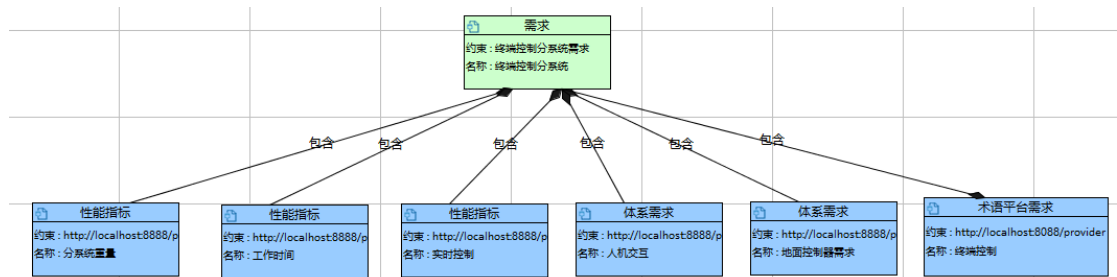


图 6.5 控制终端分系统需求

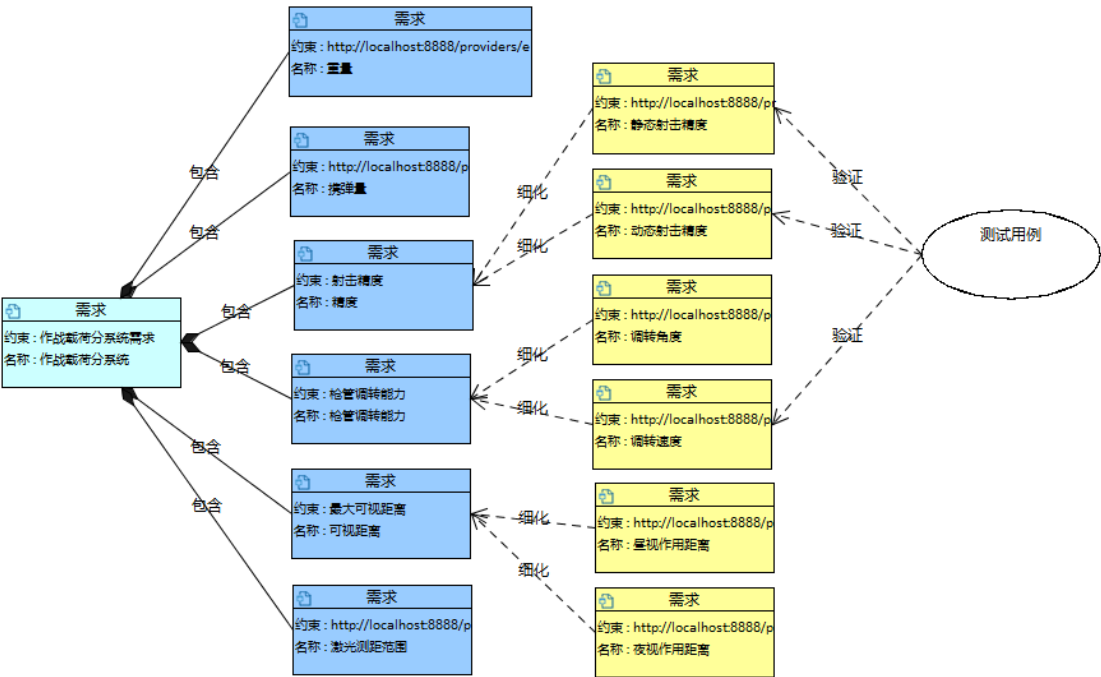


图 6.6 作战载荷分系统需求

需求引用	需求
1	实时监控 UAV 的状态（身份和传感器负载）
2	允许操作员将搜索区域分配给选定的飞行中的 UAV
3	以 1 次更新/秒的频率接收来自 UAV 的传感器跟踪信息
4	在系统中保持 30 分钟的跟踪历史
5	允许操作员维护包含所采用的系统跟踪信息的资料库
6	最多维护 100 条 System Tracks（系统跟踪信息）
7	允许操作员对系统跟踪信息执行生命周期操作（创建/删除）
8	每秒更新一次系统跟踪信息，如果主传感器跟踪信息更新可用，则使用该值进行更新，否则，使用...
9	使系统跟踪信息可在显示屏上显示，并绘制其更新
10	允许操作员将操作员辅助系统跟踪信息与另一台 UAV 的传感器跟踪信息相关联
11	允许操作员将两条独立的系统跟踪信息合并成一条
12	将系统中的重要事件（比如创建和删除系统跟踪信息）通知操作员
13	允许操作员随时中止 UAV 搜索

图 6.7 无人平台地面站控制器需求表

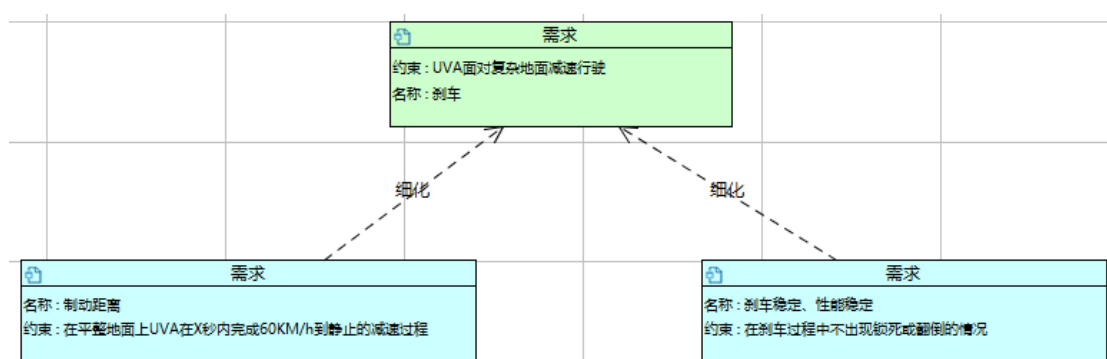


图 6.8 无人平台刹车用例需求图

MBSE 在进行需求管理时，首先收集利益相关人员对系统的要求，再将其细化为成体系的、可追溯验证的需求图或需求矩阵。在上述需求描述过程中，首先根据要求细化出系统整体的需求指标，再将其按照功能需求和性能需求的描述对其划分，其中，功能指标中包含总系统对三个子系统的需求，因此，根据利益相关人员的需求可以对三个系统的需求进行进一步的细化。图 6.5 控制终端分系统需求中包含的地面控制站需求可以细化为图 6.7 的需求表，即需求描述的另外一种方式；图 6.4 机动平台需求中也可以细化为图 6.8 的刹车用例需求图。简单来说，MBSE 的需求管理可以使用需求图、需求表、需求分配矩阵等多种形式对需求进行描述，而不同的需求之间具备一定的关系，在进一步的细化过程中实现了对系统不同场景的需求描述。

（2）系统功能分析

对系统需求进行分析管理之后，需要对系统的功能进行管理和划分。功能对象和关系提供系统要实现的目标视图，让开发人员了解最终系统是如何工作来完成上述目标。其中，包图用于描述系统中涉及到的具有共性元素的集合，可以从整体对系统进行简单的划分。其中，图6.9为无人作战平台整体包图，包括外部角色包、需求包、功能包、组件包，外部角色包中包含三个子系统的操作人员，功能包可以泛化为可视化分析包、识别感知包、战斗载荷包、行驶包等。图6.10为细化出的刹车包，包括设备中心处理器、电子液压阀门、牵引探测器等在内的五个模块，用于描述系统在刹车场景中的元素关系。

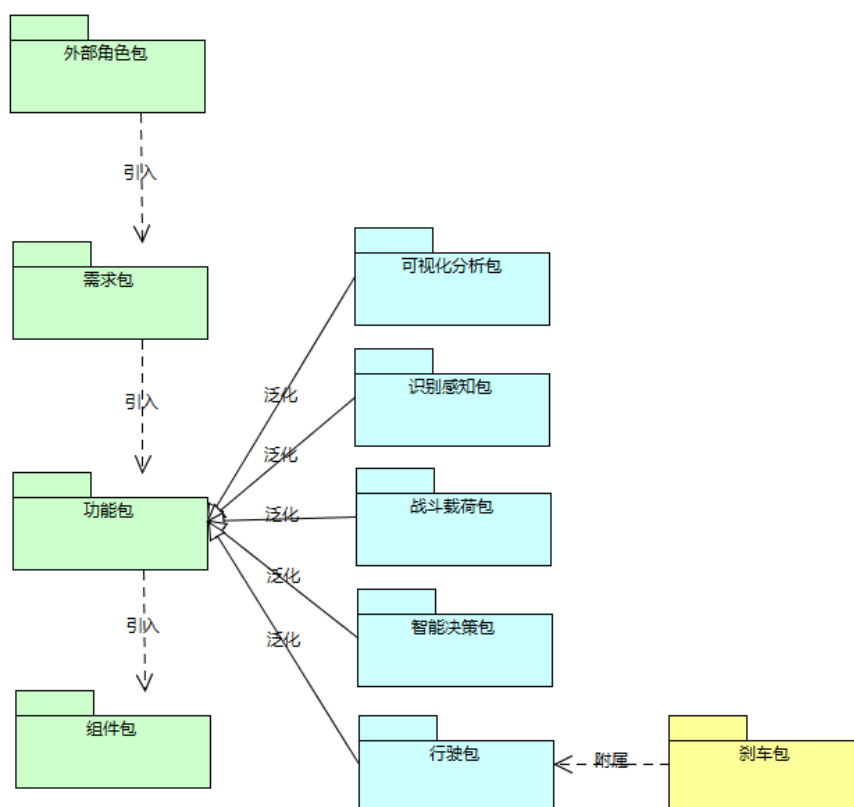


图6.9 无人作战平台整体包图

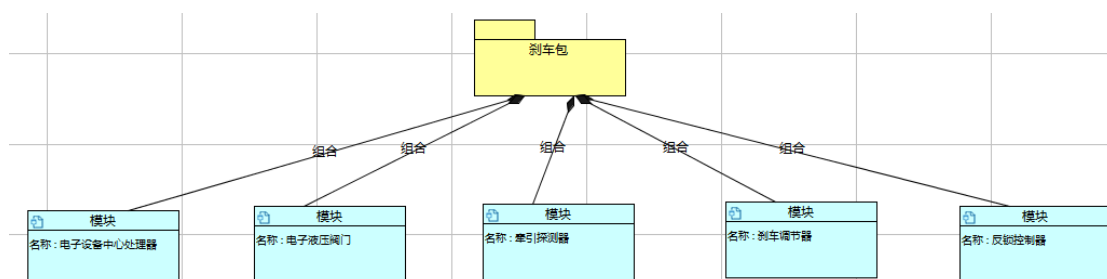


图6.10 系统刹车包

在对系统中具备共同特性的元素集合进行描述后，采用用例图对系统功能进行分析和描述。用例图可以显示各种类型的元素和关系，已说明系统提供的服务信息，以及需要服务的利益相关者的信息。可以传递一系列用例（系统提供的外部可见的服务），以及触发和参与用例的参与者，是系统的一种黑盒视图。其中，图6.11所示为系统整体用例，从系统级描述该系统可以完成行驶、越障、定位、导航、射击、接受分析处理信号等功能。图6.12为系统在实现地面站控制的需求时系统的黑盒用例图，地面站在管理控制无人车时可以执行区域搜索以及UVA管理跟踪。在后续的行为动作描述时对区域搜索的用例进行详细描述。图6.13为无人

车的行驶用例图，无人车在执行搜索案例时可以实现行驶、避障、待命用例，其中，行驶用例可以扩展为加速、刹车、涉水三个用例。图6.14为设备及地面维和保障用例，在地面维护场景中涉及机体结构、火控、电气能源、武器弹药、设备维护、操作人员六个外部参与者，涉及到悬挂/随动、运载、检测、装载、提供状态信息五个用例，参与者与用例之间形成了黑盒模型用于描述系统功能。

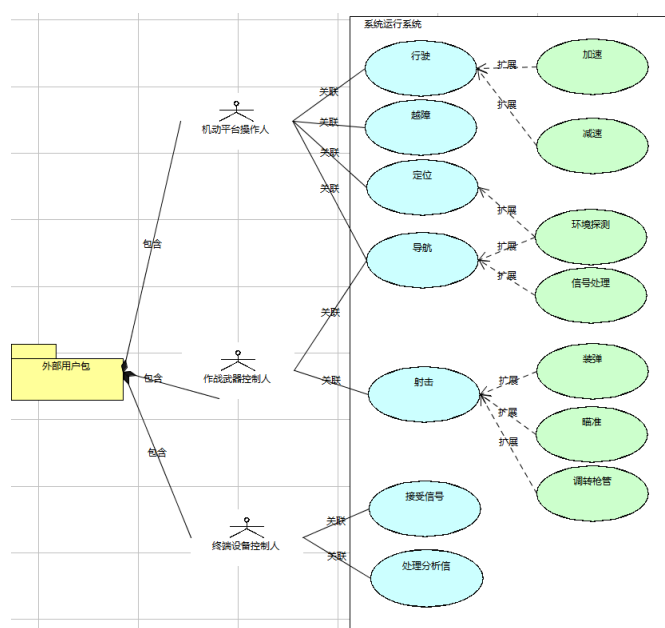


图6.11 系统整体用例图

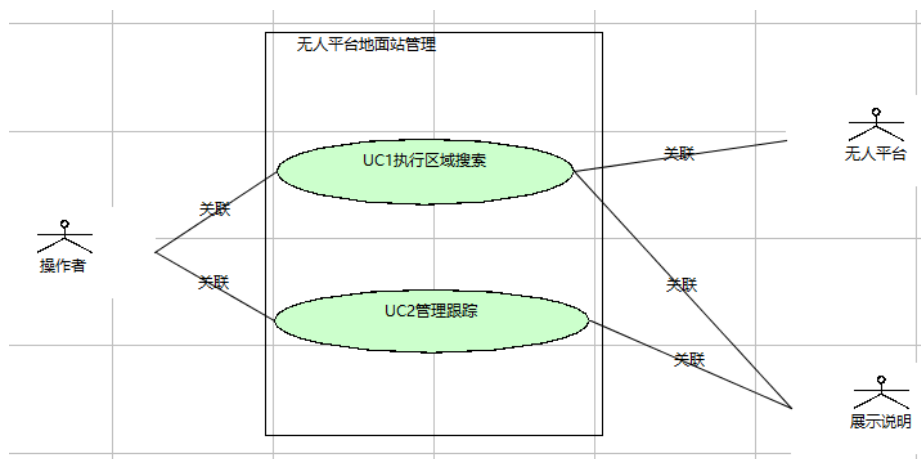


图6.12 无人平台地面管理用例

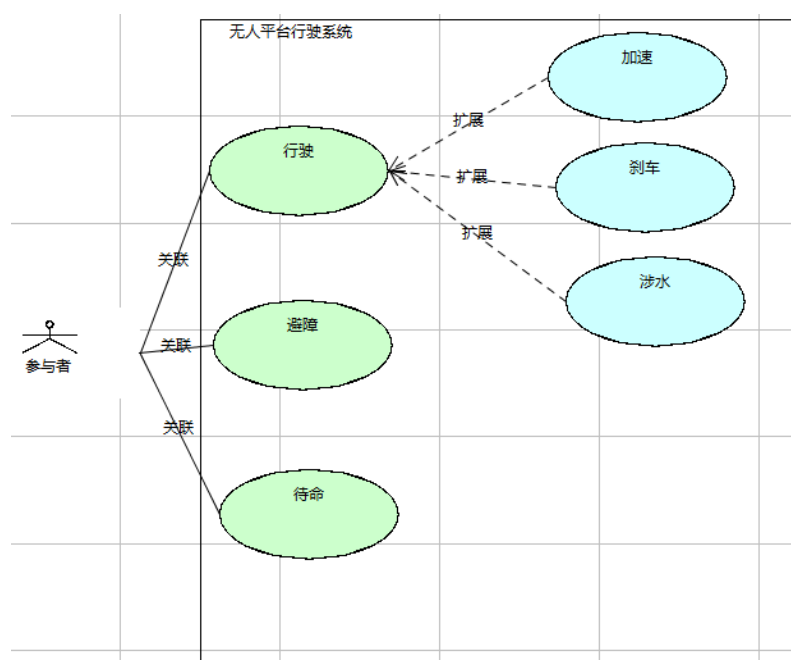


图6.13 无人车行驶用例图

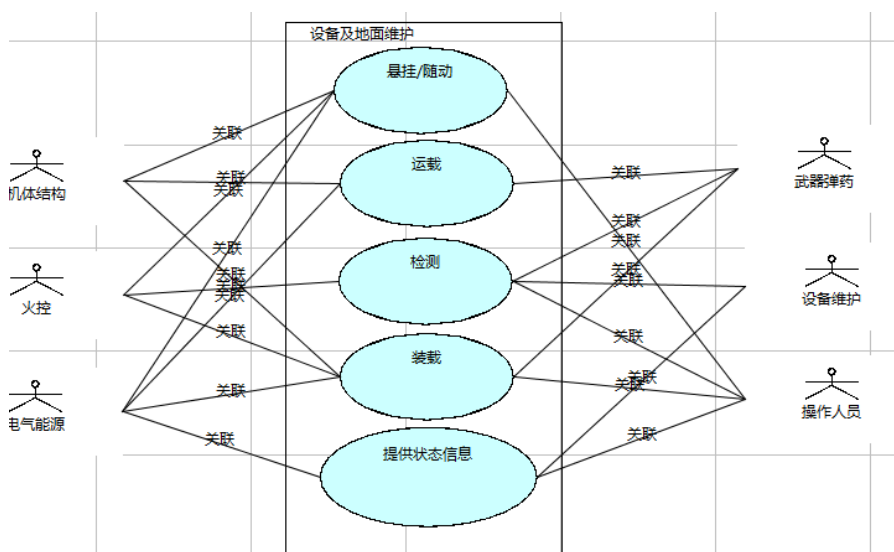


图6.14 设备及地面维护和保障用例

此外，功能分析的意义是形成能够描述系统功能和功能之间相互关系的一系列模型元素，可通过活动图、顺序图和状态图来展现，由不同的分析侧重面对系统功能进行分析。活动图本质是一种流程图，给出了系统功能流的活动逻辑关系，即完成某项系统功能，需要按照一定的控制流执行对应的命令与动作，进一步分析能够为系统功能接口定义与逻辑控制算法提供设计支撑。顺序图又称时序图或序列图，其描述并定义操作、不同功能间的信息发送和接收的时间交互关系。状态图又称状态机，主要用于描述一个对象在其生命周期内的动作行为，将活动图

和顺序图的信息汇集在一起，可以展现融合系统状态的背景及不同优先级外界激励产生的系统行为。

系统功能分析的焦点是将功能性的、表现的、接口的以及其他的需求翻译成系统功能的清晰描述，并用来指导后续的设计集成与综合。针对系统需求进行详细系统功能分析和分解，对需求进行传递与功能实现分配，形成具备一定功能和可执行的功能分析模型。图 6.15 为系统总体活动图，描述系统的行为活动之间的传递转移关系，在无人作战平台执行命令时，首先启动平台，然后识别感知附近环境及平台自身状态，对感知到的信息进行可视化分析及智能决策，随后按照是否有障碍物、是否有敌人、是否电力充足、是否有后续任务四种条件执行具体任务。图 6.16 为作战活动图，无人作战平台在接收到作战指令后，转换为作战状态，首先在启动电源后对系统进行状态分析，如果处于故障状态则执行故障模式，反之依次执行探测目标、武器占位、武器线控制、弹药发射几个动作。图 6.17 为设备及地面维护和保障活动图，用于描述上述设备及地面维护和保障用例中具体的行为活动。图 6.18 为无人平台刹车活动图，描述系统在执行刹车用例时的行为活动，其中用两个泳道描述两个子系统的交互。图 6.19 为执行区域搜索活动图，描述在地面站在执行地面管理用例时的活动，从地面站发出搜索指令到最后无人平台执行搜索，并将搜索结果返回地面站的行为活动，以此描述系统的区域搜索功能。图 6.20 为无人平台区域探索活动图，描述无人平台在接受探测命令后，系统面对平坦地面和复杂地面时加速减速的活动变化，用于实现仿真验证。

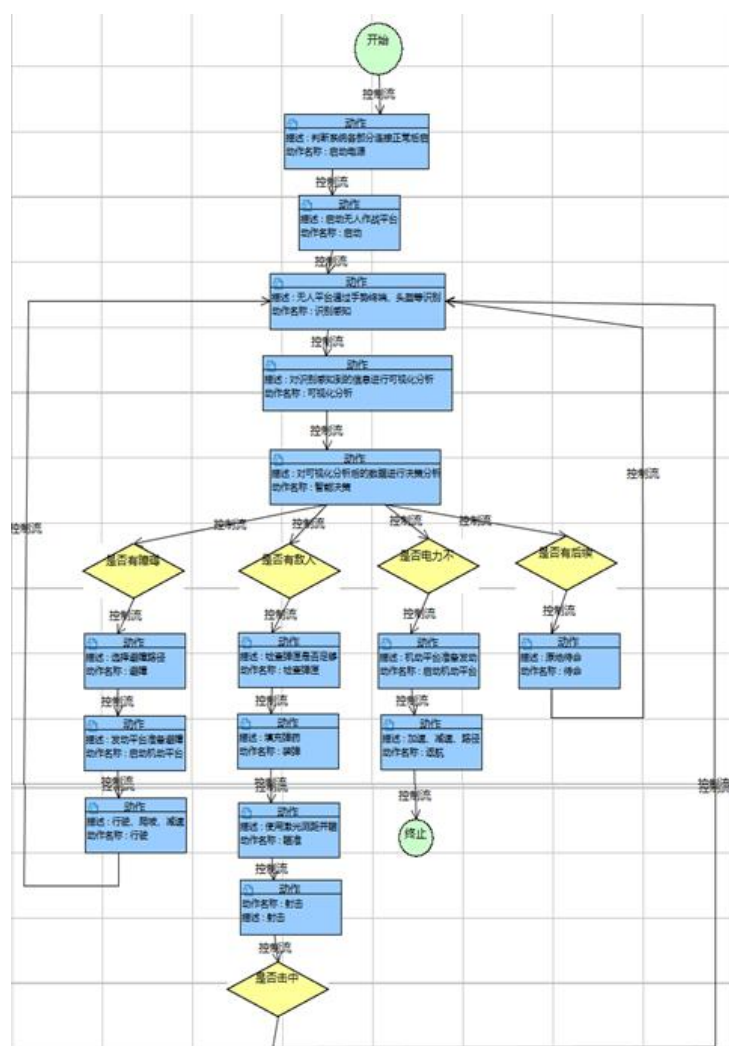


图6.15 整体活动图

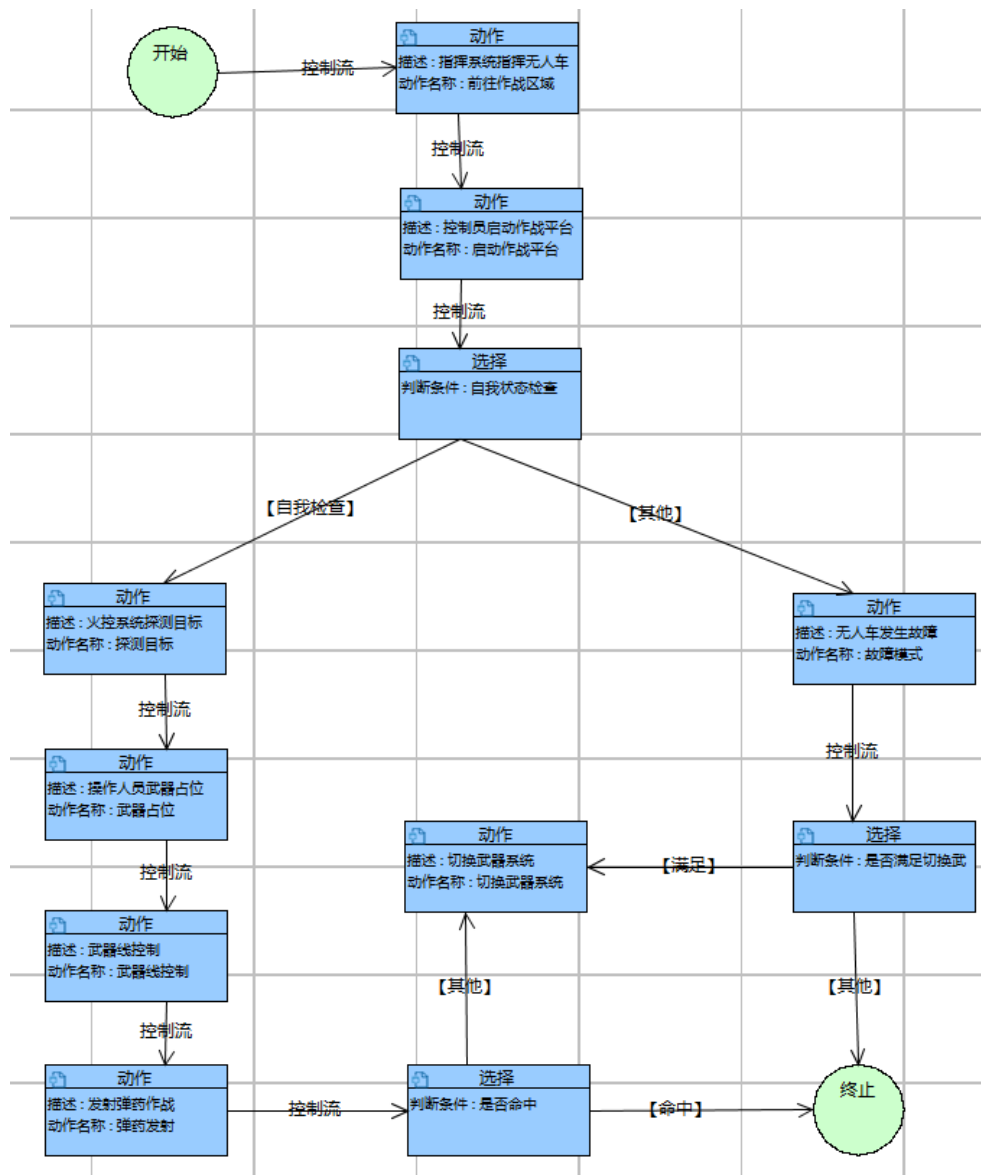


图6.16 作战活动图

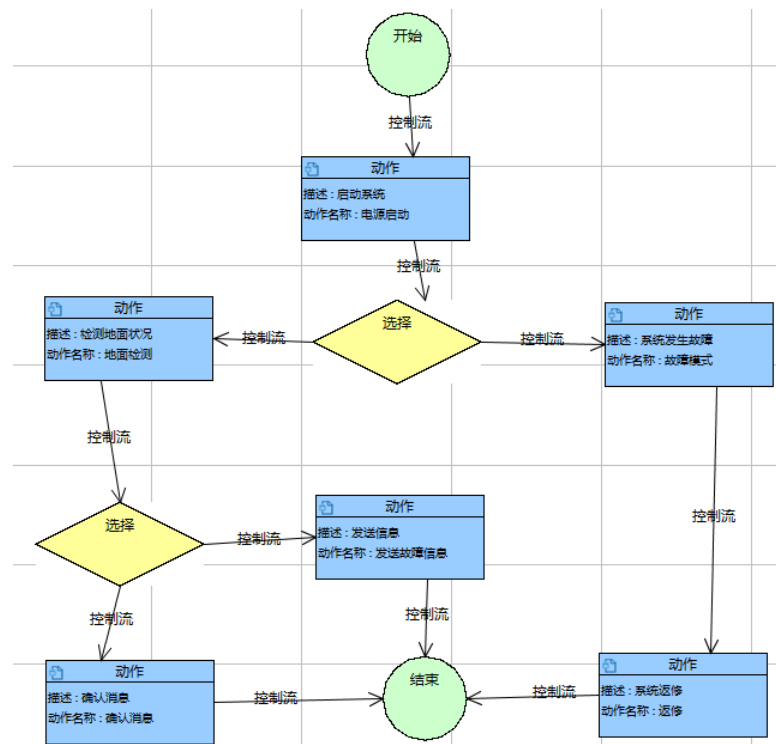


图 6.17 设备及地面维护和保障活动图

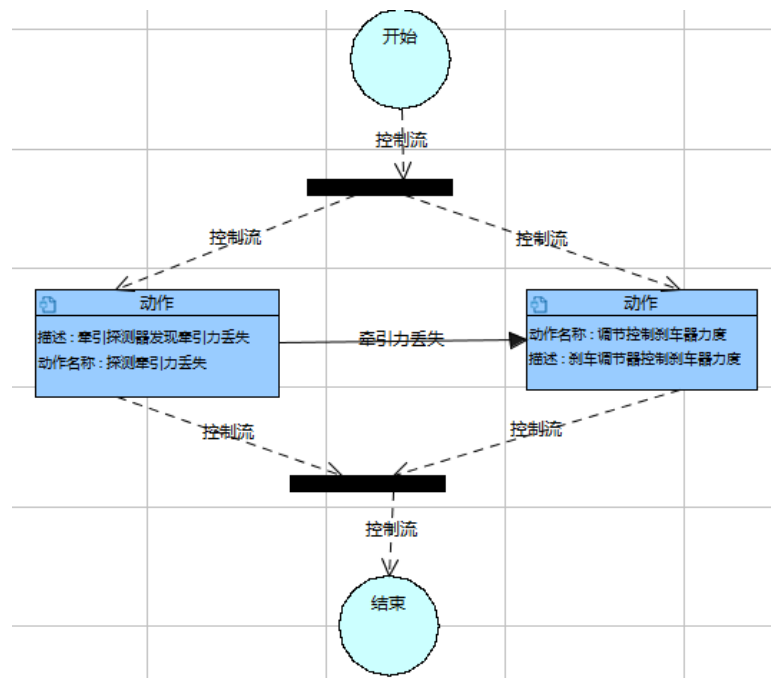


图 6.18 无人平台刹车活动图

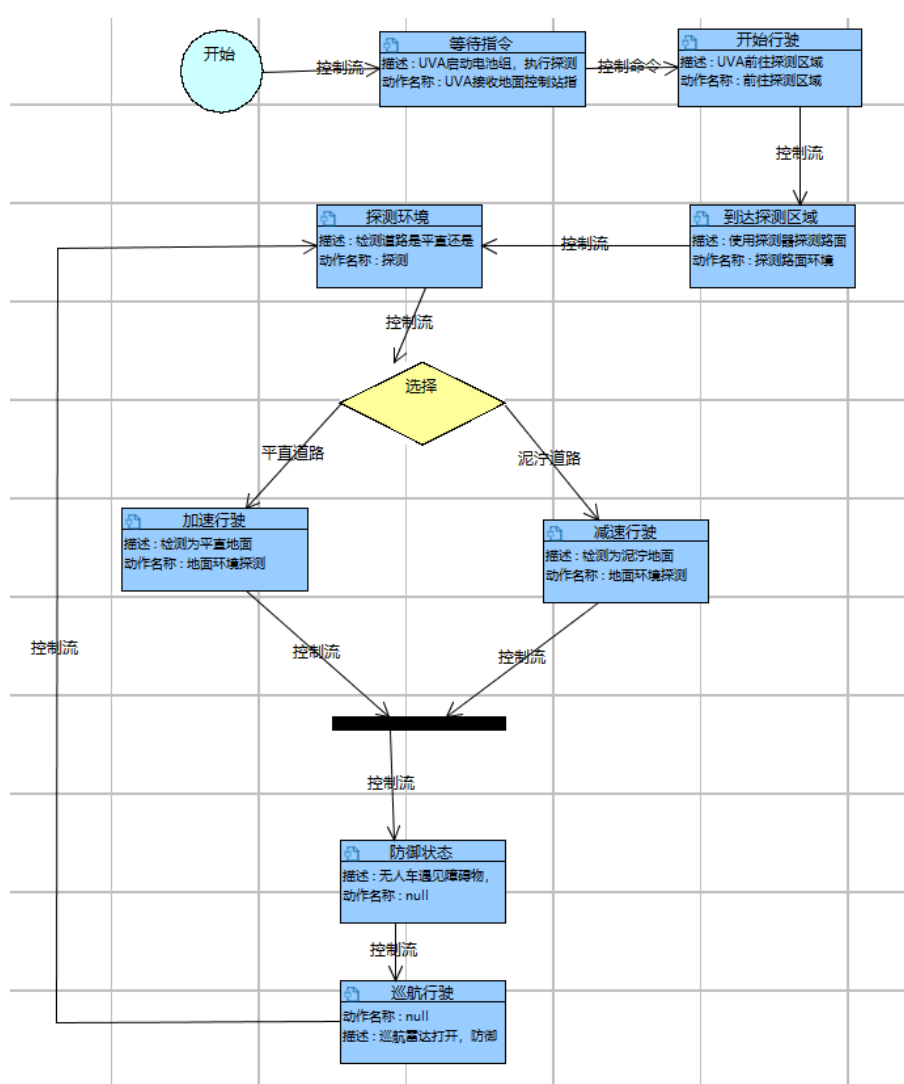


图 6.20 无人平台区域搜索活动图

功能分析的主要工作是针对需求分析，通过不同的功能描述和相互之间的关联分析，提出能够满足系统需求的功能架构和系统功能集，在系统设计综合时按照一定的原则或依据，对系统功能集评审，确定某种优选的架构并进行分配和工程实现。

(3) 系统逻辑分析

系统逻辑分析即提供一种系统可能的解决方案，如图 6.21 所示，在无人作战平台中，系统逻辑按照问题分析、解决方案、实施的流程进行设计，分别采用状态机图及时序图进行描述。图 6.22 所示为系统总状态机图，描述系统在执行整体用例时系统状态如何变化，从关闭状态、准备状态到启动状态、执行任务状态到最终的结束状态，从整体系统的角度对其进行描述说明。图 6.23 则描述系统在执

行任务时，嵌入式系统信息传递过程中随着任务的延迟、挂起、唤醒实现运行、中断、延迟等状态的变化。图 6.24 为平台行驶状态机图，描述系统在执行行驶任务时，系统状态由待命到检测到加速或减速状态的转变。

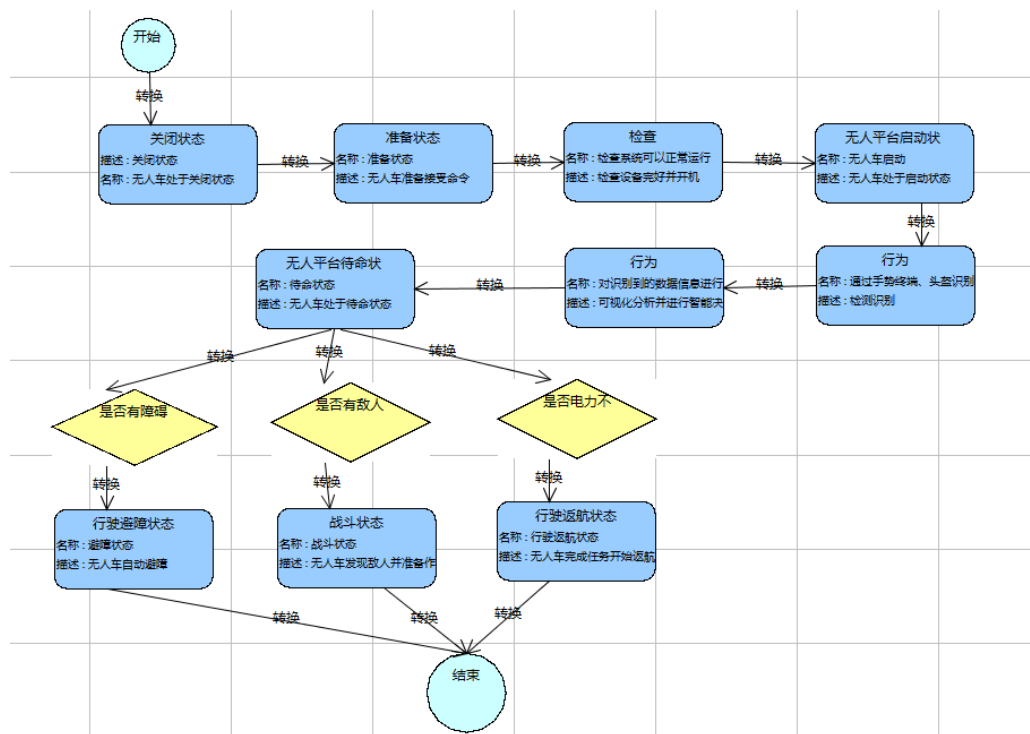


图 6.21 总状态机图

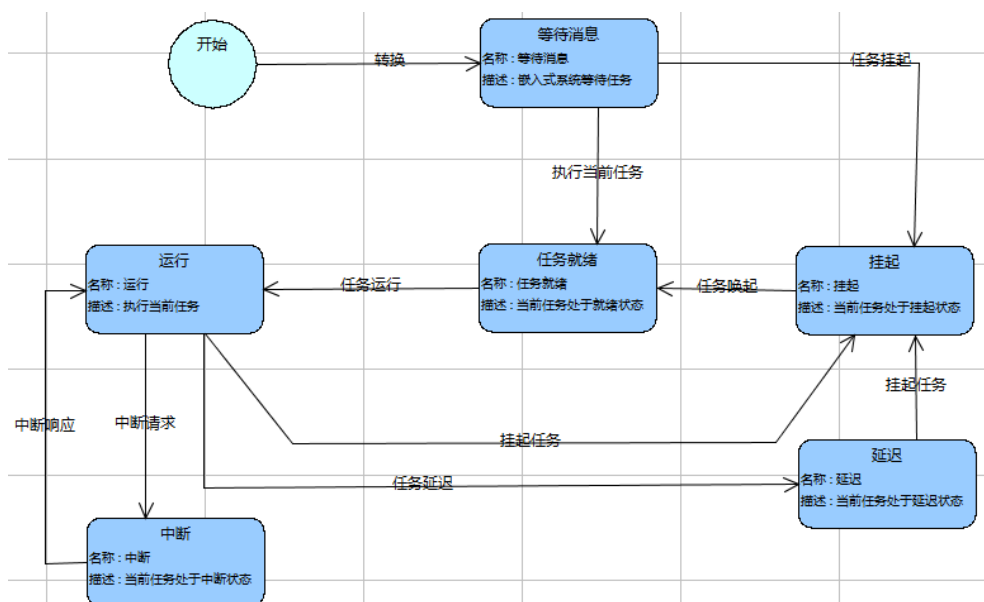


图 6.22 嵌入式系统运行状态机图

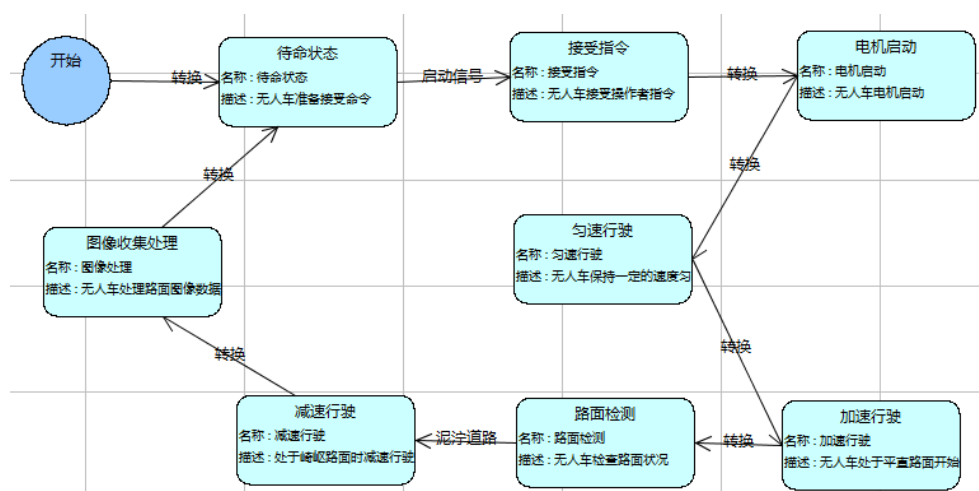


图 6.23 行驶状态机图

在描述系统逻辑时状态机图多数用作黑盒模型，在描述系统的状态变迁后，使用时序图描述系统、子系统、组件随着时间的变化，动作行为的转变。图 6.24 所示为作战任务时序图，描述当系统在接收作战指令后，操作者、控制器、武器弹药、传动系统、武器弹药子系统中信息的传递。图 6.25 所示为区域搜索序列图，描述无人作战平台在接收到搜索任务时系统中时序变化。图 6.26 所示为传感警报时序图，描述系统在检测到危险状态时警报器发出警报的行为活动。图 6.27 所示为嵌入式系统硬件系统序列图，描述系统内的软件信息传递过程，操作人员、手持操作板、CPLD、BUS、CPU、LED 之间的数据传递。

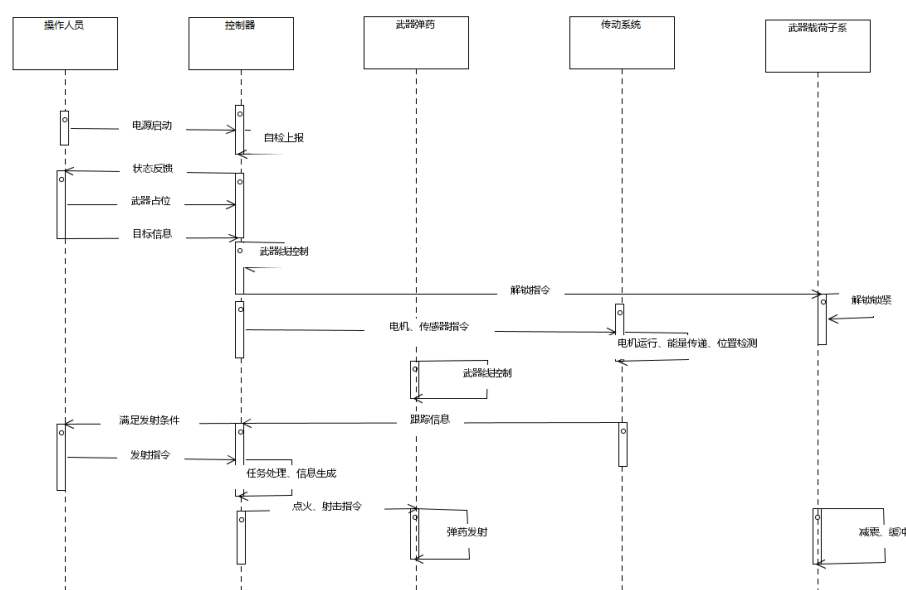


图6.24 作战时序图

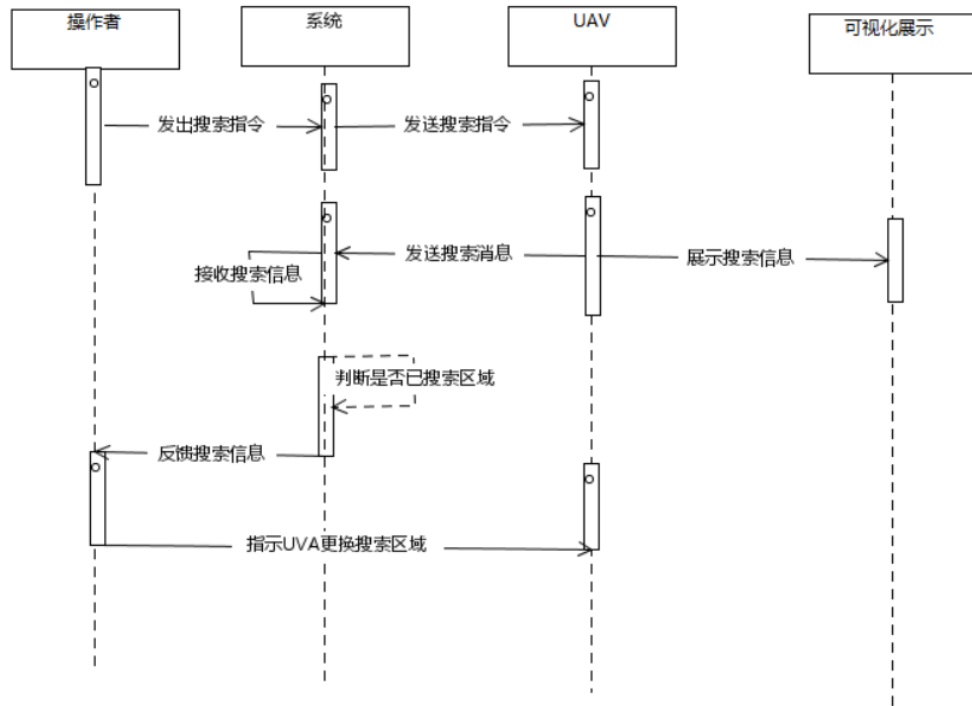


图6.25 区域搜索序列图

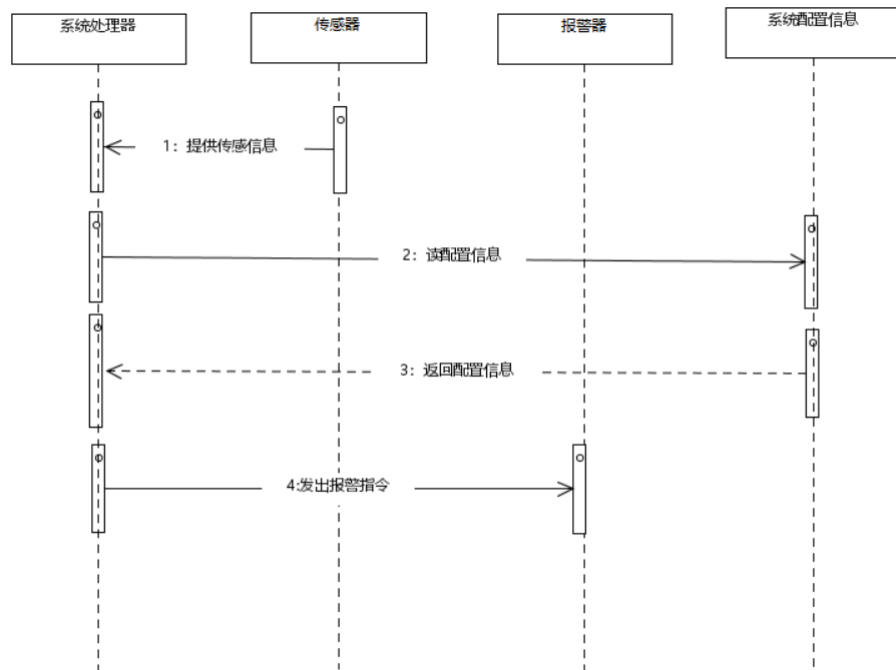


图6.26 传感警报时序图

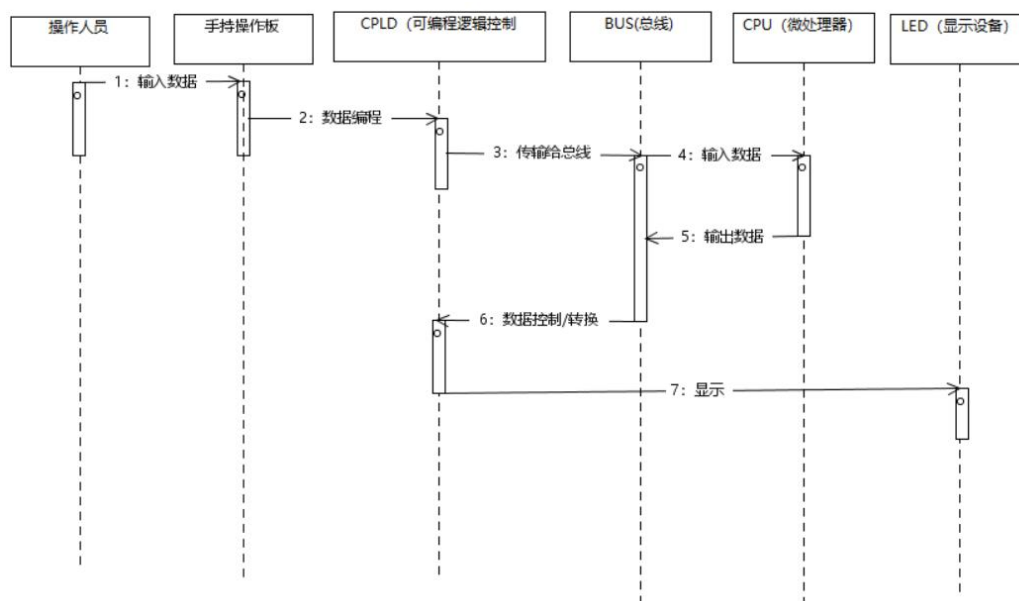


图6.27 嵌入式系统硬件系统序列图

系统逻辑分析主要完成系统与时间有关的各项分析及仿真验证。如时序图可以清楚地表现出随着时间变化，系统如何完成各项功能，通过状态机图可以仿真出相关参数是否满足利益相关人员的需求，从而完成系统的相关验证与确认。

(4) 系统架构设计

系统的架构设计是对系统的物理结构进行设计，以满足系统的逻辑结构，最终得到系统在设计时需要的各个组件，以及为了满足系统的各项需求，各组件需要满足的各项参数。因此，设计模块定义图、内部模块图及参数图对系统架构进行描述。如图 6.28 所示为系统的模块定义图，无人作战平台可以由三个子系统组合而成，每个子系统又分别由各模块组合而成。约束包在整体上满足系统的指标约束，包括续航能力、重量、经费、携弹能力；续航能力可以进一步泛化成续航里程和续航时间。图 6.29 到 6.32 所示分别为机动平台子系统内部模块图、控制子系统内部模块图、作战载荷子系统内部模块图、刹车用例内部模块图。

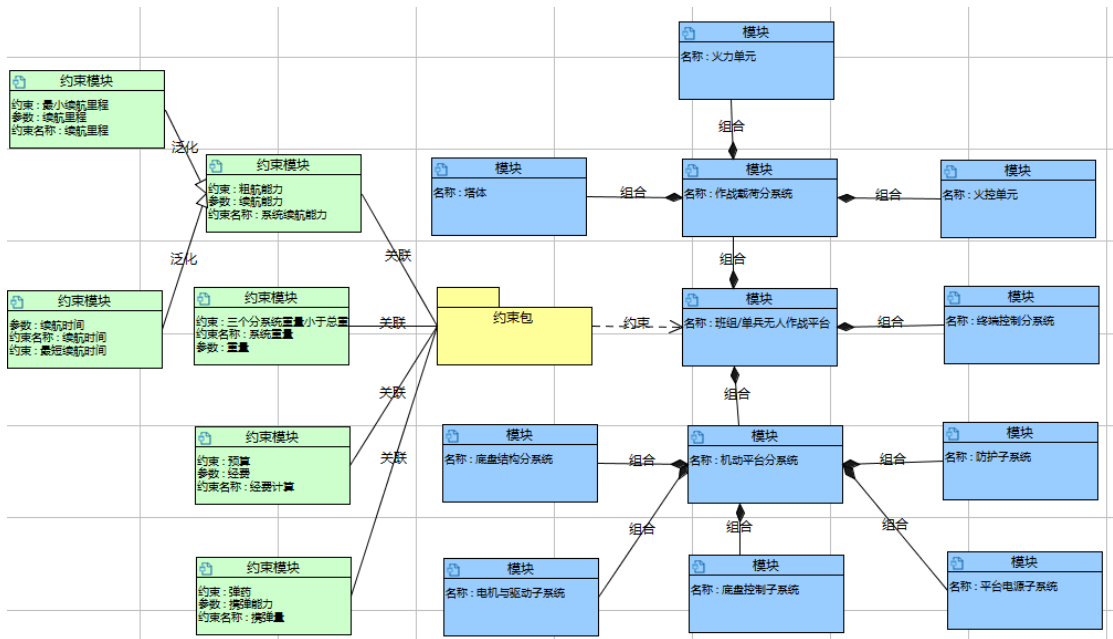


图 6.28 模块定义图

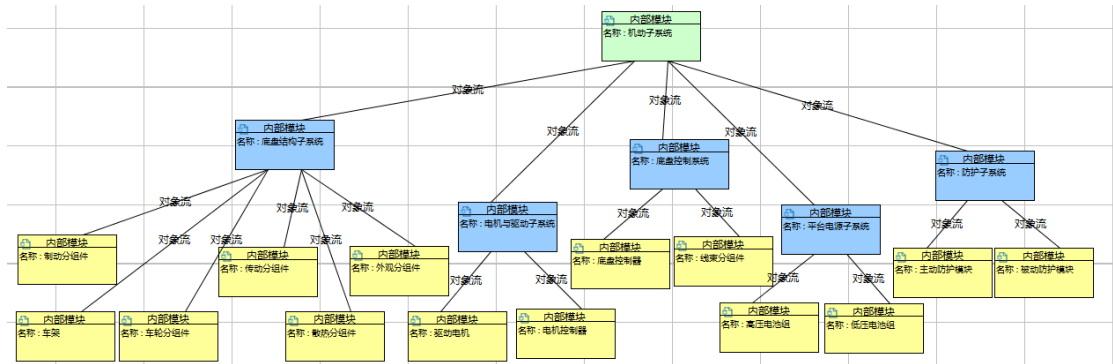


图 6.29 机动平台子系统内部模块图

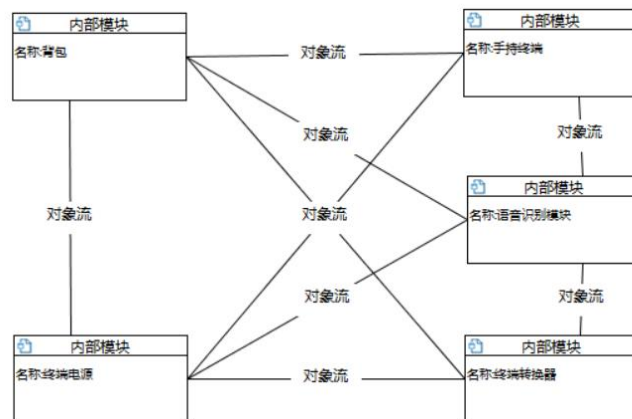


图 6.30 控制子系统内部模块图

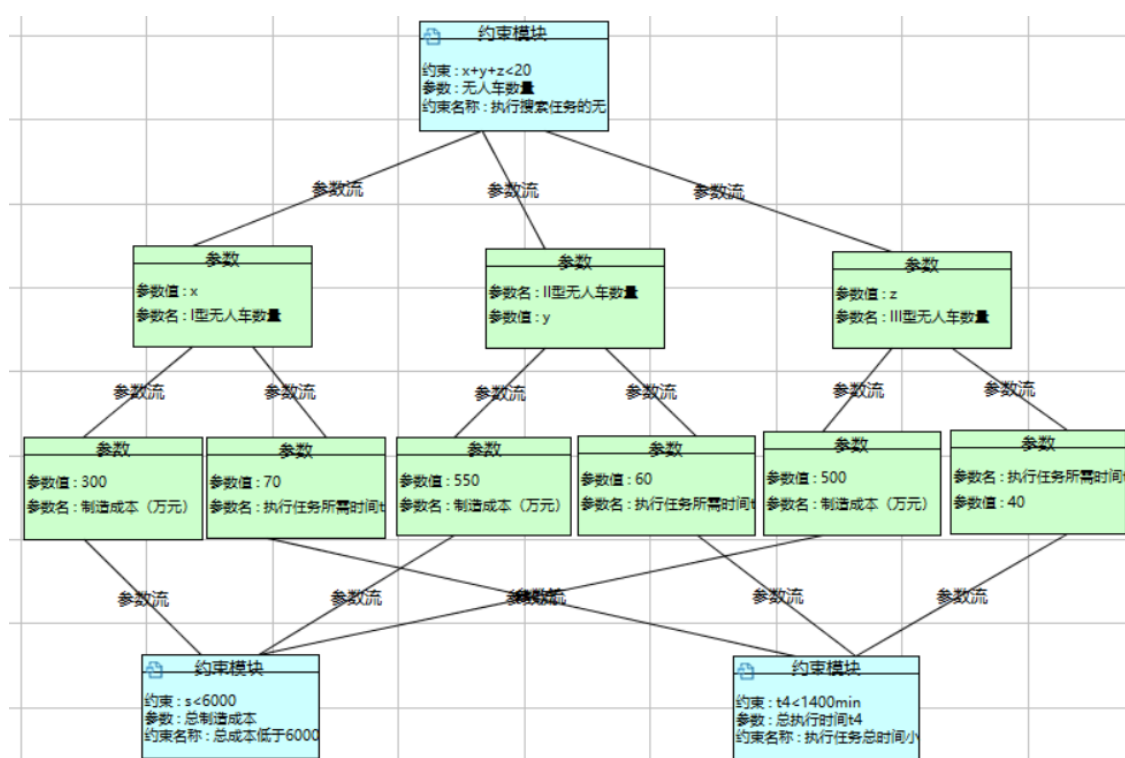


图 6.33 无人平台探测场景参数设计图

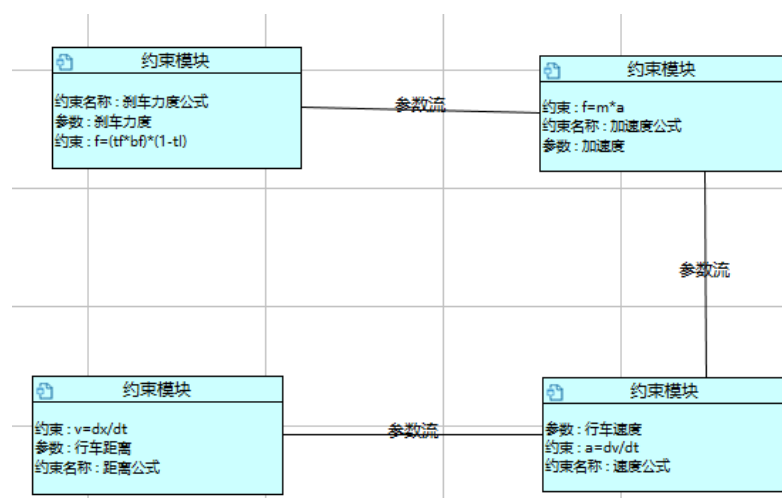


图 6.34 无人车直线行车参数图

二、基于本体的系统验证

基于多架构建模方法实现无人平台需求、功能、逻辑、物理的形式化表达后，采用混合状态机仿真技术实现系统的动态验证，基于可满足性模理论实现系统指标的静态验证及优化设计。其中，本体设计方法可以提取异构模型中的信息，将其传输到求解器中，进而得到验证结果。以无人平台区域探测场景中的参数传递为例实现需求分解及系统建模，并通过状态机仿真和指标验证的方式完成对系统

需求的概念验证和确认如图 6.35 所示。

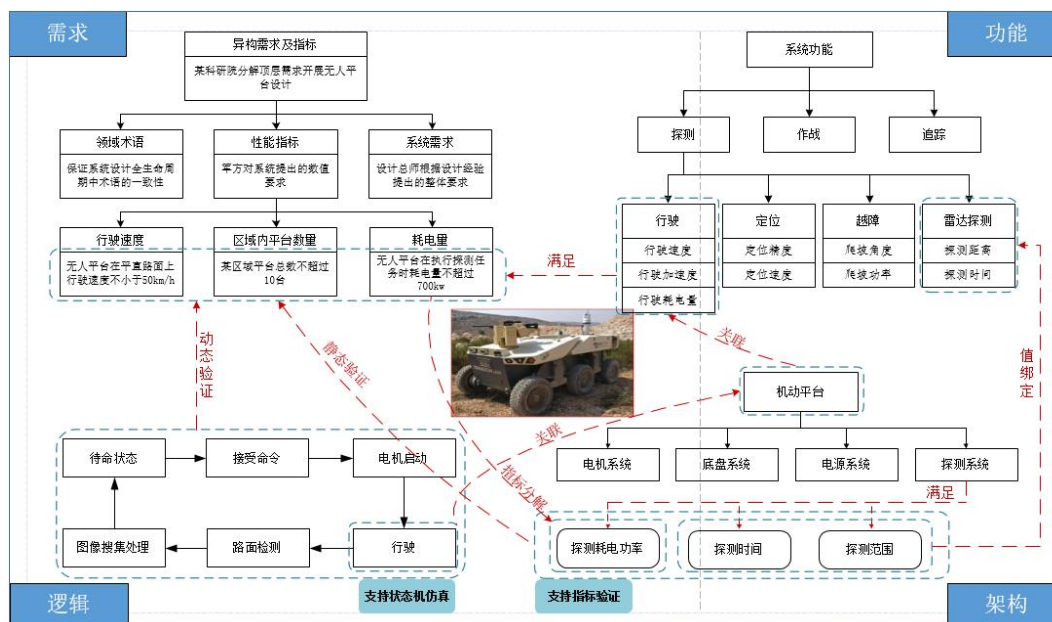


图 6.35 无人平台区域探测场景参数传递

其中，采用状态机仿真对图 6.23 所描述的无人平台行驶状态机图进行验证，采用指标验证方法对图 6.33 所描述的无人平台探测场景参数设计图进行验证和优化。具体验证方法如下：

(1) 状态机仿真

本文以图 6.23 无人平台区域探测场景为例进行动态行为仿真，其中，行驶状态机图中参数与系统架构设计中的相关参数进行关联。如图 6.36-a 所示，建立的状态机图包括“待命状态”、“接受指令状态”、“电机启动”、“匀速行驶”、“加速行驶”、“路面检测”、“减速行驶”和“图像收集”八种状态，描述系统在行驶过程中的状态变化。在模型属性中分别设定状态转移时间及状态参数，图 6.36-b 所示为图中状态及转换条件中的变量，其中定义初始加速度为 0，耗电量为 2.0Kwh，加速时加速度为 3m/s^2 ，耗电量 3.2Kwh，减速时加速度为 -1m/s^2 ，耗电量 1.8Kwh；图 6.36-c 所示为定义图中状态具备的属性，布尔值用于体现当前状态的变化；图 6.36-d 所示为图中“待命状态”的状态属性值，将状态初始化并将系统定位为待命状态；图 6.36-e 所示为图中“转换”关系的状态变迁属性值，定义当时间 $t > 5\text{min}$ ，且检测到系统正常则转为“电机启动”状态。定义好上述信息后仿真结果如 6.36-f 所示，其中，蓝线代表无人平台直线行驶的耗电量，表示当行驶时间为 75min 时系统耗电 1350w，可以验证系统正常运行后耗电量满足系统设计需求。

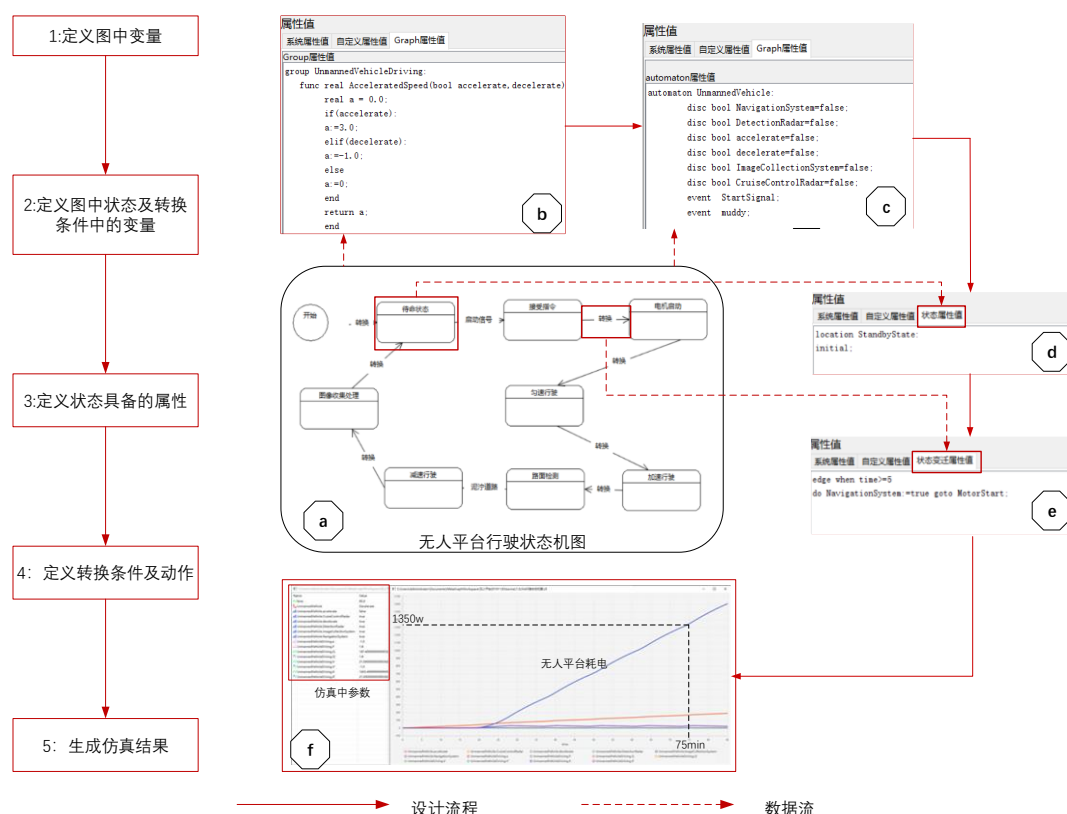


图 6.36 无人平台行驶状态机行为仿真

(2) 指标合理性验证

基于无人平台区域探测场景，设计人员经过功能分析、逻辑分析和架构设计可以得到若干设计方案，为了验证该场景下探测范围、执行任务时间及耗电量是否满足系统性能需求，建立协同区域探测参数图如图 6.37-a 所示。现有三种型号的无人平台可用于执行区域探测任务，并在满足下述三个约束的情况下使探测区域面积最大：如公式（1）所示，平台总数不超过 10 台；如公式（2）所示，无人平台执行区域探测任务总耗电低于 700kw；如公式（3）所示，所有无人平台执行任务总时间低于 600 分钟。其中，x、y、z 分别为执行区域探测任务的三种无人平台数量。图 6.37-b 所示为模型的代码及相关的求解结果，具体图 6.37-c 所示为区域探测参数图中的参数设置，首先定义上述三个指标为整数类型，然后从图中模块中读取属性，定义变量的约束范围及优化目标，优化目标如公式（4）所示。图 6.37-d 所示为验证结果，可以得到，三种无人平台数量分别为 1 台、8 台、1 台，根据优化函数可得，实现协同区域探测任务时总探测范围是 8850m²，满足设计需求。

$$x + y + z \leq 10 \quad (1)$$

$$45 * 70 * \frac{x}{60} + 60 * 55 * \frac{y}{60} + 40 * 50 * \frac{z}{60} < 700 \quad (2)$$

$$70 * x + 60 * y + 40 * z < 600 \quad (3)$$

$$M = \max(950 * x + 900 * y + 700 * z) \quad (4)$$

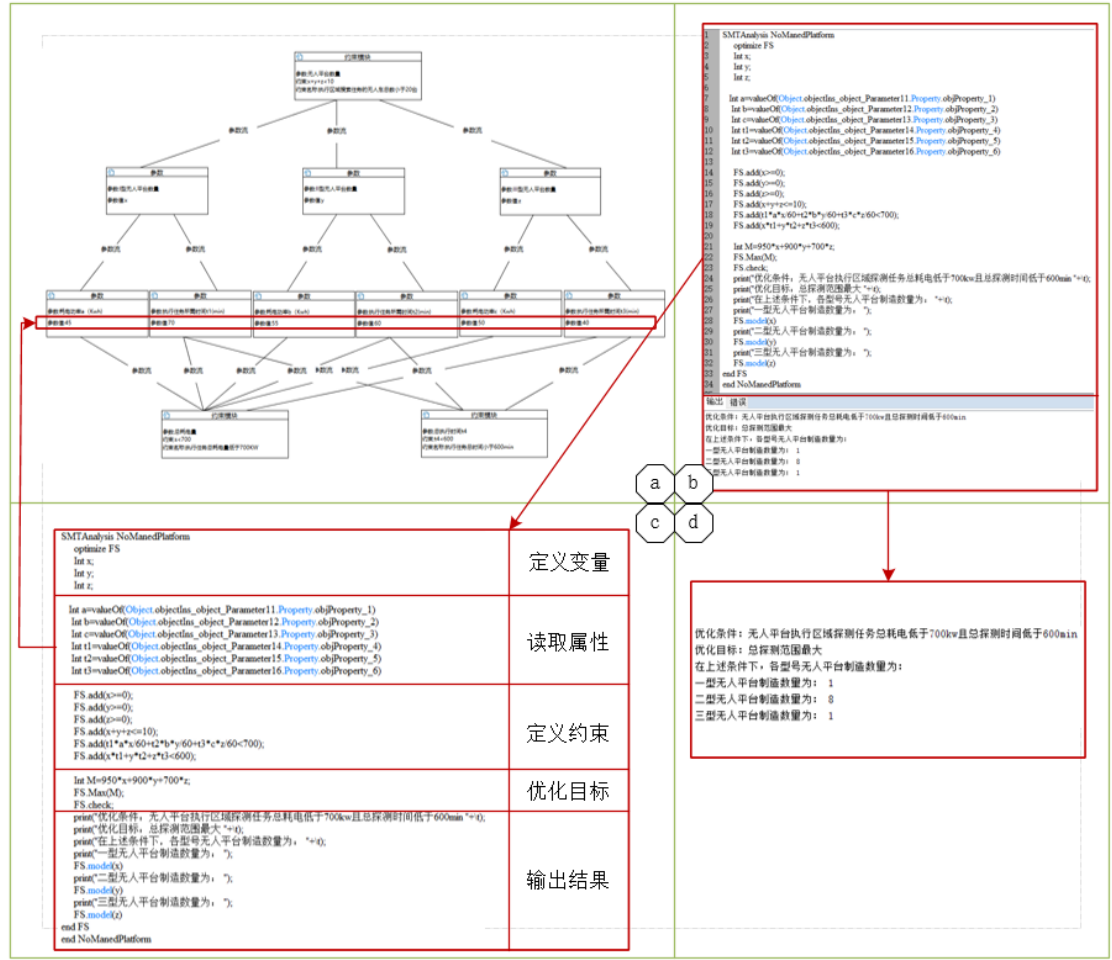


图 6.37 无人平台区域探测参数图指标优化

三、本体总结

在基于 SysML 建模语言实现无人平台的形式化表达后，按照本体框架的映射方法将其转化为 OWL 文件，如图 6.38 所示，该文件包括 318 个子类代表 318 种元模型，3263 个实例代表 3262 种实例化组件，其中，图的实例化组件共 44 种，代表建模时共构建 44 幅图用于形式化表达无人平台设计过程的各个阶段。

Ontology metrics:	
Metrics	
Axiom	23557
Logical axiom count	4319
Declaration axioms count	384
Class count	328
Object property count	32
Data property count	2
Individual count	5263
Annotation Property count	34
Class axioms	
SubClassOf	518
EquivalentClasses	676
DisjointClasses	0
GCI count	0
Hidden GCI Count	96
Object property axioms	
SubObjectPropertyOf	26
EquivalentObjectProperties	0
InverseObjectProperties	0
DisjointObjectProperties	0
FunctionalObjectProperty	0
InverseFunctionalObjectProperty	0
TransitiveObjectProperty	0

图 6.38 无人平台本体元素总结

基于 MetaGraph 建模工具可以实现无人平台模型与本体文件的互转换，即将模型转化为本体文件，也可以直接在工具中导入本体文件，从而在画布中生成相应的模型。经过验证，在工具中先将模型转化为本体文件，再将其导入到工具中时，模型和建模时保持一致，并未信息遗漏或缺失。此外，基于本体开发工具 protégé 实现 SPARQL 查询和 SQWRL 推理，经过验证推理信息与建模时的规则保持一致，因此可以确认该本体文件能够正确地描述建模中的逻辑。

总结与展望

本文工作总结

本文主要针对采用 MBSE 方法对复杂系统进行设计时出现的全生命周期中不同利益相关人员无法理解全部的模型信息，且现有方法无法有效地支持全部建模语言的问题，首先在元建模方法的框架上，基于 GOPPRR 方法进行多架构建模方法设计，在此基础上可以同时实现多种建模语言的形式化表达；然后，结合系统工程常用的方法论提出了需求、功能、逻辑和物理架构（RFLP）的正向设计方法，为系统设计提供相关范式；最后，研究了面向多架构建模的本体设计方法，以 OWL 作为数据交互的基础，从而实现模型数据和本体文件的互转换，并通过 SPARQL 查询语言和 SQWRL 推理语言实现本体关键元素的查询和推理，从而证明设计的有效性和合理性；最终，以无人平台为研究对象对多架构建模设计和本体转换方法进行验证。本文创新点总结如下：

（1）基于 M3-M0 元建模框架，在目前 GOPPRR 建模方法的基础上提出多架构建模方法，该方法可以支持系统全生命周期中多种架构模型的形式化表达，并且经过元模型的对比可知多架构建模方法在工程应用上有较为明显的优势。同时，在此方法的基础上实现基于 RFLP 的映射，为设计人员提供设计范式。

（2）面向多架构建模方法提出一种本体设计框架，该框架可以在 M3-M0 多个层级中与多架构模型信息进行转换，从而可以对系统全生命周期中的全部模型信息用统一的形式进行表达。同时，采用归纳法和演绎法对该本体框架进行验证，结果表明，该本体框架可以完整、无误地与建模数据进行互转化，并且该框架满足本体的逻辑判断，从而辅助设计人员进行设计。

研究工作展望

本文针对 MBSE 的形式化建模方法进行研究，在现有研究的基础上设计了多架构建模方法以支持系统建模设计，同时设计基于本体的转化框架实现模型数据的一致性表达。但因作者时间和能力有限，在以下方面还存在改进空间。

（1）多架构建模方法在工程应用上有较好的表现，但该方法缺少图与图之间的映射规律，即将 RFLP 的流程融入到建模过程中，实现图之间关系的表达。例如，目前仅可以实现需求图中元素（对象、点、关系和角色）到用例图的映射，但是需求图和用例图本身的关系没有在建模方法中表示出来，即用例图为设计人员

提供场景，用于描述需求图的黑盒视图。

(2) 本文采用无人平台作为对象进行方法的验证，但是由于设计参数的缺失，本文的建模多倾向于设计方法的描述，而非参数化的说明，在后续的案例验证中应采用粒度更小的建模。

参 考 文 献

- [1] 常绍舜. 系统工程和系统工程方法[J]. 系统科学学报, 2016(4): 11-14.
- [2] 罗吉贵. 复杂系统中涌现形成机理的讨论[D]. 上海大学, 2008.
- [3] Hague A G. What is systems engineering?[J]. IEEE Aerospace & Electronic Systems Magazine, 2002, 15(10): 9–10.
- [4] 杜玠. 论系统工程[J]. 国防科技大学学报, 1981(02): 4–23.
- [5] Jeff, Estefan. MBSE Methodology Survey[J]. Insight, 2015.
- [6] Ying, Li, KeWei, et al. Model-based system configuration approach for Internetwork[J]. Science China Information Sciences, 2013.
- [7] SEBok authors. INCOSE Systems Engineering Vision 2020[J]. 2013.
- [8] Mann C J H. A Practical Guide to SysML: The Systems Modeling Language[J]. Kybernetes, 2009, 38(1/2): 989–994.
- [9] Feiler P H, Gluch D P, Hudak J, et al. The Architecture Analysis & Design Language (AADL): An Introduction[R]. Carnegie-Mellon Univ Pittsburgh PA Software Engineering Inst, 2006(February).
- [10] Chinosi M, Trombetta A. BPMN: An introduction to the standard[J]. Computer Standards & Interfaces, 2012, 34(1): 124–134.
- [11] Fritzson P . Model-based development of sustainable cyber-physical systems including requirement formalization using the openModelica model-based development toolkit[C]. Proceedings of the 10th European Conference. ACM, 2016.
- [12] Torre D, Labiche Y, Genero M, et al. A systematic identification of consistency rules for UML diagrams[J]. Journal of Systems and Software, 2018, 144(June): 121–142.
- [13] Pardillo J, Cachero C. Domain-specific language modelling with UML profiles by decoupling abstract and concrete syntaxes[J]. Journal of Systems & Software, 2010, 83(12):2591-2606.
- [14] Frankel D, Parodi J. The MDA Journal: Model Driven Architecture Straight From The Masters[M]. 2004.
- [15] Feiler P H, Gluch D P. Model-Based Engineering with AADL: An Introduction to the SAE Architecture Analysis & Design Language[M]. Addison-Wesley, 2012.
- [16] Hause M, Brookshier D, Bleakley G. UPDM - Unified Profile for DoDAF/MODAF[J]. 2010.
- [17] Filho F F B D S, Castelo Branco R M G, Silva N C V, et al. [Brazilian Geophysical Society 14th International Congress of the Brazilian Geophysical Society & EXPOGEF, Rio de Janeiro, Brazil, 3-6 August 2015 - Rio de Janeiro (03 August to 06 August)] 14th International Congress of the Brazilian Geophysical Society [J]. : 761–764.
- [18] Estefan, Jeff. MBSE Methodology Survey[J]. Insight, 2009, 12(4): 16–18.
- [19] Hoffmann H-P. Systems Engineering Best Practices with the Rational Solution for Systems and Software Engineering[J]. Deskbook Release 3.1.2, 2011, 1: 134.
- [20] Reinhard Patels. OOSEM—a complete development process for O-O software[C]. Eleventh International Conference on Technology of Object-oriented Languages &

- Systems. Prentice-Hall, Inc. 1993.
- [21] authors Seb. A Survey of Model-Based Systems Engineering (MBSE) Methodologies[J]. 2013.
- [22] Bonnet S, Voirin J L, Exertier D, et al. Modeling system modes, states, configurations with Arcadia and Capella: method and tool perspectives[J]. IncoSE International Symposium, 2017, 27(1): 548–562.
- [23] Warren, Smith. Essential Model-Based Systems Engineering — Applied and Practical[J]. INCOSE International Symposium, 2013.
- [24] James, Long. Model-Based Systems Engineering For Project Success: The Complete Process[J]. IncoSE International Symposium, 2010.
- [25] Cantor M. Rational Unified Process for Systems Engineering: Part 1[J]. Journal of Systems Architecture - JSA, 2003.
- [26] Dori D. Object-Process Methodology – A Holistic Systems Paradigm[M]. Springer Berlin Heidelberg, 2002.
- [27] Nag S. Satellite Constellation Mission Design using Model-Based Systems Engineering and Observing System Simulation Experiments[J]. 2014.
- [28] Deursen A V, Klint P. Little Languages: Little Maintenance?[J]. 2015.
- [29] Hui L I U, ZhiYi M A, WeiZhong S, et al. Progress of Research on Metamodeling 元建模技术研究进展[J]. 软件学报, 2008, 19(6): 1317–1327.
- [30] Neema S, Bapty T, Sztipanovits J. Multi-Model Language Suite for Cyber Physical Systems Final Report[J]. 2013.
- [31] Sorenson P, Tremblay J-P, McAllister A. The Metaview System for Many Specification Environments[J]. Software, IEEE, 1988, 5: 30–38.
- [32] Jürgen Ebert, Roger Süttenbach, Ingar Uhe. Meta-CASE in practice: A CASE for KOGGE[M]. Advanced Information Systems Engineering. Springer Berlin Heidelberg, 1997.
- [33] Lara J, Vangheluwe H. Using AToM3 as a Meta-Case Tool[J]. 4th International Conference on Enterprise Information Systems, 2002, 2: 642–649.
- [34] OMG Object Management Group. Meta Object Facility (MOF) Core Specification v2. 0.. 2006[J]. <https://www.omg.org/>, 2006(January).
- [35] Bichler L. Tool support for generating implementations of MOF-based modeling languages[J]. 2003.
- [36] Costagliola G, Deufemia V, Ferrucci F, et al. Constructing Meta-CASE Workbenches by Exploiting Visual Language Generators[J]. IEEE Transactions on Software Engineering, 2006, 32(3): p.156-175.
- [37] Ledeczi A, Bakay Á, Maróti M, et al. Composing Domain-specific Design Environments[J]. Computer, 2001, 34: 44–51.
- [38] Hause M, Buitelaar J, Marcel V D V, et al. The Use of MBSE in Infrastructure Projects -- An MBSE Challenge Team Paper[J]. IncoSE International Symposium, 2015, 25(1): 371–387.
- [39] Risto Pohjonen. Metamodeling Made Easy - MetaEdit+ (Tool Demonstration) [C]. Generative Programming & Component Engineering, International Conference, Gpce, Tallinn, Estonia, September 29-october. DBLP, 2005.
- [40] Kelly S, Tolvanen J P. Domain-Specific Modeling: Enabling Full Code Generation[M]. IEEE Xplore, 2008.

- [41]Kindler E. Model-Based Software Engineering and Process-Aware Information Systems[J]. T. Petri Nets and Other Models of Concurrency, 2009, 2: 27–45.
- [42]Spiby P. AP233 Systems Engineering and Design: Overview[J]. INCOSE International Symposium, John Wiley & Sons, Ltd, 2008, 18(1): 1863–1940.
- [43]胡峻豪, 冯雷, 朱睿等. Research and Implementation of System Design Application Framework for MBSE%支持MBSE的系统全过程设计应用框架研究与实现[J]. 航空制造技术, 2016, 000(13): 105–109.
- [44]Oliver D W. AP233'INCOSE Status Report[J]. INSIGHT, John Wiley & Sons, Ltd, 2002, 5(3): 21–22.
- [45]Vanderperren Y, Dehaene W. From UML/SysML to Matlab/Simulink: current state and future perspectives.[M]. 2006.
- [46]Kaslow D , Soremekun G , Kim H , et al. Integrated model-based systems engineering (MBSE) applied to the Simulation of a CubeSat mission[C].IEEE Aerospace Conference. IEEE, 2014.
- [47]Fabrice Bouquet, JeanMarie Gauthier, Ahmed Hammad, et al. Transformation of SysML structure diagrams to VHDL-AMS[C]. Workshop on Design. IEEE, 2012.
- [48]Verries J, Sahraoui A. Case Study On SYSML and VHDL-AMS for Designing and Validating Systems[J]. Lecture Notes in Engineering & Computer Science, 2013, 2207(1).
- [49]Raslan W , Sameh A . Mapping SysML to SystemC [C] . Forum on Specification & Design Languages, Fdl, September, Barcelona, Spain. DBLP, 2007.
- [50]W. Raslan, A. Sameh. System-Level Modeling and Design Using SysML and SystemC[C].International Symposium on Integrated Circuits. IEEE, 2008.
- [51]Paredis C J J , Bernard Y , Burkhart R M , et al. 5.5.1 An Overview of the SysML - Modelica Transformation Specification[J]. incose international symposium, 2010.
- [52]October C P is currently leading a working group on the S T S and serving on the S R T F O th. ICES Guest Lecture: SysML-Modelica Transformation Specification - an overview[J]. Kungliga Tekniska Högskolan, .
- [53]俞宣孟. 本体论研究[M]. 上海人民出版社, 2005.
- [54]van Ruijven L. Ontology and Model Based Systems Engineering.[M]. Procedia Computer Science, 2011, 8.
- [55]Walter T , Parreiras F S , Staab S . OntoDSL: An Ontology-Based Framework for Domain-Specific Languages[C].International Conference on Model Driven Engineering Languages & Systems. Springer Berlin Heidelberg, 2010.
- [56]Panov P , Soldatova L N , Dzeroski S . Towards an Ontology of Data Mining Investigations[C].International Conference on Discovery Science. Springer-Verlag, 2009.
- [57]Hasnat, Abul M, Habib , et al.A high performance domain specific OCR for Bangla script[M]. Springer Netherlands, 2008.
- [58]Neuendorf D. Review of MagicDraw UML® 11.5 Professional Edition[J]. Journal of Object Technology, 2006, 5(7): 115–118.
- [59]Gruber T R. A translation approach to portable ontology specifications[J]. Knowledge Acquisition, 1993, 5(2): 199–220.

- [60] Claus, Ballegaard, Nielsen等. Systems of Systems Engineering: Basic Concepts, Model-Based Techniques, and Research Directions[J]. *Acm Computing Surveys*, 2015.
- [61] Grogan P T, De Weck O L. Collaboration and complexity: an experiment on the effect of multi-actor coupled design[J]. *Research in Engineering Design*, 2016, 27(3): 221–235.
- [62] 马向玲, 雷宇曜. 有人/无人机协同作战关键技术[J]. *火力与指挥控制*, (1): 80–83.

攻读学位期间发表论文与研究成果清单

学术论文:

[1] **Wang H** , Wang G , Lu J , et al. Ontology Supporting Model-Based Systems Engineering Based on a GOPPRR Approach[M]// New Knowledge in Information Systems and Technologies. 2019.

[2] **Wang H** , Wang G , Lu J , et al. 基于模型的需求驱动无人平台设计方法. (审稿中)

致谢

在论文进展到这一步时才开始感受到毕业就在眼前了。也就是在这一刻，才发现在北理工仍有太多的不舍与留恋。从 2013 年本科入学到现在，我和北理工已经有 7 年的缘分，这种缘分也将在往后的日子中继续下去。回首往日，硕士生涯中取得的微不足道的成功离不开各位老师、同学、家人和朋友的帮助。借着这个机会，向一直帮助我、鼓励我的各位表示最衷心的感谢。

首先，我要感谢我的导师王国新老师，本科到硕士，王老师在 7 年的学习生涯中给了我太多的教导和鼓励，除了在科研领域上给我高屋建瓴的指导，在生活上也给我无微不至的关系。除此以外，王老师在工作科研中严谨的学术态度和不断追求细节的精神也深深影响了我，帮助我成为更好的自己。万分感谢王老师对我的包容和悉心教导，谢谢您！

其次，感谢帮助我的各位师兄师姐，师弟师妹。感谢鲁金直博士在学术领域中给我的诸多帮助，从入门到现在的略知一二，都离不开鲁博士对我的悉心教导；感谢马君达师兄、黄思翰师兄、明振军师兄、牛红伟师兄、贾良跃师兄、鲍衍地师兄对我的帮助，你们在科研生活上给了我很大的帮助，感谢你们！还要感谢课题组的各位学弟学妹，郭江敏师弟和余士严师弟在科研上也给了我很大帮助，谢谢你们！

最后还要感谢我的父母和亲人，感谢你们一直以来的鼓励和支持，希望你们永远身体健康！

在校七载，需要感谢的人太多太多，只希望各位诸事顺心，工作顺利，我将以各位为榜样，不负韶华，为社会做出自己的贡献。